Pension Preparedness and Financial Security: An Analysis of Indian Pension Subscriber Data

Data Analytics Final Project

Domain: Finance / Pension & Retirement Planning
Dataset: India Pension Scheme Dataset
Dataset Type: Synthetic Indian pension subscriber records
Source: Kaggle

1. Problem Statement

The Problem

Pension subscribers may differ considerably in their ability to prepare financially for retirement. Differences in income, contribution levels, investment allocation, accumulated pension fund value, retirement age, and expected pension income may influence the level of financial preparedness among subscribers.

However, meaningful analysis of these factors requires reliable and consistent data. Pension-related datasets may contain data-quality issues such as missing values, duplicate records, inconsistent categories, incorrect data types, invalid numerical values, and logical inconsistencies between related variables. If these issues are not identified and addressed, they may affect the accuracy and reliability of the analysis.

What This Project Aims to Solve

This project aims to address two related problems:

1. **Data reliability problem:**
   How can the current pension subscriber dataset be explored, cleaned, validated, and transformed into a reliable dataset suitable for further analysis?

2. **Pension preparedness problem:**
   After obtaining a reliable dataset, what demographic, employment, contribution, investment, and retirement-related factors are associated with stronger or weaker pension preparedness among subscribers?

The analysis will use exploratory data analysis, statistical summaries, and visualizations to identify patterns, relationships, differences, and potential areas of financial vulnerability within the dataset.

2. Dataset Description

The dataset used for this project is the India Pension Scheme Dataset obtained from Kaggle. It is a synthetic dataset containing pension subscriber records designed to represent demographic, employment, financial, investment, pension, and retirement-related information.

The current dataset has been manually prepared from the original dataset by removing certain unnecessary rows and columns before the analysis. Therefore, the structure and dimensions reported in this project will be based on the current version of the dataset used for analysis.

The dataset is expected to contain a combination of numerical and categorical variables related to pension subscribers. These variables will be examined during the data exploration stage to understand their structure, data types, distributions, and suitability for further analysis.

The dataset is intentionally treated as an initially uncleaned analytical dataset. Potential data-quality issues such as missing values, duplicate records, inconsistent formatting, incorrect data types, invalid values, and logical inconsistencies will be investigated before conducting the main exploratory analysis.

Dataset Nature

The dataset is synthetic and does not represent actual individuals or real pension subscriber records. Therefore, the findings from this project will be interpreted as analytical patterns within the dataset and will not be generalized to the entire Indian population or used for individual pension or financial decisions.

Source

Source: Kaggle
Dataset: India Pension Scheme Dataset
Domain: Finance / Pension & Retirement Planning

3. Import Required Libraries

Before loading and analysing the dataset, the required Python libraries are imported.

Pandas is used for loading, cleaning, manipulating, and analysing the dataset. NumPy is used for numerical calculations and data processing. Matplotlib and Seaborn are used to create statistical and exploratory visualizations.

These libraries provide the basic tools required for the data exploration, data cleaning, statistical analysis, and visualization stages of the project.

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

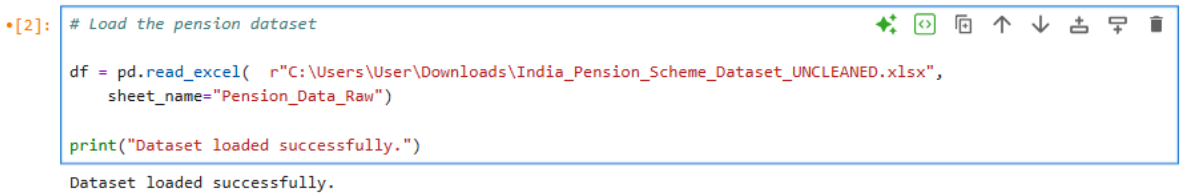
4. Loading the Dataset

The current India Pension Scheme Dataset is loaded into Python using Pandas.

The dataset is stored in an Excel workbook, and
the Pension_Data_Raw worksheet is selected as the working dataset for this project. This worksheet contains the pension subscriber records that will be examined throughout the analysis.

The current version of the file is used as the starting point for this project because unnecessary rows and columns have already been manually removed during dataset preparation. Therefore, the dimensions, variables, and characteristics of the dataset will be determined from the current file rather than being assumed from the original dataset description.

At this stage, no further cleaning or transformation is performed. The dataset is loaded in its current form so that its structure and data-quality issues can first be examined objectively.



5. Data Exploration

After loading the dataset, the next stage is to explore its structure and contents before performing any data cleaning.

Data exploration helps us understand the variables available in the dataset, how the values are represented, and whether any obvious data-quality issues can be observed. This initial examination provides a basis for deciding which cleaning and preprocessing steps may be required.

The exploration will include viewing the first and last records, examining the dataset information and dimensions, checking column names and data types, obtaining descriptive statistics, and examining the frequency of categorical values.

5.1 Viewing the First Five Records

The first five records of the dataset are displayed using the head() function.

This provides an initial view of the dataset and helps us understand how individual subscriber records are organized across the available variables. It also allows us to observe the types of values stored in the columns and identify any obvious formatting or data-quality issues that may require further investigation.

```
[3]: df.head()
```

[3]:

	Gender	Date_of_Birth	Age	Marital_Status	State	City	Aadhaar_Linked	PAN_Linked	Occupation_Type	Sector	...	Years_to_Retirement	Nominee_N
0	female	1969/01/26	NaN	NaN	uttar pradesh	Varanasi	Yes	Yes	Professional	All Citizen Model	...	3.0	Hemang
1	female	09/04/1998	28	Divorced	Karnataka	Hubli	Yes	Yes	Government Employee	Central/State Government	...	32.0	Devansh
2	female	1967-06-05	59	Single	Uttar Pradesh	Varanasi	Yes	yes	Private Sector Employee	All Citizen Model	...	1.0	Logan
3	female	1958-11-03	68	Married	Haryana	Gurugram	Yes	Yes	Factory Worker	Organized Private Sector	...	0.0	Gagan L
4	female	1982-07-18	44	Single	Uttar Pradesh	Kanpur	Yes	No	Private Sector Employee	Organized Private Sector	...	16.0	Siddharth J

5 rows × 38 columns

Initial Observations from the First Five Records

The first five records provide an initial understanding of the structure and format of the pension subscriber data. The current dataset contains 38 columns, covering demographic, employment, pension, financial, and other subscriber-related information.

A preliminary examination also indicates that some data-quality issues may be present. For example, missing values can be observed in fields such as Age and Marital Status. Date of birth values also appear to use different date formats, while some categorical fields show differences in capitalization, such as variations in State and PAN-linked status values.

These observations are only preliminary findings based on the first five records. They will be investigated systematically during the data-quality assessment and cleaning stages rather than being modified at this stage.

5.2 Viewing the Last Five Records

The last five records of the dataset are displayed using the tail() function.

Examining the end of the dataset provides a complementary view to the first five records and helps determine whether the data structure remains consistent throughout the dataset. It may also reveal unusual values, formatting differences, missing values, or other potential issues that are not visible at the beginning of the dataset.

```
[4]: df.tail()
```

[4]:

	Gender	Date_of_Birth	Age	Marital_Status	State	City	Aadhaar_Linked	PAN_Linked	Occupation_Type	Sector	...	Years_to_Retirement	Nomi
5548	Male	1994-01-08	32	Married	West Bengal	Howrah	NaN	Yes	Government Employee	Central/State Government	...	28.0	
5549	Male	1958-04-02	68 years	Married	Tamil Nadu	Madurai	NaN	Yes	Private Sector Employee	All Citizen Model	...	0.0	
5550	Female	1969-01-25	57	Single	Jharkhand	Jamshedpur	NaN	1	Government Employee	Central/State Government	...	3.0	
5551	Female	NaN	29	Married	Tamil Nadu	Coimbatore	NaN	Yes	Private Sector Employee	All Citizen Model	...	31.0	T
5552	Female	1991-06-04	NaN	Married	Delhi	New Delhi	NaN	NaN	Factory Worker	Organized Private Sector	...	25.0	

5 rows × 38 columns

Initial Observations from the Last Five Records

The last five records provide an additional view of the dataset and show that some data-quality issues are also present near the end of the dataset.

Missing values can be observed in fields such as Aadhaar Linked and Date of Birth. In addition, some variables appear to contain inconsistent representations. For example, the Age field includes a value represented as "68 years", while other records contain numeric-looking age values. The PAN Linked field also contains different representations, including "Yes" and "1".

These observations indicate that data-quality checks will be required during the preprocessing stage. However, no values are modified at this stage because the purpose of the current section is to explore the dataset before cleaning.

5.3 Understanding Dataset Information

The info() function is used to obtain a concise summary of the dataset structure.

This summary provides information about the number of records and columns, the number of non-null values in each column, and the data type assigned to each variable. Examining this information is important for identifying missing values and determining whether the variables have been stored using appropriate data types.

The output will also help guide the subsequent data-cleaning process. For example, variables expected to contain numerical values can be checked to determine whether they have been interpreted as numeric data or stored as text/object values.

At this stage, the information is used only for assessment and no changes are made to the dataset.

```
[5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5553 entries, 0 to 5552
Data columns (total 38 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   Gender                                    5498 non-null   object
 1   Date_of_Birth                             5441 non-null   object
 2   Age                                       5441 non-null   object
 3   Marital_Status                            5499 non-null   object
 4   State                                     5552 non-null   object
 5   City                                      5497 non-null   object
 6   Aadhaar_Linked                            5497 non-null   object
 7    PAN_Linked                               5497 non-null   object
 8   Occupation_Type                           5497 non-null   object
 9   Sector                                    5497 non-null   object
 10  Pension_Scheme_Type                       5552 non-null   object
 11  Date_of_Joining                           5469 non-null   object
 12  Years_of_Contribution                     5469 non-null   object
 13  Employer_Name                             5453 non-null   object
 14  Annual_Income_INR                         5441 non-null   object
 15  Monthly_Contribution_INR                  5469 non-null   object
 16  Employer_Monthly_Contribution_INR         5469 non-null   float64
 17  Total_Contribution_Till_Date_INR          5469 non-null   object
 18  Investment_Choice                         5486 non-null   object
 19  Pension_Fund_Manager                      5486 non-null   object
 20  Asset_Allocation_Equity_E_Percent         5537 non-null   float64
 21  Asset_Allocation_CorporateDebt_C_Percent  5534 non-null   float64
 22  Asset_Allocation_GovtSecurities_G_Percent 5543 non-null   float64
 23  Asset_Allocation_Alternative_A_Percent    5539 non-null   float64
 24  Current_NAV_INR                           5486 non-null   float64
 25  Total_Units_Held                          5486 non-null   float64
 26  Current_Fund_Value_INR                    5486 non-null   float64
 27  Expected_Monthly_Pension_INR              5486 non-null   float64
 28  Years_to_Retirement                       5497 non-null   float64
 29  Nominee_Name                              5452 non-null   object
 30  Nominee_Relationship                      5486 non-null   object
 31  Account_Status                            5497 non-null   object
 32  KYC_Status                                5497 non-null   object
 33  Bank_Name                                 5486 non-null   object
 34  IFSC_Code                                 5497 non-null   object
 35  Tax_Benefit_Section_Claimed               5483 non-null   object
 36  Withdrawal_Status                         5497 non-null   object
 37  Last_Transaction_Date                     5469 non-null   object
dtypes: float64(10), object(28)
memory usage: 1.6+ MB
```

Observations from Dataset Information

The info() output shows that the current dataset contains 38 columns with a combination of categorical, textual, and numerical variables. The dataset contains 28 columns with an object data type and 10 columns with a float64 data type.

Several columns contain missing values, as indicated by the difference between the total number of records and the non-null count. Missing observations can be seen in variables such as Date of Birth, Age, Marital Status, City, Employer Name, Annual Income, and Current Fund Value.

The output also identifies potential data-type issues that will require further investigation during preprocessing. Variables such as Age, Years of Contribution, Annual Income, Monthly Contribution, and Total Contribution Till Date are currently stored as object values even though they represent numerical information. Similarly, Date of Birth, Date of Joining, and Last Transaction Date are currently stored as object values and may require conversion to appropriate date formats.

A formatting issue is also visible in the column name PAN_Linked, which appears to contain a leading space. This will be considered during the data-cleaning stage.

These findings provide an initial assessment of the dataset's data-quality characteristics. No modifications are made at this stage because the purpose of the exploration phase is to understand the raw dataset before cleaning.

5.4 Checking Dataset Dimensions

The dimensions of the dataset are examined using the shape attribute.

The shape attribute returns the number of rows and columns in the dataset. This provides an exact measure of the current dataset size after the manual removal of unnecessary rows and columns.

Confirming the dimensions at this stage is important because the current dataset may differ from the original source dataset. The resulting number of records and variables will therefore be used throughout this project when describing the dataset.

```
[6]: df.shape

[6]: (5553, 38)
```

Result of Dataset Dimension Check

The current dataset contains **5,553 records and 38 columns**. This represents the dataset after the manual removal of unnecessary rows and columns during the preparation stage.

The dataset therefore provides 5,553 subscriber-level observations across 38 variables covering different aspects of demographic, employment, pension, financial, investment, and retirement-related information.

These dimensions will be used as the basis for describing the current dataset throughout the remainder of the project.

5.5 Examining Column Names

The column names of the dataset are examined using the columns attribute.

Reviewing the column names helps us understand the variables available for analysis and provides an opportunity to identify naming inconsistencies, unnecessary spaces, unusual characters, or other formatting issues.

This step is particularly important before data cleaning because column names are frequently used in Python code for data selection, transformation, analysis, and visualization. Any inconsistencies in naming may therefore affect subsequent analytical operations.

At this stage, the column names are only examined and no changes are made to them.

```
[7]: df.columns

[7]: Index(['Gender', 'Date_of_Birth', 'Age', 'Marital_Status', 'State', 'City',
       'Aadhaar_Linked', ' PAN_Linked', 'Occupation_Type', 'Sector',
       'Pension_Scheme_Type', 'Date_of_Joining', 'Years_of_Contribution',
       'Employer_Name', 'Annual_Income_INR', 'Monthly_Contribution_INR',
       'Employer_Monthly_Contribution_INR', 'Total_Contribution_Till_Date_INR',
       'Investment_Choice', 'Pension_Fund_Manager',
       'Asset_Allocation_Equity_E_Percent',
       'Asset_Allocation_CorporateDebt_C_Percent',
       'Asset_Allocation_GovtSecurities_G_Percent',
       'Asset_Allocation_Alternative_A_Percent', 'Current_NAV_INR',
       'Total_Units_Held', 'Current_Fund_Value_INR',
       'Expected_Monthly_Pension_INR', 'Years_to_Retirement', 'Nominee_Name',
       'Nominee_Relationship', 'Account_Status', 'KYC_Status', 'Bank_Name',
       'IFSC_Code', 'Tax_Benefit_Section_Claimed', 'Withdrawal_Status',
       'Last_Transaction_Date'],
      dtype='object')
```

Observations from the Column Names

The dataset contains **38 variables** representing different aspects of pension subscriber information, including demographic details, employment information, pension scheme characteristics, financial contributions, investment allocation, retirement information, nominee details, and account status.

A review of the column names indicates that most variables follow a consistent naming convention using underscores to separate words. However, one column (PAN_Linked) appears to contain a leading whitespace character. Such formatting inconsistencies may create difficulties when selecting or manipulating variables during analysis.

The column names will therefore be reviewed and standardized during the data-cleaning stage to ensure consistency throughout the project.

5.6 Examining Data Types

The data types of all variables are examined using the dtypes attribute.

Understanding the data type of each variable is an essential part of exploratory data analysis because it determines how the data can be processed, analysed, and visualized. Variables containing numerical values should ideally be stored using appropriate numeric data types, while dates should be stored using datetime formats.

Reviewing the data types also helps identify variables that may require conversion during the data-cleaning stage due to incorrect or inconsistent storage formats.

```
[8]: df.dtypes

[8]: Gender                                  object
     Date_of_Birth                           object
     Age                                     object
     Marital_Status                          object
     State                                   object
     City                                    object
     Aadhaar_Linked                          object
      PAN_Linked                             object
     Occupation_Type                         object
     Sector                                  object
     Pension_Scheme_Type                     object
     Date_of_Joining                         object
     Years_of_Contribution                   object
     Employer_Name                           object
     Annual_Income_INR                       object
     Monthly_Contribution_INR                object
     Employer_Monthly_Contribution_INR      float64
     Total_Contribution_Till_Date_INR        object
     Investment_Choice                       object
     Pension_Fund_Manager                    object
     Asset_Allocation_Equity_E_Percent      float64
     Asset_Allocation_CorporateDebt_C_Percent    float64
     Asset_Allocation_GovtSecurities_G_Percent   float64
     Asset_Allocation_Alternative_A_Percent      float64
     Current_NAV_INR                        float64
     Total_Units_Held                       float64
     Current_Fund_Value_INR                 float64
     Expected_Monthly_Pension_INR           float64
     Years_to_Retirement                    float64
     Nominee_Name                            object
     Nominee_Relationship                    object
     Account_Status                          object
     KYC_Status                              object
     Bank_Name                               object
     IFSC_Code                               object
     Tax_Benefit_Section_Claimed             object
     Withdrawal_Status                       object
     Last_Transaction_Date                   object
     dtype: object
```

Observations from Data Types

The data-type inspection shows that the dataset contains both categorical/textual and numerical variables. However, several variables that represent numerical information are currently stored as object data types. These include Age, Years of Contribution, Annual Income, Monthly Contribution, and Total Contribution Till Date.

The date-related variables, including Date of Birth, Date of Joining, and Last Transaction Date, are also currently stored as object values rather than datetime values.

These observations indicate that some variables may require data-type conversion during the preprocessing stage. However, conversion will only be performed after examining the actual values and identifying the reasons for the current data types.

5.7 Descriptive Statistical Summary

The describe() function is used to obtain a statistical summary of the numerical variables in the dataset.

The summary includes measures such as the count, mean, standard deviation, minimum value, quartiles, and maximum value. These statistics provide an initial understanding of the distribution and range of numerical variables and may also help identify unusually large or small values that require further investigation.

At this stage, the summary reflects the current data types of the dataset. Therefore, variables that are currently stored as object values, even if they represent numerical information, may not appear in the numerical summary. Such variables will be examined and corrected during the preprocessing stage where necessary.

```
[9]: df.describe()
```

[9]:

	Employer_Monthly_Contribution_INR	Asset_Allocation_Equity_E_Percent	Asset_Allocation_CorporateDebt_C_Percent	Asset_Allocation_GovtSecurities_G_Percent	Asse
count	5469.000000	5537.000000	5534.000000	5543.000000	
mean	4579.846407	36.667329	32.551319	23.920440	
std	6097.142076	16.115804	8.530433	14.557951	
min	0.000000	-50.000000	-60.000000	-45.000000	
25%	0.000000	15.000000	30.000000	15.000000	
50%	1250.000000	35.000000	35.000000	25.000000	
75%	8710.000000	50.000000	35.000000	45.000000	
max	20960.000000	77.000000	77.000000	66.000000	

Observations from Descriptive Statistics

The descriptive statistical summary provides an initial overview of the numerical variables currently recognized by the dataset.

The results indicate considerable variation across several financial and investment-related variables. For example, Current Fund Value and Expected Monthly Pension show substantial differences between their lower quartiles,

medians, upper quartiles, and maximum values, suggesting that these variables may have highly dispersed or skewed distributions.

The summary also identifies several values that require further investigation. Negative values are present in variables such as asset allocation percentages, Current NAV, Total Units Held, Expected Monthly Pension, and Years to Retirement. Some of these values may represent invalid observations because they are not logically meaningful for the corresponding variables.

These observations do not immediately imply that the records should be removed. The potentially invalid or extreme values will be examined during the data-quality assessment and outlier detection stages, where appropriate validation rules and statistical methods will be applied.

It should also be noted that several numerical variables are currently stored as object data types and therefore are not included in this numerical summary. These variables will be examined during the data-cleaning stage.

5.8 Examining Categorical Value Frequencies

Frequency distributions are examined using the value_counts() function to understand how categorical values are distributed within the dataset.

This analysis helps identify the number of observations belonging to each category and can reveal uncommon categories, inconsistent labels, unexpected values, or possible missing-value representations.

5.8.1 Gender Frequency Analysis

Gender is examined first as an example of a categorical demographic variable. Similar frequency checks will be performed for other important categorical variables during the data exploration and data-quality assessment stages.

```
[10]: df["Gender"].value_counts(dropna=False)
```

```
[10]: Gender
      Male       3021
      Female     2139
      Other        60
      NaN          55
      MALE         48
      male         38
       male        32
      Female       28
      M            28
      Male         24
      female       22
       female      20
      F            18
      FEMALE       15
      other         3
      O             1
      OTHER         1
      Name: count, dtype: int64
```

Observations from Gender Frequency Analysis

The frequency analysis of the Gender variable shows that the dataset contains three main gender categories: Male, Female, and Other. However, these categories are represented using multiple variations in capitalization and formatting.

For example, Male appears as Male, MALE, male, and M, while Female appears as Female, FEMALE, female, and F. Similarly, Other appears in different forms including Other, other, OTHER, and O.

There are also 55 missing observations in the Gender variable.

These variations indicate an inconsistency in categorical data representation. During the data-cleaning stage, these values will need to be standardized into consistent categories so that frequency calculations, grouping, and visualizations are accurate.

At this stage, the original values are retained because the purpose of the exploration phase is to identify data-quality issues before applying any corrections.

5.8.2 Occupation Frequency Analysis

```
[11]: df["Occupation_Type"].value_counts(dropna=False)
```

```
[11]: Occupation_Type
Private Sector Employee      1686
Government Employee          1246
Factory Worker                473
Self-Employed                 423
Professional                  386
Domestic Worker               234
Driver                        220
Vendor                        219
Daily Wage Worker             208
Farmer                        180
NaN                            56
GOVERNMENT EMPLOYEE            20
private sector employee        20
Government Employee            20
PRIVATE SECTOR EMPLOYEE        19
Private Sector Employee        16
 Private Sector Employee       14
 Government Employee           11
government employee             8
self-employed                   7
driver                          5
Self-Employed                   5
domestic worker                 5
Domestic Worker                 5
 Professional                   5
 Self-Employed                  5
FACTORY WORKER                  4
professional                    4
SELF-EMPLOYED                   4
Daily Wage Worker               4
factory worker                  4
PROFESSIONAL                    4
Factory Worker                  3
 Driver                         3
vendor                          3
Professional                    3
 Farmer                         3
VENDOR                          3
 Domestic Worker                3
Farmer                          2
FARMER                          2
DRIVER                          2
 Factory Worker                 2
Driver                          1
daily wage worker               1
 Daily Wage Worker              1
```

Observations from Occupation Frequency Analysis

The frequency analysis of the Occupation Type variable shows that the dataset contains several major occupational categories, with Private Sector Employee and Government Employee representing the largest groups.

However, the analysis also reveals inconsistencies in the representation of occupation categories. The same occupation appears in different capitalization formats, such as Government Employee, GOVERNMENT EMPLOYEE, and government employee. Similar variations can be observed for Private Sector Employee, Self-Employed, Factory Worker, Professional, Driver, Vendor, Farmer, and other categories.

Some occupation values also contain leading spaces, resulting in additional representations of categories that should logically belong to the same group. The variable also contains 56 missing observations.

These inconsistencies could lead to incorrect frequency counts and misleading group-based analysis if they are not addressed. Therefore, the Occupation Type variable will require categorical standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

5.8.3 Pension Scheme Frequency Analysis

```
[12]: df["Pension_Scheme_Type"].value_counts(dropna=False)
```

```
[12]: Pension_Scheme_Type
      NPS - All Citizen                  1187
      Atal Pension Yojana (APY)          1070
      Employees' Pension Scheme (EPS-95)  994
      NPS - Government Sector             910
      NPS - Corporate Sector              769
      Unified Pension Scheme (UPS)        344
      NPS-Government                       32
      NPS Corporate                        30
      NPS-AllCitizen                       27
      Atal Pension Yojana                  25
      APY                                  23
      EPS                                  20
      EPS-95                               19
      NPS-Corp                             19
      nps - all citizen                    17
      EPS 95                               17
      NPS                                  16
      NPS Govt                             13
      Unified Pension Scheme               12
      UPS                                   8
      NaN                                   1
      Name: count, dtype: int64
```

Observations from Pension Scheme Frequency Analysis

The frequency analysis of the Pension Scheme Type variable shows that the dataset contains several major pension schemes, including NPS - All Citizen, Atal Pension Yojana (APY), Employees' Pension Scheme (EPS-95), NPS - Government Sector, NPS - Corporate Sector, and Unified Pension Scheme (UPS).

The analysis also reveals multiple representations of the same pension schemes. For example, government-sector NPS appears as NPS - Government Sector, NPS-Government, and NPS Govt, while corporate NPS appears as NPS - Corporate Sector, NPS Corporate, and NPS-Corp. Similar variations are present for APY, EPS, NPS - All Citizen, and UPS.

These variations indicate that the categorical values are not consistently standardized. If these values were used directly for grouping or visualization, the same pension scheme could be treated as several different categories and produce misleading results.

The variable contains one missing observation. The inconsistent scheme labels will be reviewed and standardized during the data-cleaning stage.

5.8.4 Examining Account Status Frequencies

The frequency distribution of Account Status is examined to understand the current status of subscriber accounts in the dataset.

This variable may be useful in later analysis when comparing pension-related characteristics across different account statuses. Examining its frequency distribution at this stage also helps identify missing values and inconsistencies in categorical labels before the cleaning process begins.

```
[13]: df["Account_Status"].value_counts(dropna=False)
```

```
[13]: Account_Status
      Active      4218
      Inactive     670
      Dormant      288
      Frozen       155
      active        73
      ACTIVE        56
      NaN           56
      inactive      13
      INACTIVE      13
      dormant        7
      DORMANT        3
      FROZEN         1
      Name: count, dtype: int64
```

Observations from Account Status Frequency Analysis

The frequency analysis of the Account Status variable shows that the dataset contains four main account-status categories: Active, Inactive, Dormant, and Frozen.

However, the same categories are represented using different capitalization formats. For example, Active appears as Active, active, and ACTIVE, while similar variations are present for Inactive, Dormant, and Frozen.

The variable also contains 56 missing observations.

These inconsistencies could cause the same account status to be counted as separate categories during grouping and visualization. Therefore, the Account Status variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained because the purpose of the exploration phase is to identify data-quality issues before applying corrections.

5.8.5 Examining Marital Status Frequencies

The frequency distribution of the Marital Status variable is examined using the value_counts() function.

Marital status is an important demographic variable that may be relevant when analysing pension preparedness and retirement-related financial characteristics. Examining its frequency distribution helps us understand the representation of different marital-status categories within the dataset.

The analysis also helps identify missing observations, inconsistent labels, differences in capitalization, or other formatting variations that may need to be addressed during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made. The purpose is to understand the raw categorical data before applying any cleaning or standardization.

```
[14]: df["Marital_Status"].value_counts(dropna=False)
```

```
[14]: Marital_Status
Married     3184
Single      1504
Widowed      355
Divorced     148
NaN           54
Mar.          46
MARRIED       45
M             42
married       39
SINGLE        31
Unknown       30
S             26
single        23
W             10
widowed       10
divorced       4
D              2
Name: count, dtype: int64
```

Observations from Marital Status Frequency Analysis

The frequency analysis of the Marital Status variable shows that Married and Single are the two most common categories, followed by Widowed and Divorced.

However, the variable contains multiple representations of the same marital-status categories. For example, Married appears as Married, MARRIED, married, and M. Similarly, Single appears as Single, SINGLE, single, and S, while Widowed and Divorced also appear in abbreviated and differently capitalized forms.

The variable also contains an Unknown category with 30 observations and 54 missing observations.

These inconsistencies may cause the same marital-status category to be treated as separate groups during analysis. Therefore, the values will need to be standardized during the data-cleaning stage. The treatment of the Unknown and missing values will also be considered during preprocessing based on the nature of the variable and the overall data-quality assessment.

At this stage, no modifications are made to the original values.

5.8.6 Examining Aadhaar Linkage Status

The frequency distribution of the Aadhaar Linked variable is examined to understand the linkage status of subscriber accounts.

This variable is relevant to the overall account and KYC profile of pension subscribers. Examining its frequency distribution also helps identify missing observations and inconsistent representations of binary or categorical responses.

The results will be used to determine whether the variable requires standardization during the data-cleaning stage.

```
[15]: df["Aadhaar_Linked"].value_counts(dropna=False)
```

```
[15]: Aadhaar_Linked
      Yes      4712
      No        341
      YES        77
      1          72
      true       71
      TRUE       67
      Y          64
      yes        64
      NaN        56
      N           7
      0           7
      false       5
      no          4
      NO          4
      FALSE       2
      Name: count, dtype: int64
```

Observations from Aadhaar Linkage Frequency Analysis

The frequency analysis of the Aadhaar Linked variable indicates that the intended responses are primarily Yes and No. However, the same responses are represented using several different formats.

For example, positive responses appear as Yes, YES, yes, Y, 1, true, and TRUE. Similarly, negative responses appear as No, NO, no, N, 0, false, and FALSE.

The variable also contains 56 missing observations.

These variations indicate that the Aadhaar Linked variable is not consistently standardized. If these values were used directly in frequency analysis or grouping, the different representations of the same response could be treated as separate categories and produce inaccurate results.

During the data-cleaning stage, these representations will be standardized into consistent categories. The missing observations will also be assessed before deciding how they should be handled.

At this stage, the original values are retained and no modifications are made.

5.8.7 Examining PAN Linkage Status

The frequency distribution of the PAN Linked variable is examined to understand the PAN linkage status of pension subscribers.

This variable is relevant to the subscriber's account and KYC profile. Examining its frequency distribution also helps identify missing values, inconsistent categorical representations, abbreviations, numeric or Boolean-style values, and other formatting issues.

The column name was also observed to contain a leading whitespace character during the column-name examination. The variable is therefore examined using its current raw column name without modifying it at this stage.

```
[16]: df[" PAN_Linked"].value_counts(dropna=False)
```

```
[16]:  PAN_Linked
Yes      4069
No        984
1          71
true       65
Y          60
NaN        56
TRUE       53
YES        49
yes        47
N          25
false      19
no         18
FALSE      16
NO         11
0          10
Name: count, dtype: int64
```

Observations from PAN Linkage Frequency Analysis

The frequency analysis of the PAN Linked variable shows that the intended responses are primarily Yes and No. However, these responses are represented using several different formats.

Positive responses appear as Yes, YES, yes, Y, 1, true, and TRUE. Negative responses appear as No, NO, no, N, 0, false, and FALSE.

The variable also contains 56 missing observations.

In addition to the inconsistent values, the column name itself was previously observed to contain a leading whitespace character. These issues may affect variable selection and produce inaccurate frequency counts or group-based analysis if they are not addressed.

During the data-cleaning stage, the column name will be standardized and the different representations of PAN linkage status will be converted into consistent categories. Missing observations will be assessed before deciding on their treatment.

At this stage, the original values remain unchanged.

5.8.8 Examining Sector Frequencies

The frequency distribution of the Sector variable is examined to understand the representation of different employment sectors within the dataset.

Sector is relevant to pension analysis because employment-sector characteristics may be associated with differences in pension scheme participation, contribution patterns, investment choices, and retirement preparedness.

The frequency analysis also helps identify missing observations and inconsistencies in category names or formatting that may need to be addressed during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

```
[17]: df["Sector"].value_counts(dropna=False)
```

```
[17]: Sector
      All Citizen Model             1192
      Unorganized Sector            1057
      Organized Private Sector       995
      Central/State Government       913
      Corporate                      777
      Central Government             341
      NaN                             56
       Unorganized Sector             17
      Unorganized Sector              15
      organized private sector        14
      ORGANIZED PRIVATE SECTOR        14
      all citizen model               14
      central/state government        13
       Organized Private Sector       11
      ALL CITIZEN MODEL               11
      UNORGANIZED SECTOR              10
      unorganized sector              10
      All Citizen Model                9
       Corporate                       9
       Central/State Government        8
      central government               8
      Organized Private Sector         8
       Central Government              8
      Central/State Government         7
      Corporate                        7
      corporate                        7
       All Citizen Model               6
      CORPORATE                        5
      Central Government               5
      CENTRAL/STATE GOVERNMENT         4
      CENTRAL GOVERNMENT               2
      Name: count, dtype: int64
```

Observations from Sector Frequency Analysis

The frequency analysis of the Sector variable shows that the dataset contains several major sector categories, including All Citizen Model, Unorganized Sector, Organized Private Sector, Central/State Government, Corporate, and Central Government.

However, multiple representations of the same sector are present in the dataset. These differences are mainly caused by variations in capitalization and the presence of leading whitespace. For example, Unorganized Sector appears as Unorganized Sector, Unorganized Sector, unorganized sector, and UNORGANIZED SECTOR. Similar variations are present for Organized Private Sector, All Citizen Model, Corporate, Central/State Government, and Central Government.

The variable also contains 56 missing observations.

These inconsistencies could cause identical sector categories to be treated as separate groups during analysis and visualization. Therefore, the Sector variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

5.8.9 Examining Investment Choice Frequencies

The frequency distribution of the Investment Choice variable is examined to understand the investment preferences represented in the dataset.

Investment choice is an important financial variable for this project because investment allocation may be associated with pension fund value and expected retirement income. Examining its frequency distribution also helps identify missing observations and inconsistencies in category names or formatting.

The results will provide an initial understanding of the investment choices represented in the dataset and will help determine whether categorical standardization is required during preprocessing.

```
[18]: df["Investment_Choice"].value_counts(dropna=False)
```

```
[18]: Investment_Choice
      Auto Choice - Moderate          1358
      Auto Choice - Conservative      1341
      Active Choice                   1321
      Auto Choice - Aggressive        1300
      NaN                               67
      AUTO CHOICE - MODERATE            14
      active choice                     13
      Auto Choice - Moderate            13
      AUTO CHOICE - CONSERVATIVE        13
       Auto Choice - Aggressive         12
      Auto Choice - Conservative        12
      Active Choice                     11
      Auto Choice - Aggressive          10
      auto choice - conservative        10
      auto choice - aggressive          10
      ACTIVE CHOICE                      9
       Auto Choice - Moderate            9
       Active Choice                     9
       Auto Choice - Conservative        9
      AUTO CHOICE - AGGRESSIVE           7
      auto choice - moderate             5
      Name: count, dtype: int64
```

Observations from Investment Choice Frequency Analysis

The frequency analysis of the Investment Choice variable shows four major investment choices: Active Choice, Auto Choice - Conservative, Auto Choice - Moderate, and Auto Choice - Aggressive.

The frequencies of the main categories are relatively similar, indicating that all four investment choices are represented substantially within the dataset. However, the same investment choices are also represented using different capitalization formats and, in some cases, leading whitespace.

For example, Active Choice appears as Active Choice, active choice, ACTIVE CHOICE, and Active Choice. Similar variations are present for the three Auto Choice categories.

The variable contains 67 missing observations.

These inconsistencies may cause the same investment choice to be treated as separate categories during grouping, statistical analysis, and visualization. Therefore, the Investment Choice variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

5.8.10 Examining Pension Fund Manager Frequencies

The frequency distribution of the Pension Fund Manager variable is examined to understand the representation of different fund managers within the dataset.

Pension Fund Manager may be relevant for later analysis of pension fund values, investment patterns, and expected monthly pension. Examining its frequency distribution also helps identify missing observations and inconsistencies in the representation of manager names.

The results will be used to determine whether categorical standardization is required before conducting group-based analysis.

```
[19]: df["Pension_Fund_Manager"].value_counts(dropna=False)
```

```
[19]: Pension_Fund_Manager
Tata Pension Management           563
Kotak Mahindra Pension Fund       560
UTI Retirement Solutions          558
Aditya Birla Sun Life Pension     553
LIC Pension Fund                  541
SBI Pension Funds                 533
Axis Pension Fund Management      527
Max Life Pension Fund             519
HDFC Pension Management           490
ICICI Prudential Pension Fund     476
NaN                                67
 ICICI Prudential Pension Fund      8
 Axis Pension Fund Management       7
SBI PENSION FUNDS                   7
 HDFC Pension Management            7
tata pension management             7
UTI Retirement Solutions            6
SBI Pension Funds                   6
AXIS PENSION FUND MANAGEMENT        6
 Aditya Birla Sun Life Pension      5
lic pension fund                    5
uti retirement solutions            5
Tata Pension Management             5
max life pension fund               5
Max Life Pension Fund               5
MAX LIFE PENSION FUND               5
Aditya Birla Sun Life Pension       5
 Tata Pension Management            5
TATA PENSION MANAGEMENT             4
kotak mahindra pension fund         4
 Max Life Pension Fund              4
hdfc pension management             4
sbi pension funds                   4
 Kotak Mahindra Pension Fund        4
icici prudential pension fund       3
Axis Pension Fund Management        3
 UTI Retirement Solutions           3
KOTAK MAHINDRA PENSION FUND         3
ICICI PRUDENTIAL PENSION FUND       3
aditya birla sun life pension       3
ICICI Prudential Pension Fund       3
LIC PENSION FUND                    3
ADITYA BIRLA SUN LIFE PENSION       3
HDFC PENSION MANAGEMENT             3
 SBI Pension Funds                  3
LIC Pension Fund                    2
 LIC Pension Fund                   2
UTI RETIREMENT SOLUTIONS            2
Kotak Mahindra Pension Fund         2
HDFC Pension Management             2
Name: count, dtype: int64
```

Observations from Pension Fund Manager Frequency Analysis

The frequency analysis of the Pension Fund Manager variable shows that the dataset contains ten major pension fund managers, with relatively substantial representation across the different managers.

The largest observed categories include Tata Pension Management, Kotak Mahindra Pension Fund, UTI Retirement Solutions, Aditya Birla Sun Life Pension, and LIC Pension Fund. However, the same fund manager names are represented in multiple formats throughout the dataset.

These variations include differences in capitalization and the presence of leading whitespace. For example, Tata Pension Management appears as Tata Pension Management, tata pension management, TATA PENSION MANAGEMENT, and Tata Pension Management. Similar variations occur for several other pension fund managers.

The variable contains 67 missing observations.

If these values were used directly for grouping or comparison, the same fund manager could be treated as multiple separate categories. Therefore, the Pension Fund Manager variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

5.8.11 Examining Nominee Relationship Frequencies

The frequency distribution of the Nominee Relationship variable is examined to understand the types of relationships represented between subscribers and their nominees.

This variable provides additional information about the nominee structure of pension accounts. Examining its frequency distribution also helps identify missing values, inconsistent category labels, abbreviations, or formatting variations that may need to be addressed during the data-cleaning stage.

The original values will be retained during this exploratory stage, and any required standardization will be performed later during preprocessing.

```
[20]: df["Nominee_Relationship"].value_counts(dropna=False)
```

```
[20]: Nominee_Relationship
      Spouse      824
      Father      776
      Brother     770
      Mother      758
      Son         736
      Sister      719
      Daughter    681
      NaN          67
      mother       34
      brother      33
      sister       30
      father       20
      daughter     18
      FATHER       16
      S/o          15
      son          14
      spouse       14
      D/o          14
      SPOUSE       14
      Name: count, dtype: int64
```

Observations from Nominee Relationship Frequency Analysis

The frequency analysis of the Nominee Relationship variable shows that the major nominee relationships represented in the dataset are Spouse, Father, Brother, Mother, Son, Sister, and Daughter.

The variable also contains multiple representations of the same relationship due to differences in capitalization. For example, Mother appears as Mother and mother, while Father appears as Father, father, and FATHER. Similar variations are present for Brother, Sister, Daughter, Son, and Spouse.

The variable contains 67 missing observations. In addition, abbreviated values such as S/o and D/o are present and will require further examination before deciding how they should be standardized.

These inconsistencies may result in the same nominee relationship being treated as separate categories during analysis. Therefore, the variable will be reviewed and standardized during the data-cleaning stage, while ambiguous abbreviations will be handled based on their meaning rather than capitalization alone.

At this stage, the original values are retained and no modifications are made.

5.8.12 Examining KYC Status Frequencies

The frequency distribution of the KYC Status variable is examined to understand the verification status of pension subscriber accounts.

KYC status is relevant to the overall account-quality and compliance profile of subscribers. Examining its frequency distribution helps identify the different KYC categories represented in the dataset, along with any missing observations or inconsistent categorical representations.

The results will be used to determine whether the variable requires standardization during the data-cleaning stage.

```
[21]: df["KYC_Status"].value_counts(dropna=False)

[21]: KYC_Status
      Verified    4731
      Pending      462
      Rejected     138
      verified      74
      V             71
      NaN           56
      pending        8
      rejected       6
      P              5
      R              2
      Name: count, dtype: int64
```

Observations from KYC Status Frequency Analysis

The frequency analysis of the KYC Status variable shows three main categories: Verified, Pending, and Rejected. The majority of observations are classified as Verified, followed by Pending and Rejected.

However, the same KYC statuses are represented using different formats. For example, Verified appears as Verified, verified, and V. Similarly, Pending appears as Pending, pending, and P, while Rejected appears as Rejected, rejected, and R.

The variable also contains 56 missing observations.

These variations may cause the same KYC status to be treated as separate categories during frequency analysis, grouping, and visualization. Therefore, the KYC Status variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

5.8.13 Examining Tax Benefit Section Frequencies

The frequency distribution of the Tax Benefit Section Claimed variable is examined to understand the representation of different tax-benefit categories in the dataset.

This variable is relevant to the financial characteristics of pension subscribers because tax benefits may be associated with pension contributions and retirement planning. Examining its frequency distribution also helps identify missing observations, inconsistent labels, abbreviations, or other formatting variations.

The results will be used to determine whether the variable requires standardization during the data-cleaning stage.

```
[22]: df["Tax_Benefit_Section_Claimed"].value_counts(dropna=False)
```

```
[22]: Tax_Benefit_Section_Claimed
80CCD(2)       1353
80CCD(1B)      1352
Not Claimed    1318
80CCD(1)       1308
NaN              70
80 CCD (1B)      26
80 CCD (2)       24
80ccd1b          23
80ccd1           21
80 CCD (1)       20
80ccd2           14
Not claimed      14
NOT CLAIMED      10
Name: count, dtype: int64
```

Observations from Tax Benefit Section Frequency Analysis

The frequency analysis of the Tax Benefit Section Claimed variable shows four major categories: 80CCD(2), 80CCD(1B), Not Claimed, and 80CCD(1).

The frequencies of these major categories are relatively similar, indicating that all four categories are substantially represented in the dataset. However, the same tax-benefit categories are represented using different formatting conventions.

For example, 80CCD(1B) appears as 80CCD(1B), 80 CCD (1B), and 80ccd1b. Similar variations are present for 80CCD(1) and 80CCD(2). The Not Claimed category also appears as Not Claimed, Not claimed, and NOT CLAIMED.

The variable contains 70 missing observations.

These inconsistencies could cause identical tax-benefit categories to be treated as separate groups during analysis. Therefore, the Tax Benefit Section Claimed variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

5.8.14 Examining Withdrawal Status Frequencies

The frequency distribution of the Withdrawal Status variable is examined to understand the withdrawal status of pension subscriber accounts.

Withdrawal status is relevant to pension-account activity and may be useful in later analysis when examining account conditions and retirement-related financial patterns. Examining its frequency distribution also helps identify missing observations and inconsistent categorical representations.

The results will be reviewed for variations in capitalization, abbreviations, whitespace, or other formatting issues that may require correction during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

```
[23]: df["Withdrawal_Status"].value_counts(dropna=False)
```

```
[23]: Withdrawal_Status
No Withdrawal              4651
Partial Withdrawal          575
Full Withdrawal - Closed    160
no withdrawal               101
NaN                          56
partial withdrawal            9
full withdrawal - closed      1
Name: count, dtype: int64
```

Observations from Withdrawal Status Frequency Analysis

The frequency analysis of the Withdrawal Status variable shows that the majority of records are classified as No Withdrawal, followed by Partial Withdrawal and Full Withdrawal - Closed.

The variable also contains inconsistent capitalization for the same categories. For example, No Withdrawal also appears as no withdrawal, while Partial Withdrawal appears as partial withdrawal and Full Withdrawal - Closed appears as full withdrawal - closed.

The variable contains 56 missing observations.

These inconsistencies may cause identical withdrawal-status categories to be treated as separate groups during analysis and visualization. Therefore, the Withdrawal Status variable will require categorical standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

6. Data Quality Assessment and Cleaning

The initial exploratory analysis has been completed. The next stage is to assess and improve the quality of the dataset based on the issues identified during exploration.

The exploratory analysis revealed several data-quality concerns, including missing values, inconsistent categorical labels, inconsistent formatting, potentially incorrect data types, and potentially invalid numerical values.

The cleaning process will be performed systematically. Each issue will first be identified and quantified before any modification is applied. This helps ensure that cleaning decisions are based on evidence from the dataset rather than assumptions.

The preprocessing stage will include duplicate detection, missing-value assessment, column-name standardization, categorical-value standardization, data-type conversion, validation of numerical and date variables, and identification of potentially invalid or extreme observations.

6.1 Checking for Duplicate Records

Duplicate records can affect the accuracy of statistical summaries, frequency distributions, group-based analysis, and visualizations because the same observation may be counted more than once.

The number of completely duplicated rows will first be identified using the duplicated() function.

No records will be removed at this stage. The duplicate count will first be examined so that an appropriate treatment can be determined based on the results.

```
[24]: df.duplicated().sum()
```

```
[24]: np.int64(0)
```

Observations from Duplicate Check

The duplicate check identified **0 completely duplicated records** in the dataset.

This indicates that no two rows are identical across all 38 columns. Therefore, no records need to be removed on the basis of complete row duplication.

Although there are no exact duplicate rows, repeated values within individual columns are expected because multiple subscribers can legitimately have the same characteristics, such as gender, occupation, pension scheme, or account status. These repeated categorical values should not be treated as duplicate records.

The dataset will therefore be retained without any row removal at this stage.

6.2 Assessing Missing Values

Missing values can affect statistical analysis, visualizations, calculations, and machine-learning or predictive methods if they are not appropriately handled.

The number of missing observations in each variable will therefore be calculated to assess the completeness of the dataset.

At this stage, missing values will only be identified and quantified. No values will be filled or rows removed until the extent and pattern of missingness have been examined.

This approach helps determine whether different variables require different missing-value treatment based on their data type and analytical importance.

```
[25]: df.isnull().sum()
```

```
[25]: Gender                                    55
      Date_of_Birth                            112
      Age                                      112
      Marital_Status                            54
      State                                      1
      City                                      56
      Aadhaar_Linked                            56
       PAN_Linked                               56
      Occupation_Type                           56
      Sector                                    56
      Pension_Scheme_Type                        1
      Date_of_Joining                           84
      Years_of_Contribution                     84
      Employer_Name                            100
      Annual_Income_INR                        112
      Monthly_Contribution_INR                  84
      Employer_Monthly_Contribution_INR         84
      Total_Contribution_Till_Date_INR          84
      Investment_Choice                         67
      Pension_Fund_Manager                      67
      Asset_Allocation_Equity_E_Percent         16
      Asset_Allocation_CorporateDebt_C_Percent  19
      Asset_Allocation_GovtSecurities_G_Percent 10
      Asset_Allocation_Alternative_A_Percent    14
      Current_NAV_INR                           67
      Total_Units_Held                          67
      Current_Fund_Value_INR                    67
      Expected_Monthly_Pension_INR              67
      Years_to_Retirement                       56
      Nominee_Name                             101
      Nominee_Relationship                      67
      Account_Status                            56
      KYC_Status                                56
      Bank_Name                                 67
      IFSC_Code                                 56
      Tax_Benefit_Section_Claimed               70
      Withdrawal_Status                         56
      Last_Transaction_Date                     84
      dtype: int64
```

Observations from Missing Value Assessment

The missing-value assessment shows that missing observations are present across several variables, although the extent of missingness differs between columns.

State and Pension Scheme Type contain only one missing observation each, whereas Date of Birth, Age, and Annual Income contain the highest number of missing observations, with 112 missing values each. Employer Name contains 100 missing observations, while several date, contribution, and financial variables contain between 67 and 84 missing observations.

Several categorical account-related variables, including Gender, Aadhaar Linked, PAN Linked, Occupation Type, Sector, Account Status, KYC Status, IFSC Code, and Withdrawal Status, contain approximately 55–56 missing observations.

The results indicate that missing data is present across demographic, employment, financial, investment, and account-related variables. Since the amount and nature of missingness differ across variables, a single missing-value treatment would not be appropriate for the entire dataset.

Therefore, missing values will be examined according to the type and meaning of each variable before deciding whether they should be retained, replaced, or otherwise handled during preprocessing.

6.3 Calculating Missing Value Percentage

The number of missing observations alone does not indicate the relative extent of missingness in each variable. Therefore, the percentage of missing values is calculated for every column.

The percentage is calculated relative to the total number of records in the dataset. This provides a comparable measure of data completeness across variables with different numbers of missing observations.

The resulting percentages will help determine the severity of missingness and support later decisions regarding appropriate treatment during the cleaning process.

```
[26]: missing_percentage = (df.isnull().sum() / len(df)) * 100
      missing_percentage
```

```
[26]: Gender                                    0.990456
      Date_of_Birth                             2.016928
      Age                                       2.016928
      Marital_Status                            0.972447
      State                                     0.018008
      City                                      1.008464
      Aadhaar_Linked                            1.008464
       PAN_Linked                               1.008464
      Occupation_Type                           1.008464
      Sector                                    1.008464
      Pension_Scheme_Type                       0.018008
      Date_of_Joining                           1.512696
      Years_of_Contribution                     1.512696
      Employer_Name                             1.800828
      Annual_Income_INR                         2.016928
      Monthly_Contribution_INR                  1.512696
      Employer_Monthly_Contribution_INR         1.512696
      Total_Contribution_Till_Date_INR          1.512696
      Investment_Choice                         1.206555
      Pension_Fund_Manager                      1.206555
      Asset_Allocation_Equity_E_Percent         0.288133
      Asset_Allocation_CorporateDebt_C_Percent  0.342157
      Asset_Allocation_GovtSecurities_G_Percent 0.180083
      Asset_Allocation_Alternative_A_Percent    0.252116
      Current_NAV_INR                           1.206555
      Total_Units_Held                          1.206555
      Current_Fund_Value_INR                    1.206555
      Expected_Monthly_Pension_INR              1.206555
      Years_to_Retirement                       1.008464
      Nominee_Name                              1.818837
      Nominee_Relationship                      1.206555
      Account_Status                            1.008464
      KYC_Status                                1.008464
      Bank_Name                                 1.206555
      IFSC_Code                                 1.008464
      Tax_Benefit_Section_Claimed               1.260580
      Withdrawal_Status                         1.008464
      Last_Transaction_Date                     1.512696
      dtype: float64
```

Observations from Missing Value Percentage

The missing-value percentage analysis shows that the overall level of missingness is relatively low across the dataset.

The highest missing-value percentage is approximately 2.02%, observed for Date of Birth, Age, and Annual Income. Other variables have missing-value percentages generally below 2%, while the asset-allocation variables have less than 0.35% missingness.

No variable has an extremely high proportion of missing observations. Therefore, removing entire columns because of missingness is not considered necessary at this stage.

Similarly, removing every row containing at least one missing value could result in unnecessary loss of observations. Instead, missing values will be handled according to the nature and meaning of each variable.

For categorical variables, appropriate categorical treatment will be considered, while numerical and date variables will be assessed separately before deciding on an appropriate imputation or retention strategy.

No missing values are modified at this stage.

6.4 Creating a Missing Value Summary

A consolidated missing-value summary is created to provide a clearer view of the completeness of each variable.

The summary combines the number of missing observations and the corresponding percentage of missing data for every column. This makes it

easier to compare missingness across variables and supports transparent decision-making during the preprocessing stage.

Only variables containing missing observations are displayed in the summary so that attention can be focused on columns requiring further assessment.

[27]:
```
missing_summary = pd.DataFrame({
    "Missing_Count": df.isnull().sum(),
    "Missing_Percentage": (df.isnull().sum() / len(df)) * 100
})

missing_summary = missing_summary[missing_summary["Missing_Count"] > 0]

missing_summary.sort_values("Missing_Count", ascending=False)
```

[27]:

	Missing_Count	Missing_Percentage
Age	112	2.016928
Annual_Income_INR	112	2.016928
Date_of_Birth	112	2.016928
Nominee_Name	101	1.818837
Employer_Name	100	1.800828
Last_Transaction_Date	84	1.512696
Date_of_Joining	84	1.512696
Employer_Monthly_Contribution_INR	84	1.512696
Monthly_Contribution_INR	84	1.512696
Years_of_Contribution	84	1.512696
Total_Contribution_Till_Date_INR	84	1.512696
Tax_Benefit_Section_Claimed	70	1.260580
Current_NAV_INR	67	1.206555
Current_Fund_Value_INR	67	1.206555
Expected_Monthly_Pension_INR	67	1.206555
Nominee_Relationship	67	1.206555
Bank_Name	67	1.206555
Total_Units_Held	67	1.206555
Pension_Fund_Manager	67	1.206555
Investment_Choice	67	1.206555
PAN_Linked	56	1.008464
Withdrawal_Status	56	1.008464
IFSC_Code	56	1.008464
KYC_Status	56	1.008464
Account_Status	56	1.008464
City	56	1.008464
Aadhaar_Linked	56	1.008464
Years_to_Retirement	56	1.008464
Occupation_Type	56	1.008464
Sector	56	1.008464
Gender	55	0.990456
Marital_Status	54	0.972447
Asset_Allocation_CorporateDebt_C_Percent	19	0.342157
Asset_Allocation_Equity_E_Percent	16	0.288133
Asset_Allocation_Alternative_A_Percent	14	0.252116
Asset_Allocation_GovtSecurities_G_Percent	10	0.180083
Pension_Scheme_Type	1	0.018008
State	1	0.018008

Observations from the Missing Value Summary

The consolidated missing-value summary shows that missing observations are present across demographic, employment, financial, investment, nominee, and account-related variables.

The highest level of missingness is observed in Age, Annual Income, and Date of Birth, with 112 missing observations each, representing approximately 2.02% of the dataset. Nominee Name and Employer Name have 101 and 100 missing observations respectively.

Several financial and account-related variables contain between 56 and 84 missing observations, while the asset-allocation variables have relatively low levels of missingness, ranging from approximately 0.18% to 0.34%.

State and Pension Scheme Type have only one missing observation each.

Since the percentage of missing values is relatively low across all variables, no column will be removed solely because of missingness. Similarly, complete-case deletion of rows is not considered appropriate at this stage because it could unnecessarily reduce the available data.

The missing values will be handled later according to the data type, meaning, and analytical importance of each variable. Numerical, categorical, and date variables will be assessed separately before an appropriate treatment is applied.

6.5 Standardizing Column Names

Consistent column names are important for reliable data manipulation and analysis. During the exploratory stage, a leading whitespace character was identified in the PAN_Linked column name.

Whitespace and formatting inconsistencies in column names can cause errors when selecting variables and can make the dataset more difficult to work with.

Before proceeding with further preprocessing, leading and trailing whitespace will therefore be removed from all column names. This step standardizes the column names without changing the underlying data values.

```
[28]: df.columns = df.columns.str.strip()

[29]: df.columns

[29]: Index(['Gender', 'Date_of_Birth', 'Age', 'Marital_Status', 'State', 'City',
       'Aadhaar_Linked', 'PAN_Linked', 'Occupation_Type', 'Sector',
       'Pension_Scheme_Type', 'Date_of_Joining', 'Years_of_Contribution',
       'Employer_Name', 'Annual_Income_INR', 'Monthly_Contribution_INR',
       'Employer_Monthly_Contribution_INR', 'Total_Contribution_Till_Date_INR',
       'Investment_Choice', 'Pension_Fund_Manager',
       'Asset_Allocation_Equity_E_Percent',
       'Asset_Allocation_CorporateDebt_C_Percent',
       'Asset_Allocation_GovtSecurities_G_Percent',
       'Asset_Allocation_Alternative_A_Percent', 'Current_NAV_INR',
       'Total_Units_Held', 'Current_Fund_Value_INR',
       'Expected_Monthly_Pension_INR', 'Years_to_Retirement', 'Nominee_Name',
       'Nominee_Relationship', 'Account_Status', 'KYC_Status', 'Bank_Name',
       'IFSC_Code', 'Tax_Benefit_Section_Claimed', 'Withdrawal_Status',
       'Last_Transaction_Date'],
      dtype='object')
```

Observations from Column Name Standardization

The column names were reviewed for leading and trailing whitespace. A formatting inconsistency was identified in the PAN_Linked column, where a leading space was present in the original column name.

The whitespace was removed using the str.strip() method. The resulting column list confirms that PAN_Linked is now consistently named and can be referenced reliably during further analysis.

No other column names were modified at this stage, and no underlying data values were changed.

6.6 Standardizing Categorical Variables

The exploratory analysis identified several categorical variables containing inconsistent representations of the same category. These inconsistencies include differences in capitalization, leading or trailing whitespace, abbreviations, and alternative representations such as Yes/No, 1/0, and True/False.

Such inconsistencies can cause a single logical category to be counted as multiple categories during analysis. Therefore, categorical variables will be standardized before further exploratory analysis and visualization.

The cleaning will be performed one variable at a time. First, the existing values will be reviewed, followed by an appropriate standardization rule based on the values observed in the dataset.

This approach avoids making assumptions about categories that have not been examined.

```
[30]: df["Gender"].value_counts(dropna=False)
```

```
[30]: Gender
      Male       3021
      Female     2139
      Other        60
      NaN          55
      MALE         48
      male         38
       male        32
      Female       28
      M            28
      Male         24
      female       22
       female      20
      F            18
      FEMALE       15
      other         3
      O             1
      OTHER         1
      Name: count, dtype: int64
```

6.6.1 Cleaning the Gender Variable

The Gender variable contains three meaningful categories: Male, Female, and Other. However, the exploratory analysis identified several inconsistent representations of these categories.

The inconsistencies are caused by differences in capitalization, leading whitespace, and abbreviated values. For example, Male appears as Male, MALE, male, male, and M, while Female appears as Female, female, female, FEMALE, and F. The Other category is similarly represented by Other, other, O, and OTHER.

These values represent the same logical categories and therefore need to be standardized. The cleaning process will remove unnecessary whitespace, make the text comparison case-insensitive, and convert the abbreviations to their corresponding full category names.

Missing values will not be replaced during this step because they represent unavailable information rather than a known gender category.

[31]:
```
df["Gender"] = (
    df["Gender"]
    .astype("string")
    .str.strip()
    .str.lower()
    .replace({
        "m": "male",
        "f": "female",
        "o": "other"
    })
    .str.title()
)
```

[32]:
```
df["Gender"].value_counts(dropna=False)
```

[32]:
```
Gender
Male      3191
Female    2242
Other       65
<NA>        55
Name: count, dtype: Int64
```

Observations After Cleaning Gender

The Gender variable has been successfully standardized into three consistent categories: Male, Female, and Other.

The different representations caused by capitalization, leading whitespace, and abbreviations have been consolidated into their corresponding categories. As a result, the variable now contains only the standardized category labels along with the original missing observations.

The 55 missing values have been retained because there is insufficient information to determine the actual gender of these records. They will be considered separately during the missing-value treatment stage.

The cleaned Gender variable is now suitable for further analysis and visualization.

6.6.2 Cleaning the Marital Status Variable

The Marital Status variable contains four primary categories: Married, Single, Widowed, and Divorced. However, the exploratory analysis identified inconsistent representations caused by capitalization, abbreviations, and formatting differences.

Examples include Married, MARRIED, and married, as well as abbreviated forms such as M. Similar variations are present for Single, Widowed, and Divorced. The value Mar. is also present and will be treated as a representation of Married based on its context.

The values will therefore be standardized by removing unnecessary whitespace, converting the text to a consistent case, and mapping the observed abbreviations to their corresponding full category names.

The Unknown category will not be converted into a known marital-status category because it explicitly indicates that the marital status is unknown. Missing values will also be retained separately.

```
[33]: df["Marital_Status"].value_counts(dropna=False)
```

```
[33]: Marital_Status
Married     3184
Single      1504
Widowed      355
Divorced     148
NaN           54
Mar.          46
MARRIED       45
M             42
married       39
SINGLE        31
Unknown       30
S             26
single        23
W             10
widowed       10
divorced       4
D              2
Name: count, dtype: int64
```

```
[34]: df["Marital_Status"] = (
          df["Marital_Status"]
          .astype("string")
          .str.strip()
          .str.lower()
          .replace({
              "m": "married",
              "mar.": "married",
              "s": "single",
              "w": "widowed",
              "d": "divorced"
          })
          .str.title()
      )
```

```
[35]: df["Marital_Status"].value_counts(dropna=False)
```

```
[35]: Marital_Status
Married     3356
Single      1584
Widowed      375
Divorced     154
<NA>          54
Unknown       30
Name: count, dtype: Int64
```

Observations After Cleaning Marital Status

The Marital Status variable has been successfully standardized into the categories Married, Single, Widowed, Divorced, and Unknown.

The different representations caused by capitalization, leading or trailing whitespace, and abbreviations have been consolidated into their corresponding categories. In particular, M, Mar., and other variations were standardized as Married, while S, W, and D were standardized as Single, Widowed, and Divorced respectively.

The Unknown category has been retained separately because it represents an explicitly unknown marital status rather than a missing observation.

The 54 missing observations have also been retained and will be considered separately during the missing-value treatment stage.

The cleaned Marital Status variable is now suitable for further analysis and visualization.

6.6.3 Cleaning the Aadhaar Linked Variable

The Aadhaar Linked variable indicates whether a subscriber's Aadhaar is linked to the pension account.

The exploratory analysis identified several representations of the same logical values. These include differences in capitalization, abbreviations, numeric representations, and Boolean representations. For example, Yes is represented by Yes, YES, yes, Y, 1, true, and TRUE, while No is represented by No, NO, no, N, 0, false, and FALSE.

These representations will be standardized into two consistent categories: Yes and No.

Missing observations will be retained as missing because they do not provide sufficient information to determine the Aadhaar-linking status.

```
[36]: df["Aadhaar_Linked"] = (
          df["Aadhaar_Linked"]
          .astype("string")
          .str.strip()
          .str.lower()
          .replace({
              "yes": "yes",
              "y": "yes",
              "1": "yes",
              "true": "yes",
              "no": "no",
              "n": "no",
              "0": "no",
              "false": "no"
          })
          .str.title()
      )

[37]: df["Aadhaar_Linked"].value_counts(dropna=False)

[37]: Aadhaar_Linked
      Yes     5127
      No       370
      <NA>      56
      Name: count, dtype: Int64
```

Observations After Cleaning Aadhaar Linked

The Aadhaar Linked variable has been successfully standardized into two consistent categories: Yes and No.

The different representations identified during the exploratory analysis, including variations in capitalization, abbreviations, numeric values, and Boolean values, have been consolidated into their corresponding categories.

The 56 missing observations have been retained because the original data does not provide sufficient information to determine the Aadhaar-linking status for these records.

The cleaned Aadhaar Linked variable is now suitable for further analysis and visualization.

6.6.4 Cleaning the PAN Linked Variable

The PAN Linked variable indicates whether a subscriber's PAN is linked to the pension account.

The exploratory analysis identified multiple representations of the same logical values, including differences in capitalization, abbreviations, numeric representations, and Boolean representations. For example, Yes is represented by Yes, YES, yes, Y, 1, true, and TRUE, while No is represented by No, NO, no, N, 0, false, and FALSE.

These representations will be standardized into two consistent categories: Yes and No.

Missing observations will be retained as missing because they do not provide sufficient information to determine the PAN-linking status

[38]:
```
df["PAN_Linked"] = (
    df["PAN_Linked"]
    .astype("string")
    .str.strip()
    .str.lower()
    .replace({
        "yes": "yes",
        "y": "yes",
        "1": "yes",
        "true": "yes",
        "no": "no",
        "n": "no",
        "0": "no",
        "false": "no"
    })
    .str.title()
)
```

[39]:
```
df["PAN_Linked"].value_counts(dropna=False)
```

[39]:
```
PAN_Linked
Yes     4414
No      1083
<NA>      56
Name: count, dtype: Int64
```

Observations After Cleaning PAN Linked

The PAN Linked variable has been successfully standardized into two consistent categories: Yes and No.

The different representations identified during the exploratory analysis, including variations in capitalization, abbreviations, numeric values, and Boolean values, have been consolidated into their corresponding categories.

The 56 missing observations have been retained because the original data does not provide sufficient information to determine the PAN-linking status for these records.

The cleaned PAN Linked variable is now suitable for further analysis and visualization.

6.6.5 Cleaning the Occupation Type Variable

The Occupation Type variable contains several employment and occupational categories, including Private Sector Employee, Government Employee, Factory Worker, Self-Employed, Professional, Domestic Worker, Driver, Vendor, Daily Wage Worker, and Farmer.

The exploratory analysis identified multiple representations of the same occupation caused mainly by differences in capitalization and leading or trailing whitespace. For example, Private Sector Employee appears in uppercase, lowercase, and whitespace-containing forms.

These variations will be standardized by removing unnecessary whitespace and converting the values to a consistent capitalization format. This will ensure that each occupation is treated as a single category during subsequent analysis.

Missing observations will be retained as missing because no reliable occupation can be inferred from the available data.

[40]:
```
df["Occupation_Type"] = (
    df["Occupation_Type"]
    .astype("string")
    .str.strip()
    .str.lower()
    .str.title()
)
```

[41]:
```
df["Occupation_Type"].value_counts(dropna=False)
```

[41]:
```
Occupation_Type
Private Sector Employee    1755
Government Employee        1305
Factory Worker              486
Self-Employed               444
Professional                402
Domestic Worker             247
Driver                      231
Vendor                      225
Daily Wage Worker           215
Farmer                      187
<NA>                         56
Name: count, dtype: Int64
```

Creating the Occupation Group Derived Variable

After standardizing the original Occupation Type variable, the detailed occupations are grouped into broader employment categories to support more meaningful comparative analysis.

The original Occupation_Type variable is retained so that the detailed occupational information is preserved. A new derived variable, Occupation_Group, is created by combining related occupations into five broader groups:

Private Sector

Government

Self-Employed

Professional

Other/Informal

The Other/Informal group includes Factory Worker, Domestic Worker, Driver, Vendor, Daily Wage Worker, and Farmer. This grouping reduces fragmentation among smaller occupational categories while retaining the overall employment structure of the dataset.

The derived variable will later be useful for comparing income, contributions, fund value, and expected pension across broader occupational groups.

[42]:
```
occupation_group_map = {
    "Private Sector Employee": "Private Sector",
    "Government Employee": "Government",
    "Self-Employed": "Self-Employed",
    "Professional": "Professional",
    "Factory Worker": "Other/Informal",
    "Domestic Worker": "Other/Informal",
    "Driver": "Other/Informal",
    "Vendor": "Other/Informal",
    "Daily Wage Worker": "Other/Informal",
    "Farmer": "Other/Informal"
}

df["Occupation_Group"] = df["Occupation_Type"].map(occupation_group_map)
```

[43]:
```
df["Occupation_Group"].value_counts(dropna=False)
```

[43]:
```
Occupation_Group
Private Sector    1755
Other/Informal    1591
Government        1305
Self-Employed      444
Professional       402
NaN                 56
Name: count, dtype: int64
```

Observations After Creating Occupation Groups

The detailed occupation categories have been successfully consolidated into five broader occupational groups: Private Sector, Government, Self-Employed, Professional, and Other/Informal.

The Private Sector group has the highest number of observations, followed by Other/Informal and Government. Self-Employed and Professional represent smaller groups within the dataset.

The 56 missing values in Occupation Group correspond directly to records where Occupation Type was missing. These values have been retained as missing rather than assigning an occupation group without sufficient information.

The original Occupation Type variable has been preserved, while Occupation Group will provide a simplified classification for subsequent comparative analysis.

6.6.6 Cleaning the Sector Variable

The Sector variable describes the broad sector associated with the subscriber and contains several sector categories.

The exploratory analysis identified inconsistent representations of the same sector caused mainly by differences in capitalization and leading or trailing whitespace. For example, Unorganized Sector appears in uppercase, lowercase, and whitespace-containing forms.

These variations will be standardized by removing unnecessary whitespace and converting the values to a consistent title-case format. This will ensure that identical sector categories are treated as a single category during subsequent analysis.

Missing observations will be retained as missing because no reliable sector classification can be inferred from the available value.

```
[44]: df["Sector"] = (
          df["Sector"]
          .astype("string")
          .str.strip()
          .str.lower()
          .str.title()
      )
```

```
[45]: df["Sector"].value_counts(dropna=False)
```

```
[45]: Sector
      All Citizen Model          1232
      Unorganized Sector         1109
      Organized Private Sector   1042
      Central/State Government    945
      Corporate                   805
      Central Government          364
      <NA>                         56
      Name: count, dtype: Int64
```

Observations After Cleaning Sector

The Sector variable has been successfully standardized into six consistent sector categories: All Citizen Model, Unorganized Sector, Organized Private Sector, Central/State Government, Corporate, and Central Government.

The inconsistent representations identified during exploratory analysis, including differences in capitalization and leading or trailing whitespace, have been consolidated into their corresponding categories.

The 56 missing observations have been retained because no reliable sector classification can be determined from the available data.

The cleaned Sector variable is now suitable for subsequent grouping, comparison, and visualization.

6.6.7 Cleaning the Pension Scheme Type Variable

The Pension Scheme Type variable contains several pension scheme categories, but the exploratory analysis identified multiple representations of the same scheme.

For example, NPS - All Citizen is represented as NPS - All Citizen, NPS-AllCitizen, and nps - all citizen. Similar variations are present for Atal Pension Yojana, Employees' Pension Scheme, NPS Government, NPS Corporate, and Unified Pension Scheme.

These differences are primarily caused by variations in capitalization, punctuation, abbreviations, and the inclusion or omission of descriptive text.

To avoid treating the same pension scheme as multiple categories, the observed representations will be mapped to six standardized scheme categories:

- NPS - All Citizen
- Atal Pension Yojana
- Employees' Pension Scheme
- NPS - Government
- NPS - Corporate
- Unified Pension Scheme

[46]:
```
pension_scheme_map = {
    "NPS - All Citizen": "NPS - All Citizen",
    "NPS-AllCitizen": "NPS - All Citizen",
    "nps - all citizen": "NPS - All Citizen",

    "Atal Pension Yojana (APY)": "Atal Pension Yojana",
    "Atal Pension Yojana": "Atal Pension Yojana",
    "APY": "Atal Pension Yojana",

    "Employees' Pension Scheme (EPS-95)": "Employees' Pension Scheme",
    "EPS": "Employees' Pension Scheme",
    "EPS-95": "Employees' Pension Scheme",
    "EPS 95": "Employees' Pension Scheme",

    "NPS - Government Sector": "NPS - Government",
    "NPS-Government": "NPS - Government",
    "NPS Govt": "NPS - Government",

    "NPS - Corporate Sector": "NPS - Corporate",
    "NPS Corporate": "NPS - Corporate",
    "NPS-Corp": "NPS - Corporate",

    "Unified Pension Scheme (UPS)": "Unified Pension Scheme",
    "Unified Pension Scheme": "Unified Pension Scheme",
    "UPS": "Unified Pension Scheme"
}

df["Pension_Scheme_Type"] = (
    df["Pension_Scheme_Type"]
    .astype("string")
    .str.strip()
    .replace(pension_scheme_map)
)
```

[47]:
```
df["Pension_Scheme_Type"].value_counts(dropna=False)
```

[47]:
```
Pension_Scheme_Type
NPS - All Citizen            1231
Atal Pension Yojana          1118
Employees' Pension Scheme    1050
NPS - Government              955
NPS - Corporate               818
Unified Pension Scheme        364
NPS                            16
<NA>                            1
Name: count, dtype: Int64
```

[48]:
```
df.loc[df["Pension_Scheme_Type"] == "NPS", ["Pension_Scheme_Type", "Sector", "Occupation_Type"]]
```

[48]:

	Pension_Scheme_Type	Sector	Occupation_Type
81	NPS	All Citizen Model	Professional
925	NPS	All Citizen Model	Private Sector Employee
1035	NPS	All Citizen Model	Professional
1234	NPS	All Citizen Model	Self-Employed
1687	NPS	All Citizen Model	Private Sector Employee
1736	NPS	All Citizen Model	Self-Employed
1893	NPS	All Citizen Model	Self-Employed
2160	NPS	All Citizen Model	Private Sector Employee
2365	NPS	All Citizen Model	Professional
2834	NPS	All Citizen Model	Professional
3449	NPS	All Citizen Model	Professional
4057	NPS	All Citizen Model	Professional
4070	NPS	All Citizen Model	Professional
4371	NPS	All Citizen Model	Professional
4612	NPS	All Citizen Model	Professional
5266	NPS	All Citizen Model	Self-Employed

[49]:
```
df.loc[
    (df["Pension_Scheme_Type"] == "NPS") &
    (df["Sector"] == "All Citizen Model"),
    "Pension_Scheme_Type"
] = "NPS - All Citizen"
```

[50]:
```
df["Pension_Scheme_Type"].value_counts(dropna=False)
```

[50]:
```
Pension_Scheme_Type
NPS - All Citizen            1247
Atal Pension Yojana          1118
Employees' Pension Scheme    1050
NPS - Government              955
NPS - Corporate               818
Unified Pension Scheme        364
<NA>                            1
```

Observations After Cleaning Pension Scheme Type

The Pension Scheme Type variable has been successfully standardized into six consistent pension scheme categories: NPS - All Citizen, Atal Pension Yojana, Employees' Pension Scheme, NPS - Government, NPS - Corporate, and Unified Pension Scheme.

Several inconsistent representations, including abbreviations, differences in capitalization, punctuation, and variations in scheme names, were consolidated into their corresponding standardized categories.

During the cleaning process, 16 records containing the generic value NPS were examined further. Since all 16 records belonged to the All Citizen Model sector, they were classified as NPS - All Citizen.

Only one observation remains missing. It has been retained as missing because there is insufficient information to reliably determine the pension scheme.

The cleaned Pension Scheme Type variable is now suitable for subsequent comparative analysis and visualization.

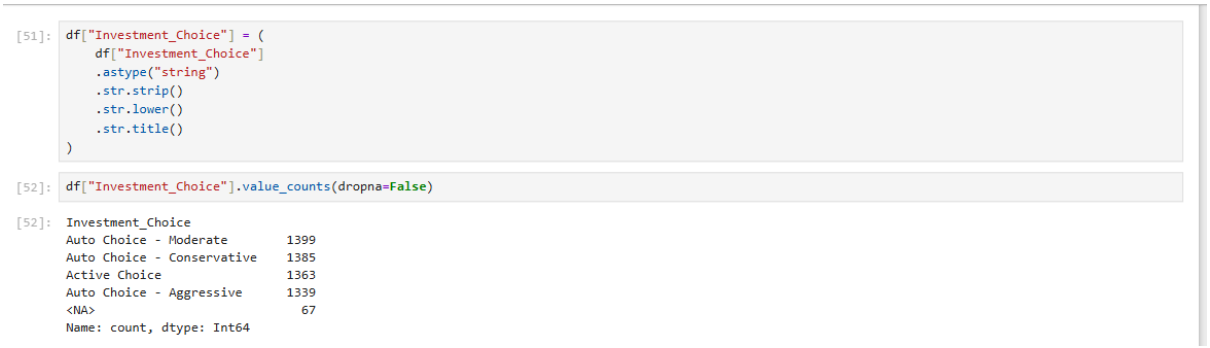
6.6.8 Cleaning the Investment Choice Variable

The Investment Choice variable describes the investment strategy selected by pension subscribers.

The exploratory analysis identified four main investment choices: Auto Choice - Moderate, Auto Choice - Conservative, Active Choice, and Auto Choice - Aggressive. Several inconsistent representations of these categories were also observed due to differences in capitalization and leading or trailing whitespace.

These variations will be standardized so that each investment choice is represented by a single consistent category. Missing observations will be retained as missing because no reliable investment choice can be determined for those records.

The cleaned variable will support later analysis of investment preferences and their relationship with asset allocation, fund value, contributions, and expected pension.



```
[51]: df["Investment_Choice"] = (
          df["Investment_Choice"]
          .astype("string")
          .str.strip()
          .str.lower()
          .str.title()
      )

[52]: df["Investment_Choice"].value_counts(dropna=False)

[52]: Investment_Choice
      Auto Choice - Moderate       1399
      Auto Choice - Conservative   1385
      Active Choice                1363
      Auto Choice - Aggressive     1339
      <NA>                           67
      Name: count, dtype: Int64
```

Observations After Cleaning Investment Choice

The Investment Choice variable has been successfully standardized into four consistent categories: Auto Choice - Moderate, Auto Choice - Conservative, Active Choice, and Auto Choice - Aggressive.

The inconsistent representations identified during exploratory analysis, including differences in capitalization and leading or trailing whitespace, have been consolidated into their corresponding categories.

The 67 missing observations have been retained because no reliable investment choice can be determined for these records. These missing values will be addressed later as part of the overall missing-value treatment.

The cleaned Investment Choice variable is now suitable for subsequent analysis of subscriber investment preferences and their relationship with pension and financial characteristics.

6.6.9 Cleaning the Pension Fund Manager Variable

The Pension Fund Manager variable identifies the pension fund management organization associated with each subscriber record.

The exploratory analysis identified ten main pension fund managers. However, several managers appear in multiple formats due to differences in capitalization and leading or trailing whitespace.

For example, Tata Pension Management appears as Tata Pension Management, tata pension management, TATA PENSION MANAGEMENT, and versions containing leading whitespace. These variations represent the same organization and therefore need to be consolidated.

The values will be standardized by removing unnecessary whitespace and converting the text to a consistent title-case format. Missing observations will be retained as missing because no fund manager can be reliably determined for those records.

[53]:
```
df["Pension_Fund_Manager"] = (
    df["Pension_Fund_Manager"]
    .astype("string")
    .str.strip()
    .str.lower()
    .str.title()
)
```

[54]:
```
df["Pension_Fund_Manager"].value_counts(dropna=False)
```

[54]:
```
Pension_Fund_Manager
Tata Pension Management          584
Uti Retirement Solutions         574
Kotak Mahindra Pension Fund      573
Aditya Birla Sun Life Pension    569
Sbi Pension Funds                553
Lic Pension Fund                 553
Axis Pension Fund Management     543
Max Life Pension Fund            538
Hdfc Pension Management          506
Icici Prudential Pension Fund    493
<NA>                              67
Name: count, dtype: Int64
```

Observations After Cleaning Pension Fund Manager

The Pension Fund Manager variable has been successfully standardized into ten consistent fund manager categories.

The inconsistencies identified during exploratory analysis, including differences in capitalization and leading or trailing whitespace, have been consolidated into single representations for each fund manager.

The 67 missing observations have been retained because no reliable pension fund manager can be determined for these records. The cleaned Pension Fund

Manager variable is now suitable for subsequent analysis and comparison of fund-related characteristics.

6.6.10 Cleaning the Nominee Relationship Variable

The Nominee Relationship variab|le identifies the relationship between the pension subscriber and the nominated person.

The exploratory analysis identified several relationship categories, including Spouse, Father, Mother, Brother, Sister, Son, and Daughter. The variable also contains inconsistent representations caused by differences in capitalization, as well as abbreviated forms such as S/o and D/o.

These values will be standardized by removing unnecessary whitespace, converting the text to a consistent format, and mapping the observed abbreviations to their corresponding relationship categories.
Specifically, S/o will be standardized as Son and D/o as Daughter.

Missing observations will be retained as missing because no reliable nominee relationship can be determined from the available data.

[55]:
```
df["Nominee_Relationship"] = (
    df["Nominee_Relationship"]
    .astype("string")
    .str.strip()
    .str.lower()
    .replace({
        "s/o": "son",
        "d/o": "daughter"
    })
    .str.title()
)
```

[56]:
```
df["Nominee_Relationship"].value_counts(dropna=False)
```

[56]:
```
Nominee_Relationship
Spouse      852
Father      812
Brother     803
Mother      792
Son         765
Sister      749
Daughter    713
<NA>         67
Name: count, dtype: Int64
```

Observations After Cleaning Nominee Relationship

The Nominee Relationship variable has been successfully standardized into seven consistent relationship categories: Spouse, Father, Brother, Mother, Son, Sister, and Daughter.

The inconsistent representations identified during exploratory analysis, including differences in capitalization, leading or trailing whitespace, and abbreviated forms such as S/o and D/o, have been consolidated into their corresponding categories.

The 67 missing observations have been retained because no reliable nominee relationship can be determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned Nominee Relationship variable is now suitable for subsequent analysis and visualization.

6.6.11 Cleaning the Account Status Variable

The Account Status variable describes the current status of the pension account.

The exploratory analysis identified four main categories: Active, Inactive, Dormant, and Frozen. Multiple representations of these categories were observed because of differences in capitalization and formatting.

These variations will be standardized by removing unnecessary whitespace and converting the values to a consistent title-case format. This will ensure that each account status is treated as a single category during subsequent analysis.

Missing observations will be retained as missing because the account status cannot be reliably determined for those records.

```
[57]: df["Account_Status"] = (
          df["Account_Status"]
          .astype("string")
          .str.strip()
          .str.lower()
          .str.title()
      )

[58]: df["Account_Status"].value_counts(dropna=False)

[58]: Account_Status
      Active      4347
      Inactive     696
      Dormant      298
      Frozen       156
      <NA>          56
      Name: count, dtype: Int64
```

Observations After Cleaning Account Status

The Account Status variable has been successfully standardized into four consistent categories: Active, Inactive, Dormant, and Frozen.

The inconsistent representations identified during exploratory analysis, mainly differences in capitalization, have been consolidated into their corresponding categories.

The Active category contains the largest number of records, while Inactive, Dormant, and Frozen accounts represent smaller groups within the dataset.

The 56 missing observations have been retained because the account status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned Account Status variable is now suitable for subsequent analysis and visualization.

6.6.12 Cleaning the KYC Status Variable

The KYC Status variable indicates the verification status of the subscriber's Know Your Customer (KYC) information.

The exploratory analysis identified three main categories: Verified, Pending, and Rejected. However, the same categories appear in different forms because of variations in capitalization and abbreviated values. For example, V, P, and R are used as abbreviated representations of Verified, Pending, and Rejected.

These variations will be standardized into the three corresponding categories. Missing observations will be retained as missing because the KYC status cannot be reliably determined for those records.

[59]:
```
df["KYC_Status"] = (
    df["KYC_Status"]
    .astype("string")
    .str.strip()
    .str.lower()
    .replace({
        "v": "verified",
        "p": "pending",
        "r": "rejected"
    })
    .str.title()
)
```

[60]:
```
df["KYC_Status"].value_counts(dropna=False)
```

[60]:
```
KYC_Status
Verified    4876
Pending      475
Rejected     146
<NA>          56
Name: count, dtype: Int64
```

Observations After Cleaning KYC Status

The KYC Status variable has been successfully standardized into three consistent categories: Verified, Pending, and Rejected.

The inconsistent representations identified during exploratory analysis, including differences in capitalization and abbreviated values such as V, P, and R, have been consolidated into their corresponding categories.

The Verified category contains the largest number of observations, followed by Pending and Rejected.

The 56 missing observations have been retained because the KYC status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned KYC Status variable is now suitable for subsequent analysis and visualization.

6.6.13 Cleaning the Tax Benefit Section Claimed Variable

The Tax Benefit Section Claimed variable indicates the tax benefit section associated with the pension contribution.

The exploratory analysis identified four main categories: 80CCD(2), 80CCD(1B), 80CCD(1), and Not Claimed. However, the same categories appear in multiple formats because of differences in spacing, capitalization, and punctuation.

For example, 80 CCD (1B) and 80ccd1b represent the same tax benefit section as 80CCD(1B). Similarly, Not claimed and NOT CLAIMED represent the same category as Not Claimed.

These representations will therefore be mapped to four standardized categories. Missing observations will be retained as missing because no reliable tax-benefit status can be determined for those records.

[61]:
```
tax_benefit_map = {
    "80CCD(2)": "80CCD(2)",
    "80 CCD (2)": "80CCD(2)",
    "80ccd2": "80CCD(2)",

    "80CCD(1B)": "80CCD(1B)",
    "80 CCD (1B)": "80CCD(1B)",
    "80ccd1b": "80CCD(1B)",

    "80CCD(1)": "80CCD(1)",
    "80 CCD (1)": "80CCD(1)",
    "80ccd1": "80CCD(1)",

    "Not Claimed": "Not Claimed",
    "Not claimed": "Not Claimed",
    "NOT CLAIMED": "Not Claimed"
}

df["Tax_Benefit_Section_Claimed"] = (
    df["Tax_Benefit_Section_Claimed"]
    .astype("string")
    .str.strip()
    .replace(tax_benefit_map)
)
```

[62]:
```
df["Tax_Benefit_Section_Claimed"].value_counts(dropna=False)
```

[62]:
```
Tax_Benefit_Section_Claimed
80CCD(1B)      1401
80CCD(2)       1391
80CCD(1)       1349
Not Claimed    1342
<NA>             70
Name: count, dtype: Int64
```

Observations After Cleaning Tax Benefit Section Claimed

The Tax Benefit Section Claimed variable has been successfully standardized into four consistent categories: 80CCD(1B), 80CCD(2), 80CCD(1), and Not Claimed.

The inconsistent representations identified during exploratory analysis, including differences in spacing, capitalization, and formatting, have been consolidated into their corresponding categories.

The 70 missing observations have been retained because the tax-benefit status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned Tax Benefit Section Claimed variable is now suitable for subsequent analysis of tax-benefit patterns and their relationship with pension contributions and subscriber characteristics.

6.6.14 Cleaning the Withdrawal Status Variable

The Withdrawal Status variable indicates the withdrawal status of the pension account.

The exploratory analysis identified three main categories: No Withdrawal, Partial Withdrawal, and Full Withdrawal - Closed. The same categories also appear in different formats because of differences in capitalization.

These variations will be standardized by removing unnecessary whitespace and converting the values to a consistent title-case format.

Missing observations will be retained as missing because the withdrawal status cannot be reliably determined for those records.

[63]:
```
df["Withdrawal_Status"] = (
    df["Withdrawal_Status"]
    .astype("string")
    .str.strip()
    .str.lower()
    .str.title()
)
```

[64]:
```
df["Withdrawal_Status"].value_counts(dropna=False)
```

[64]:
```
Withdrawal_Status
No Withdrawal              4752
Partial Withdrawal          584
Full Withdrawal - Closed    161
<NA>                         56
Name: count, dtype: Int64
```

Observations After Cleaning Withdrawal Status

The Withdrawal Status variable has been successfully standardized into three consistent categories: No Withdrawal, Partial Withdrawal, and Full Withdrawal - Closed.

The inconsistent representations identified during exploratory analysis, mainly differences in capitalization and formatting, have been consolidated into their corresponding categories.

The No Withdrawal category contains the largest number of observations, followed by Partial Withdrawal and Full Withdrawal - Closed.

The 56 missing observations have been retained because the withdrawal status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned Withdrawal Status variable is now suitable for subsequent analysis of withdrawal patterns and their relationship with account and pension characteristics.

6.7.1 Cleaning and Converting Date of Birth

The Date of Birth variable represents the subscriber's date of birth. During the initial data inspection, this variable was identified as an object data type rather than a proper date/time data type.

Since Date of Birth is a date field, it needs to be converted into Pandas' datetime format before it can be used reliably for analysis, validation, and calculations.

The variable also contains missing observations. These missing values will be preserved during the conversion and will be handled separately during the missing-value treatment stage.

Before converting the variable, its existing values will be inspected to identify any formatting or invalid-date issues.

[65]:
```
df["Date_of_Birth"].head(20)
```

[65]:
```
0     1969/01/26
1     09/04/1998
2     1967-06-05
3     1958-11-03
4     1982-07-18
5     1965-10-05
6     1958-08-11
7     1980-11-19
8     1975-12-18
9     1984-03-01
10    1987-03-23
11    2001-11-17
12    1984-01-30
13    1976-08-09
14    20/08/1979
15    1971-03-02
16    1974-06-28
17    1969-09-23
18    1974-03-09
19    1992-07-15
Name: Date_of_Birth, dtype: object
```

[66]:
```
df["Date_of_Birth"].dtype
```

[66]:
```
dtype('O')
```

[67]:
```
df["Date_of_Birth"].value_counts(dropna=False).head(20)
```

[67]:
```
Date_of_Birth
NaN           112
2031-01-15     16
1972-07-17      4
1980-09-18      4
1970-07-22      4
1999-03-02      3
1997-05-19      3
1999-11-27      3
1992-07-03      3
1994-12-13      3
1980-03-11      3
1998-07-02      3
1967-12-16      3
1980-04-09      3
1984-04-28      3
1962-08-24      3
1974-01-28      3
1967-11-01      3
1976-05-25      3
1992-06-13      3
Name: count, dtype: int64
```

[68]:
```
df[["Date_of_Birth", "Age"]].head(20)
```

[68]:

	Date_of_Birth	Age
0	1969/01/26	NaN
1	09/04/1998	28
2	1967-06-05	59
3	1958-11-03	68
4	1982-07-18	44
5	1965-10-05	61
6	1958-08-11	68
7	1980-11-19	46
8	1975-12-18	51
9	1984-03-01	42
10	1987-03-23	39
11	2001-11-17	25
12	1984-01-30	42
13	1976-08-09	50
14	20/08/1979	47
15	1971-03-02	55
16	1974-06-28	52
17	1969-09-23	57
18	1974-03-09	52
19	1992-07-15	34

```
[69]: df.loc[
          df["Date_of_Birth"].astype(str) == "2031-01-15",
          ["Date_of_Birth", "Age"]
      ]
```

[69]:

	Date_of_Birth	Age
1196	2031-01-15	57
1320	2031-01-15	38
1409	2031-01-15	-45
1420	2031-01-15	57
1696	2031-01-15	36
1910	2031-01-15	49
2122	2031-01-15	38
2856	2031-01-15	55
3220	2031-01-15	51
3268	2031-01-15	35
3348	2031-01-15	67
3638	2031-01-15	NaN
3861	2031-01-15	32
4196	2031-01-15	37
4495	2031-01-15	62
4699	2031-01-15	54

```
[70]: df["Age"].dtype
```

```
[70]: dtype('O')
```

6.7.2 Converting Age to Numeric Data Type

The Age variable represents the age of each pension subscriber and was identified as an object data type during the initial data inspection.

Since Age is a numerical variable, it needs to be converted to a numeric data type before performing calculations and logical validation.

The conversion will also allow the Age variable to be compared with Date of Birth to identify invalid or inconsistent records. Any values that cannot be converted to numeric form will be treated as missing and addressed later during the missing-value treatment stage.

```
[71]: df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
```

```
[72]: df["Age"].dtype
```

```
[72]: dtype('float64')
```

```
[73]: df["Age"].describe()
```

```
[73]: count    5330.000000
      mean       46.449531
      std        36.048051
      min       -68.000000
      25%        34.000000
      50%        46.000000
      75%        57.000000
      max       999.000000
      Name: Age, dtype: float64
```

6.7.3 Validating Age Values

After converting the Age variable to a numeric data type, the descriptive statistics reveal several potentially invalid values.

The minimum recorded age is -68 years and the maximum is 999 years. Both values are logically impossible for a pension subscriber and indicate data-quality errors rather than genuine observations.

Before deciding how to treat these records, the Age variable will be examined for values outside a reasonable human age range. This validation step will help distinguish valid observations from logically impossible values and will ensure that erroneous values are not included in subsequent analysis.

```
[74]: df.loc[df["Age"] < 0, ["Age", "Date_of_Birth"]].sort_values("Age")
```

[74]:

	Age	Date_of_Birth
4849	-68.0	1958-11-07
79	-66.0	1960-12-16
4604	-65.0	1961-12-01
2816	-65.0	16-05-1961
4476	-64.0	1962-12-20
2669	-64.0	1962-06-05
251	-61.0	1965-05-12
556	-57.0	1969-09-30
2978	-53.0	1973-12-07
5404	-51.0	1975-02-02
1830	-51.0	1975-10-06
765	-50.0	1976-05-25
1409	-45.0	2031-01-15
4919	-43.0	1983-03-26
3860	-43.0	1983-05-28
3035	-42.0	1984-11-20
2239	-40.0	1986-06-24
4630	-39.0	1987/11/12
2822	-34.0	1992-02-28
3262	-33.0	1993-01-13
3152	-32.0	1994-05-31
4544	-32.0	1994-01-02
3143	-31.0	1995-01-07
1945	-30.0	1996-04-10
3758	-29.0	1997-01-29

6.7.4 Identifying Invalid Negative Age Values

The Age variable contains several negative values, which are logically impossible because age cannot be negative.

Further investigation shows that these negative values are associated with plausible dates of birth. For example, a subscriber with a Date of Birth of 1995-01-07 has an Age recorded as -31, while the corresponding logical age is approximately 31 years.

This indicates that the negative sign is likely a data-entry or data-generation error rather than an indication of an incorrect Date of Birth.

Therefore, negative Age values will be investigated against Date of Birth before correction. Where the Date of Birth supports the corresponding positive age, the negative sign can be removed by converting the value to its absolute value.

Records with missing or invalid Date of Birth values will be handled separately rather than automatically corrected.

[75]:
```
negative_age_check = df.loc[
    (df["Age"] < 0) & (df["Date_of_Birth"].notna()),
    ["Age", "Date_of_Birth"]
].copy()

negative_age_check["Age_Absolute"] = negative_age_check["Age"].abs()

negative_age_check
```

[75]:

	Age	Date_of_Birth	Age_Absolute
79	-66.0	1960-12-16	66.0
165	-25.0	27-Oct-2001	25.0
251	-61.0	1965-05-12	61.0
556	-57.0	1969-09-30	57.0
765	-50.0	1976-05-25	50.0
1409	-45.0	2031-01-15	45.0
1830	-51.0	1975-10-06	51.0
1945	-30.0	1996-04-10	30.0
2239	-40.0	1986-06-24	40.0
2669	-64.0	1962-06-05	64.0
2816	-65.0	16-05-1961	65.0
2822	-34.0	1992-02-28	34.0
2978	-53.0	1973-12-07	53.0
3035	-42.0	1984-11-20	42.0
3143	-31.0	1995-01-07	31.0
3152	-32.0	1994-05-31	32.0
3262	-33.0	1993-01-13	33.0
3758	-29.0	1997-01-29	29.0
3860	-43.0	1983-05-28	43.0
4476	-64.0	1962-12-20	64.0
4544	-32.0	1994-01-02	32.0
4604	-65.0	1961-12-01	65.0
4849	-68.0	1958-11-07	68.0
4919	-43.0	1983-03-26	43.0
5404	-51.0	1975-02-02	51.0

[76]:
```
df.loc[
    (df["Age"] < 0) & (df["Date_of_Birth"].isna()),
    ["Age", "Date_of_Birth"]
]
```

[76]:

	Age	Date_of_Birth
3723	-27.0	NaN

[77]:
```
df.loc[
    df["Age"] > 100,
    ["Age", "Date_of_Birth"]
].sort_values("Age")
```

[77]:

	Age	Date_of_Birth
5517	150.0	06-02-1971
447	150.0	1968-08-29
4233	150.0	11/24/1993
742	150.0	1985-07-23
4067	150.0	1999-06-29
2027	150.0	1986-04-12
2779	150.0	1988-12-21
3844	210.0	05/14/1996
3701	210.0	1962-12-18
3447	210.0	1995-02-21
3354	210.0	1979-02-11
2850	210.0	1985-09-04
280	210.0	1996-11-27
1825	210.0	1981-07-27

1800	210.0	21-Aug-1959
2728	210.0	1979-08-26
2046	999.0	1967-02-16
2043	999.0	2000-12-30
1247	999.0	1969-12-23
623	999.0	1963-06-15
4314	999.0	16-11-1993
5143	999.0	1998-12-29

6.7.5 Correcting Invalid Age Values

The validation of the Age variable identified two types of logical errors.

First, 26 records contain negative ages. Examination of their corresponding Date of Birth values shows that most of these negative values represent sign errors. For example, an Age of -31 is associated with a Date of Birth of 1995-01-07, which corresponds to an age of approximately 31 years. Therefore, these validated negative ages can be corrected by removing the negative sign.

However, the record with Age -45 and Date of Birth 2031-01-15 cannot be validated because the Date of Birth itself is logically invalid. Similarly, the negative Age record with a missing Date of Birth cannot be independently validated. These records will therefore be treated separately.

Second, the dataset contains ages of 150, 210, and 999 years. These values are logically impossible and are therefore classified as invalid values rather than statistical outliers.

The invalid values will be converted to missing values so that they do not distort subsequent descriptive and exploratory analysis.

Age Validation Criteria

Because the dataset represents pension subscribers, an age below 18 is considered logically invalid for the purposes of this analysis. At the same time, an upper working-age limit is not imposed because pension subscribers may continue to hold pension accounts beyond conventional retirement ages.

Therefore, Age values below 18 or above 100 will be treated as logically invalid. Negative ages are handled separately because the initial investigation indicates that most negative values are sign errors and can be corrected using the corresponding Date of Birth where possible.

6.8 Date of Birth Standardization

The Date of Birth variable was initially stored as an object data type and contains dates represented in multiple formats, including ISO format, slash-separated dates, hyphen-separated dates, and dates containing month names.

To enable reliable date-based analysis and comparison with Age, the values will be converted into a standardized datetime format. Values that cannot be interpreted as valid dates will be converted to missing values using errors="coerce".

The standardized Date of Birth variable will then be examined for missing, future, and logically invalid dates.

[78]:
```
df["Date_of_Birth"] = pd.to_datetime(
    df["Date_of_Birth"],
    errors="coerce",
    format="mixed"
)
```

[79]:
```
df["Date_of_Birth"].dtype
```

[79]:
```
dtype('<M8[ns]')
```

[80]:
```
df["Date_of_Birth"].head(10)
```

[80]:
```
0   1969-01-26
1   1998-09-04
2   1967-06-05
3   1958-11-03
4   1982-07-18
5   1965-10-05
6   1958-08-11
7   1980-11-19
8   1975-12-18
9   1984-03-01
Name: Date_of_Birth, dtype: datetime64[ns]
```

[81]:
```
print("Missing Date of Birth:", df["Date_of_Birth"].isna().sum())
```

```
Missing Date of Birth: 112
```

Observation After Converting Date of Birth

The Date of Birth variable has been successfully converted from an object data type to datetime format.

After conversion, 112 observations remain missing. These missing values have been preserved as NaT

The standardized datetime format will now allow the Date of Birth variable to be checked for logical inconsistencies, such as future birth dates, and compared with the Age variable.

[82]:
```
df[df["Date_of_Birth"] > pd.Timestamp("2026-08-17")]["Date_of_Birth"].value_counts()
```

[82]:
```
Date_of_Birth
2031-01-15    16
Name: count, dtype: int64
```

6.8.1 Identifying and Handling Future Dates of Birth

The Date of Birth variable was examined for dates occurring after the analysis date. Sixteen records were found with a Date of Birth of `2031-01-15`.

Since a future date cannot represent the Date of Birth of an existing pension subscriber, these values are considered logically invalid.

The affected records will not be removed from the dataset because other information associated with these subscribers may still be valid. Instead, the invalid Date of Birth values will be converted to missing values (`NaT`) and will be considered during the later missing-value treatment stage.

```
[83]: df.loc[
          df["Date_of_Birth"] > pd.Timestamp("2026-08-17"),
          "Date_of_Birth"
      ] = pd.NaT

[84]: df[df["Age"] < 18]["Age"].value_counts().sort_index()

[84]: Age
      -68.0    1
      -66.0    1
      -65.0    2
      -64.0    2
      -61.0    1
      -57.0    1
      -53.0    1
      -51.0    2
      -50.0    1
      -45.0    1
      -43.0    2
      -42.0    1
      -40.0    1
      -39.0    1
      -34.0    1
      -33.0    1
      -32.0    2
      -31.0    1
      -30.0    1
      -29.0    1
      -27.0    1
      -25.0    1
      Name: count, dtype: int64

[85]: df[df["Age"] > 100]["Age"].value_counts().sort_index()

[85]: Age
      150.0    7
      210.0    9
      999.0    6
      Name: count, dtype: int64
```

6.8.2 Identifying Reasonable Age Values

The Age variable was examined for values above the maximum plausible age of 100 years established for this analysis.

The dataset contains 22 observations with implausible ages: 7 records with an age of 150, 9 records with an age of 210, and 6 records with an age of 999.

These values are logically invalid and are therefore treated as data-quality errors rather than genuine observations. The affected Age values will be converted to missing values so that they do not distort subsequent statistical analysis.

The original subscriber records will be retained because other variables in these records may still contain useful information.

```
[86]: df.loc[df["Age"] > 100, "Age"] = np.nan
```

6.8.3 Handling Negative Age Values

The Age variable contains several negative values, which are not logically valid. Initial examination of the corresponding Date of Birth values indicates that most of these negative ages are likely caused by sign errors. For example, an Age of -31 is associated with a Date of Birth in 1995, which is consistent with an age of approximately 31 years.

Therefore, negative ages will not be removed immediately. They will first be compared with the corresponding Date of Birth values. Negative ages that can be validated using a reasonable Date of Birth will be corrected by removing the negative sign. Negative ages that cannot be validated because the Date of Birth is missing or invalid will be treated as missing values.

[87]:
```
df.loc[df["Age"] < 0, ["Age", "Date_of_Birth"]]
```

[87]:

	Age	Date_of_Birth
79	-66.0	1960-12-16
165	-25.0	2001-10-27
251	-61.0	1965-05-12
556	-57.0	1969-09-30
765	-50.0	1976-05-25
1409	-45.0	NaT
1830	-51.0	1975-10-06
1945	-30.0	1996-04-10
2239	-40.0	1986-06-24
2669	-64.0	1962-06-05
2816	-65.0	1961-05-16
2822	-34.0	1992-02-28
2978	-53.0	1973-12-07
3035	-42.0	1984-11-20
3143	-31.0	1995-01-07
3152	-32.0	1994-05-31
3262	-33.0	1993-01-13
3723	-27.0	NaT
3758	-29.0	1997-01-29
3860	-43.0	1983-05-28
4476	-64.0	1962-12-20
4544	-32.0	1994-01-02
4604	-65.0	1961-12-01
4630	-39.0	1987-11-12
4849	-68.0	1958-11-07
4919	-43.0	1983-03-26
5404	-51.0	1975-02-02

[88]:
```
# Correct negative ages when a valid DOB is available
valid_negative_age = (
    (df["Age"] < 0) &
    (df["Date_of_Birth"].notna())
)

df.loc[valid_negative_age, "Age"] = (
    df.loc[valid_negative_age, "Age"].abs()
)
```

[89]:
```
df.loc[df["Age"] < 0, "Age"] = np.nan
```

[90]:
```
df[df["Age"] < 0]
```

[90]:

	Gender	Date_of_Birth	Age	Marital_Status	State	City	Aadhaar_Linked	PAN_Linked	Occupation_Type	Sector	...	Nominee_Name	Nominee_Relationship	Account_

0 rows × 39 columns

6.8.4 Verification of Age Cleaning

After handling the invalid Age values, the dataset was checked to ensure that no negative ages remained.

The verification returned zero records with negative Age values. This confirms that the identified negative ages were either corrected using the corresponding Date of Birth or converted to missing values when they could not be reliably validated.

Similarly, ages above 100 years were previously identified as implausible and converted to missing values. The cleaned Age variable can now be used for further validation and exploratory analysis.

6.8.5 Validating Age Against Date of Birth

After cleaning the Age variable, the recorded Age will be compared with Date of Birth to identify inconsistencies between the two variables.

Because Age depends on the reference date used for the dataset, a small difference of one year can occur depending on whether the subscriber's birthday has already occurred. Therefore, the comparison will focus on identifying clearly inconsistent records rather than requiring an exact year-for-year match.

This validation will help identify records where either Age or Date of Birth may contain a data-entry error.

```
[91]: df[["Date_of_Birth", "Age"]].head(10)
```

[91]:

	Date_of_Birth	Age
0	1969-01-26	NaN
1	1998-09-04	28.0
2	1967-06-05	59.0
3	1958-11-03	68.0
4	1982-07-18	44.0
5	1965-10-05	61.0
6	1958-08-11	68.0
7	1980-11-19	46.0
8	1975-12-18	51.0
9	1984-03-01	42.0

Age and Date of Birth Consistency Check

The cleaned Age variable was compared with Date of Birth to identify inconsistencies between the two fields.

The observed records indicate that Age is generally represented using the difference between the reference year (2026) and the year of birth, without considering whether the individual's birthday has already occurred during the year.

Therefore, the consistency check will compare the recorded Age with the expected age calculated as:

Age = 2026 − Year of Birth

Records where the recorded Age differs from this expected value will be examined for possible data-entry inconsistencies.

```
[92]: df["Expected_Age"] = 2026 - df["Date_of_Birth"].dt.year

[93]: (df["Age"] != df["Expected_Age"]).sum()

[93]: np.int64(368)

[94]: (df["Age"] - df["Expected_Age"]).value_counts(dropna=False).sort_index()

[94]: 0.0    5185
      NaN     368
      Name: count, dtype: int64
```

6.8.6 Result of Age and Date of Birth Consistency Validation¶

The recorded Age was compared with the expected age calculated from the year of birth using the reference year 2026.

Among records where both Age and Date of Birth were available, all 5,185 records showed an exact match between the recorded Age and the calculated expected age. The remaining 368 records contain missing values in either Age or Date of Birth and therefore could not be compared.

This indicates that there are no identified inconsistencies between Age and Date of Birth among the records with complete information.

```
[95]: df.drop(columns=["Expected_Age"], inplace=True)
```

6.9 Numeric Variable Cleaning and Validation

The dataset contains several financial and contribution-related variables that were initially stored as object data types. Before identifying outliers or performing statistical analysis, these variables must be converted into appropriate numeric formats.

The numeric variables will then be examined for missing values, impossible negative values, and other logical inconsistencies. Genuine extreme observations will be distinguished from data-entry or data-quality errors before any values are modified.

```
[96]: df["Annual_Income_INR"].dtype
```

```
[96]: dtype('O')
```

6.9.1 Converting Annual Income to Numeric

The Annual_Income_INR variable is ### 6.9.1 Converting Annual Income to Numeric

The Annual_Income_INR variable is currently stored as an object data type. Since annual income is a quantitative financial variable, it should be converted to a numeric data type before performing descriptive statistics, correlation analysis, or outlier detection.

Non-numeric or invalid entries will be converted to missing values using errors="coerce" rather than causing the conversion to fail.currently stored as an object data type. Since annual income is a quantitative financial variable, it should be converted to a numeric data type before performing descriptive statistics, correlation analysis, or outlier detection.

Non-numeric or invalid entries will be converted to missing values using errors="coerce" rather than causing the conversion to fail.

```
[97]: df["Annual_Income_INR"] = pd.to_numeric(
          df["Annual_Income_INR"],
          errors="coerce"
      )

[98]: df["Annual_Income_INR"].dtype

[98]: dtype('float64')

[99]: df["Annual_Income_INR"].describe()

[99]: count    5.219000e+03
      mean     5.863425e+07
      std      1.694526e+09
      min     -1.874400e+06
      25%      3.912500e+05
      50%      9.342000e+05
      75%      1.449200e+06
      max      5.000000e+10
      Name: Annual_Income_INR, dtype: float64
```

6.9.2 Initial Validation of Annual Income

The descriptive statistics for Annual Income reveal substantial variation and potential data-quality issues. The variable contains a negative value of ₹1,874,400, which is logically invalid for annual income. The maximum value of ₹50 billion is also extremely large compared with the median annual income of ₹934,200 and requires further investigation.

The large difference between the mean and median indicates that extreme observations are strongly influencing the mean. These observations will therefore be investigated before deciding whether they represent genuine high-income subscribers or data-entry errors.

Invalid values will be handled separately from legitimate statistical outliers.

```
[100]: df[df["Annual_Income_INR"] < 0]["Annual_Income_INR"]

[100]: 40      -183700.0
       532     -664700.0
       690     -727600.0
       1200   -1218600.0
       1363    -277400.0
       1685    -297300.0
       1974    -983100.0
       2117    -906700.0
       2184   -1046800.0
       2227    -429900.0
       2553    -205200.0
       2563    -228600.0
       2715    -904900.0
       2778    -235300.0
       3195    -545500.0
       3204    -461000.0
       4080   -1439600.0
       4387   -1352800.0
       4667    -195500.0
       5154   -1339200.0
       5347   -1874400.0
       5540    -332700.0
       Name: Annual_Income_INR, dtype: float64
```

6.9.3 Handling Negative Annual Income

The Annual Income variable contains 22 negative values. Since annual income cannot be negative in the context of this pension subscriber dataset, these observations are considered logically invalid.

The affected income values will be converted to missing values rather than removing the corresponding subscriber records. This preserves the remaining valid information contained in those records while preventing invalid income values from affecting subsequent analysis.

```
[101]: df.loc[df["Annual_Income_INR"] < 0, "Annual_Income_INR"] = np.nan
```

```
[102]: df[df["Annual_Income_INR"] > 100_000_000]["Annual_Income_INR"].sort_values(ascending=False)
```

```
[102]: 590     5.000000e+10
       1695    5.000000e+10
       2425    5.000000e+10
       3301    5.000000e+10
       4732    5.000000e+10
       5249    5.000000e+10
       3185    1.000000e+09
       Name: Annual_Income_INR, dtype: float64
```

```
[103]: df.loc[
           (df["Annual_Income_INR"] <= 100_000_000),
           "Annual_Income_INR"
       ].median()
```

```
[103]: 936050.0
```

6.9.4 Handling Extreme Annual Income Values

The Annual Income variable contains seven extremely large values: six observations of ₹50 billion and one observation of ₹1 billion.

These values are disproportionately large compared with the distribution of the remaining observations and are considered data-quality errors for the purpose of this analysis.

Because the mean is highly sensitive to extreme values, it is not appropriate to use the overall mean for replacement. The median of the valid income observations is used instead because it is more robust to extreme values.

The median Annual Income after excluding the identified extreme values is ₹936,050. Therefore, the seven extreme income values will be replaced with ₹936,050.

```
[104]: df.loc[
           df["Annual_Income_INR"] > 100_000_000,
           "Annual_Income_INR"
       ] = 936050.0
```

```
[105]: df["Monthly_Contribution_INR"].dtype
```

```
[105]: dtype('O')
```

6.9.5 Converting Monthly Contribution to Numeric

The Monthly_Contribution_INR variable is stored as an object data type even though it represents a quantitative monetary amount.

The variable will be converted to a numeric data type so that missing values, invalid values, and potential outliers can be identified correctly.

```
[106]: df["Monthly_Contribution_INR"] = pd.to_numeric(
           df["Monthly_Contribution_INR"],
           errors="coerce"
       )

[107]: df["Monthly_Contribution_INR"].dtype

[107]: dtype('float64')

[108]: df["Monthly_Contribution_INR"].describe()

[108]: count     5303.000000
       mean      4972.892702
       std       4548.803648
       min     -15580.000000
       25%       1250.000000
       50%       3780.000000
       75%       8370.000000
       max      16630.000000
       Name: Monthly_Contribution_INR, dtype: float64

[109]: df[df["Monthly_Contribution_INR"] < 0]["Monthly_Contribution_INR"]

[109]: 392    -11660.0
       815     -8870.0
       839     -4560.0
       993     -9470.0
       1388   -11320.0
       1459     -376.0
       1909    -1250.0
       2000    -4350.0
       2040     -376.0
       2435   -13630.0
       2462    -1250.0
       2544    -7240.0
       2907    -7050.0
       3205    -8810.0
       3405    -8090.0
       3410   -15580.0
       3492    -5610.0
       3674   -11900.0
       3960    -1720.0
       4091    -1250.0
       4504    -1250.0
       4848    -1250.0
       Name: Monthly_Contribution_INR, dtype: float64
```

6.9.6 Handling Negative Monthly Contributions

The Monthly Contribution variable contains 22 negative values. Since a pension contribution represents an amount contributed to the pension account, negative contribution values are logically invalid.

These values are therefore treated as data-quality errors and will be converted to missing values rather than replaced with a mean or median. The corresponding subscriber records will be retained because other variables may still contain valid information.

```
[110]: df.loc[df["Monthly_Contribution_INR"] < 0, "Monthly_Contribution_INR"] = np.nan

[111]: df["Employer_Monthly_Contribution_INR"].dtype

[111]: dtype('float64')

[112]: df["Employer_Monthly_Contribution_INR"].describe()

[112]: count     5469.000000
       mean      4579.846407
       std       6097.142076
       min          0.000000
       25%          0.000000
       50%       1250.000000
       75%       8710.000000
       max      20960.000000
       Name: Employer_Monthly_Contribution_INR, dtype: float64
```

6.9.7 Validation of Employer Monthly Contribution

The Employer Monthly Contribution variable is already stored in numeric format. The variable contains values ranging from ₹0 to ₹20,960.

No negative values are present. Although zero employer contributions occur in the dataset, they are not automatically considered invalid because employer

contributions may vary according to the subscriber's employment type and pension scheme.

Therefore, the zero values will be retained unless further analysis identifies them as inconsistent with other variables.

```
[113]: (df["Employer_Monthly_Contribution_INR"] == 0).sum()
```

```
[113]: np.int64(2315)
```

```
[114]: df.loc[
           df["Employer_Monthly_Contribution_INR"] == 0,
           "Occupation_Group"
       ].value_counts(dropna=False)
```

```
[114]: Occupation_Group
       Other/Informal    1089
       Self-Employed      421
       Professional       398
       Private Sector     384
       NaN                 23
       Name: count, dtype: int64
```

6.9.8 Validation of Zero Employer Contributions

A total of 2,315 records contain a zero Employer Monthly Contribution.

Further examination shows that these zero values occur across different occupation groups, with the largest concentration among Other/Informal workers, Self-Employed individuals, Professionals, and Private Sector employees.

Since zero employer contribution values are systematically present across occupational groups and are not logically impossible, they are retained as valid observations rather than being treated as missing values or outliers.

```
[115]: df["Years_of_Contribution"].dtype
```

```
[115]: dtype('O')
```

6.9.9 Converting Years of Contribution to Numeric

The Years_of_Contribution variable is currently stored as an object data type even though it represents a quantitative duration.

The variable will be converted to a numeric data type so that its values can be checked for missing, negative, and logically inconsistent observations.

```
[116]: df["Years_of_Contribution"] = pd.to_numeric(
           df["Years_of_Contribution"],
           errors="coerce"
       )
```

```
[117]: df["Years_of_Contribution"].dtype
```

```
[117]: dtype('float64')
```

```
[118]: df["Years_of_Contribution"].describe()
```

```
[118]: count    5358.000000
       mean        7.633651
       std         4.656570
       min       -15.000000
       25%         3.800000
       50%         7.200000
       75%        11.400000
       max        16.500000
       Name: Years_of_Contribution, dtype: float64
```

6.9.10 Validation of Years of Contribution

The Years of Contribution variable ranges from -15 to 16.5 years, with a median of 7.2 years.

Negative values are logically invalid because a subscriber cannot have a negative contribution period. These observations will therefore be treated as missing values.

The maximum value of 16.5 years is not inherently implausible and will be retained. No statistical outlier treatment will be applied at this stage because the focus is on identifying logically invalid values rather than removing legitimate extreme observations.

```
[119]: df.loc[df["Years_of_Contribution"] < 0, "Years_of_Contribution"] = np.nan

[120]: df[df["Years_of_Contribution"] < 0]["Years_of_Contribution"]

[120]: Series([], Name: Years_of_Contribution, dtype: float64)
```

6.9.11 Verification of Years of Contribution

After replacing the invalid negative values with missing values, the variable was checked again.

The verification returned no negative observations, confirming that all remaining Years_of_Contribution values are non-negative and suitable for further analysis.

```
[121]: df["Total_Contribution_Till_Date_INR"].dtype

[121]: dtype('O')
```

6.9.12 Converting Total Contribution to Numeric

The Total_Contribution_Till_Date_INR variable is currently stored as an object data type even though it represents a quantitative monetary amount.

The variable will be converted to a numeric data type so that invalid values, missing values, and potential outliers can be identified accurately.

```
[122]: df["Total_Contribution_Till_Date_INR"] = pd.to_numeric(
           df["Total_Contribution_Till_Date_INR"],
           errors="coerce"
       )

[123]: df["Total_Contribution_Till_Date_INR"].dtype

[123]: dtype('float64')

[124]: df["Total_Contribution_Till_Date_INR"].describe()

[124]: count    5.303000e+03
       mean     8.700513e+05
       std      1.187047e+06
       min      3.000000e+02
       25%      9.480000e+04
       50%      3.605000e+05
       75%      1.145900e+06
       max      7.904820e+06
       Name: Total_Contribution_Till_Date_INR, dtype: float64

[125]: df["Current_NAV_INR"].dtype

[125]: dtype('float64')

[126]: df["Current_NAV_INR"].describe()

[126]: count    5486.000000
       mean       40.636388
       std        15.196633
       min       -64.546000
       25%        29.299525
       50%        41.246100
       75%        53.288525
       max        64.992100
       Name: Current_NAV_INR, dtype: float64
```

6.9.13 Validation of Current NAV

The Current NAV variable is already stored in numeric format. The values range from -₹64.546 to ₹64.9921.

Since Net Asset Value (NAV) represents the value per unit of a pension fund, a negative NAV is logically invalid. Therefore, negative NAV values will be treated as data-quality errors and converted to missing values.

The positive values will be retained at this stage, including the relatively high values, because they are not inherently impossible.

```
[127]: df.loc[df["Current_NAV_INR"] < 0, "Current_NAV_INR"] = np.nan
```

```
[128]: df[df["Current_NAV_INR"] < 0]["Current_NAV_INR"]
```

```
[128]: Series([], Name: Current_NAV_INR, dtype: float64)
```

6.9.14 Verification of Current NAV

After replacing the invalid negative NAV values with missing values, the variable was checked again.

The verification returned no negative observations, confirming that all remaining Current NAV values are non-negative and suitable for further analysis.

```
[129]: df["Total_Units_Held"].dtype
```

```
[129]: dtype('float64')
```

```
[130]: df["Total_Units_Held"].describe()
```

```
[130]: count      5486.000000
       mean      28802.411352
       std       44015.706946
       min     -107446.947000
       25%        2715.597200
       50%       10888.418250
       75%       36638.265900
       max      497921.522200
       Name: Total_Units_Held, dtype: float64
```

6.9.15 Validation of Total Units Held

The Total Units Held variable is already stored in numeric format. The variable contains a negative minimum value of -107,446.9472 units.

Since the number of units held in a pension account cannot be negative, negative values are considered logically invalid and will be treated as missing values.

The maximum value of 497,921.5222 units is relatively large but is not inherently impossible. Therefore, it will be retained until the later outlier analysis.

```
[131]: df.loc[df["Total_Units_Held"] < 0, "Total_Units_Held"] = np.nan
```

```
[132]: df[df["Total_Units_Held"] < 0]["Total_Units_Held"]
```

```
[132]: Series([], Name: Total_Units_Held, dtype: float64)
```

6.9.16 Verification of Total Units Held

After replacing the invalid negative values with missing values, the Total Units Held variable was checked again.

The verification returned no negative observations, confirming that all remaining Total Units Held values are non-negative and suitable for further analysis.

```
[133]: df["Current_Fund_Value_INR"].dtype

[133]: dtype('float64')

[134]: df["Current_Fund_Value_INR"].describe()

[134]: count    5.486000e+03
       mean     1.070384e+06
       std      1.493917e+06
       min      3.073100e+02
       25%      1.150031e+05
       50%      4.290982e+05
       75%      1.397464e+06
       max      1.769063e+07
       Name: Current_Fund_Value_INR, dtype: float64
```

6.9.17 Validation of Current Fund Value

The Current Fund Value variable is already stored in numeric format. The values range from ₹307.31 to ₹17,690,630, with a median of approximately ₹429,098.

No negative or logically impossible values are observed in the descriptive statistics. Although some values are relatively high, they are not automatically considered invalid and will therefore be retained for the subsequent outlier analysis.

```
[135]: df["Expected_Monthly_Pension_INR"].dtype

[135]: dtype('float64')

[136]: df["Expected_Monthly_Pension_INR"].describe()

[136]: count    5.486000e+03
       mean     1.859914e+04
       std      3.568742e+05
       min     -1.539410e+04
       25%      1.399105e+03
       50%      3.000000e+03
       75%      6.970540e+03
       max      9.999999e+06
       Name: Expected_Monthly_Pension_INR, dtype: float64
```

6.9.18 Validation of Expected Monthly Pension

The Expected Monthly Pension variable is already stored in numeric format. The variable contains negative values and an extremely large maximum value.

Negative pension values are logically invalid and will be treated as missing values. The extremely large values will be investigated separately before deciding whether they represent valid observations or data-quality errors.

The large difference between the mean and median indicates that extreme observations are strongly influencing the distribution. Therefore, the mean will not be used automatically for replacing extreme values.

6.9.21 Validation of Years to Retirement

The Years to Retirement variable is already stored in numeric format. The variable ranges from -31 to 37 years, with a median of 14 years.

Negative values are logically invalid because an individual cannot have a negative number of years remaining until retirement. These observations will therefore be treated as data-quality errors and converted to missing values.

The maximum value of 37 years is not inherently implausible and will be retained for further analysis.

```
[143]: df.loc[df["Years_to_Retirement"] < 0, "Years_to_Retirement"] = np.nan

[144]: df[df["Years_to_Retirement"] < 0]["Years_to_Retirement"]

[144]: Series([], Name: Years_to_Retirement, dtype: float64)
```

6.9.22 Verification of Years to Retirement

After replacing the invalid negative values with missing values, the Years to Retirement variable was checked again.

The verification returned no negative observations, confirming that all remaining values are non-negative and suitable for further analysis.

6.9.23 Validation of Asset Allocation Percentages

The asset allocation variables represent the percentage of pension fund assets allocated to different asset classes. Therefore, valid values must fall between 0% and 100%.

The following variables will be checked for logically invalid negative values or values exceeding 100%:

- Asset_Allocation_Equity_E_Percent
- Asset_Allocation_CorporateDebt_C_Percent
- Asset_Allocation_GovtSecurities_G_Percent
- Asset_Allocation_Alternative_A_Percent

Invalid percentage values will be treated as missing values rather than replacing them with statistical estimates.

```
[145]: allocation_cols = [
           "Asset_Allocation_Equity_E_Percent",
           "Asset_Allocation_CorporateDebt_C_Percent",
           "Asset_Allocation_GovtSecurities_G_Percent",
           "Asset_Allocation_Alternative_A_Percent"
       ]

       df[allocation_cols].describe()
```

[145]:

	Asset_Allocation_Equity_E_Percent	Asset_Allocation_CorporateDebt_C_Percent	Asset_Allocation_GovtSecurities_G_Percent	Asset_Allocation_Alternative_A_Percent
count	5537.000000	5534.000000	5543.000000	5539.000000
mean	36.667329	32.551319	23.920440	6.558585
std	16.115804	8.530433	14.557951	6.302701
min	-50.000000	-60.000000	-45.000000	-20.000000
25%	15.000000	30.000000	15.000000	5.000000
50%	35.000000	35.000000	25.000000	5.000000
75%	50.000000	35.000000	45.000000	5.000000
max	77.000000	77.000000	66.000000	59.000000

import pandas **as** pd

import numpy **as** np

import matplotlib.pyplot **as** plt

import seaborn **as** sns

Initial Observations from the Last Five Records

The last five records provide an additional view of the dataset and show that some data-quality issues are also present near the end of the dataset.

Missing values can be observed in fields such as Aadhaar Linked and Date of Birth. In addition, some variables appear to contain inconsistent representations. For example, the Age field includes a value represented as "68 years", while other records contain numeric-looking age values. The PAN Linked field also contains different representations, including "Yes" and "1".

These observations indicate that data-quality checks will be required during the preprocessing stage. However, no values are modified at this stage because the purpose of the current section is to explore the dataset before cleaning.

Result of Dataset Dimension Check

The current dataset contains **5,553 records and 38 columns**. This represents the dataset after the manual removal of unnecessary rows and columns during the preparation stage.

The dataset therefore provides 5,553 subscriber-level observations across 38 variables covering different aspects of demographic, employment, pension, financial, investment, and retirement-related information.

These dimensions will be used as the basis for describing the current dataset throughout the remainder of the project.

Observations from the Column Names

The dataset contains **38 variables** representing different aspects of pension subscriber information, including demographic details, employment information, pension scheme characteristics, financial contributions, investment allocation, retirement information, nominee details, and account status.

A review of the column names indicates that most variables follow a consistent naming convention using underscores to separate words. However, one column (PAN_Linked) appears to contain a leading whitespace character. Such formatting inconsistencies may create difficulties when selecting or manipulating variables during analysis.

The column names will therefore be reviewed and standardized during the data-cleaning stage to ensure consistency throughout the project.

Observations from Descriptive Statistics

The descriptive statistical summary provides an initial overview of the numerical variables currently recognized by the dataset.

The results indicate considerable variation across several financial and investment-related variables. For example, Current Fund Value and Expected

Monthly Pension show substantial differences between their lower quartiles, medians, upper quartiles, and maximum values, suggesting that these variables may have highly dispersed or skewed distributions.

The summary also identifies several values that require further investigation. Negative values are present in variables such as asset allocation percentages, Current NAV, Total Units Held, Expected Monthly Pension, and Years to Retirement. Some of these values may represent invalid observations because they are not logically meaningful for the corresponding variables.

These observations do not immediately imply that the records should be removed. The potentially invalid or extreme values will be examined during the data-quality assessment and outlier detection stages, where appropriate validation rules and statistical methods will be applied.

It should also be noted that several numerical variables are currently stored as object data types and therefore are not included in this numerical summary. These variables will be examined during the data-cleaning stage.

Observations from Gender Frequency Analysis

The frequency analysis of the Gender variable shows that the dataset contains three main gender categories: Male, Female, and Other. However, these categories are represented using multiple variations in capitalization and formatting.

For example, Male appears as Male, MALE, male, and M, while Female appears as Female, FEMALE, female, and F. Similarly, Other appears in different forms including Other, other, OTHER, and O.

There are also 55 missing observations in the Gender variable.

These variations indicate an inconsistency in categorical data representation. During the data-cleaning stage, these values will need to be standardized into consistent categories so that frequency calculations, grouping, and visualizations are accurate.

At this stage, the original values are retained because the purpose of the exploration phase is to identify data-quality issues before applying any corrections.

Observations from Account Status Frequency Analysis

The frequency analysis of the Account Status variable shows that the dataset contains four main account-status categories: Active, Inactive, Dormant, and Frozen.

However, the same categories are represented using different capitalization formats. For example, Active appears as Active, active, and ACTIVE, while similar variations are present for Inactive, Dormant, and Frozen.

The variable also contains 56 missing observations.

These inconsistencies could cause the same account status to be counted as separate categories during grouping and visualization. Therefore, the Account Status variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained because the purpose of the exploration phase is to identify data-quality issues before applying corrections.

Observations from Marital Status Frequency Analysis

The frequency analysis of the Marital Status variable shows that Married and Single are the two most common categories, followed by Widowed and Divorced.

However, the variable contains multiple representations of the same marital-status categories. For example, Married appears as Married, MARRIED, married, and M. Similarly, Single appears as Single, SINGLE, single, and S, while Widowed and Divorced also appear in abbreviated and differently capitalized forms.

The variable also contains an Unknown category with 30 observations and 54 missing observations.

These inconsistencies may cause the same marital-status category to be treated as separate groups during analysis. Therefore, the values will need to be standardized during the data-cleaning stage. The treatment of the Unknown and missing values will also be considered during preprocessing based on the nature of the variable and the overall data-quality assessment.

At this stage, no modifications are made to the original values.

Observations from Aadhaar Linkage Frequency Analysis

The frequency analysis of the Aadhaar Linked variable indicates that the intended responses are primarily Yes and No. However, the same responses are represented using several different formats.

For example, positive responses appear as Yes, YES, yes, Y, 1, true, and TRUE. Similarly, negative responses appear as No, NO, no, N, 0, false, and FALSE.

The variable also contains 56 missing observations.

These variations indicate that the Aadhaar Linked variable is not consistently standardized. If these values were used directly in frequency analysis or grouping, the different representations of the same response could be treated as separate categories and produce inaccurate results.

During the data-cleaning stage, these representations will be standardized into consistent categories. The missing observations will also be assessed before deciding how they should be handled.

At this stage, the original values are retained and no modifications are made.

Observations from Pension Fund Manager Frequency Analysis

The frequency analysis of the Pension Fund Manager variable shows that the dataset contains ten major pension fund managers, with relatively substantial representation across the different managers.

The largest observed categories include Tata Pension Management, Kotak Mahindra Pension Fund, UTI Retirement Solutions, Aditya Birla Sun Life

Pension, and LIC Pension Fund. However, the same fund manager names are represented in multiple formats throughout the dataset.

These variations include differences in capitalization and the presence of leading whitespace. For example, Tata Pension Management appears as Tata Pension Management, tata pension management, TATA PENSION MANAGEMENT, and Tata Pension Management. Similar variations occur for several other pension fund managers.

The variable contains 67 missing observations.

If these values were used directly for grouping or comparison, the same fund manager could be treated as multiple separate categories. Therefore, the Pension Fund Manager variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

Observations from KYC Status Frequency Analysis

The frequency analysis of the KYC Status variable shows three main categories: Verified, Pending, and Rejected. The majority of observations are classified as Verified, followed by Pending and Rejected.

However, the same KYC statuses are represented using different formats. For example, Verified appears as Verified, verified, and V. Similarly, Pending appears as Pending, pending, and P, while Rejected appears as Rejected, rejected, and R.

The variable also contains 56 missing observations.

These variations may cause the same KYC status to be treated as separate categories during frequency analysis, grouping, and visualization. Therefore, the KYC Status variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

Observations from Tax Benefit Section Frequency Analysis

The frequency analysis of the Tax Benefit Section Claimed variable shows four major categories: 80CCD(2), 80CCD(1B), Not Claimed, and 80CCD(1).

The frequencies of these major categories are relatively similar, indicating that all four categories are substantially represented in the dataset. However, the same tax-benefit categories are represented using different formatting conventions.

For example, 80CCD(1B) appears as 80CCD(1B), 80 CCD (1B), and 80ccd1b. Similar variations are present for 80CCD(1) and 80CCD(2). The Not Claimed category also appears as Not Claimed, Not claimed, and NOT CLAIMED.

The variable contains 70 missing observations.

These inconsistencies could cause identical tax-benefit categories to be treated as separate groups during analysis. Therefore, the Tax Benefit Section Claimed variable will require standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

Observations from Withdrawal Status Frequency Analysis

The frequency analysis of the Withdrawal Status variable shows that the majority of records are classified as No Withdrawal, followed by Partial Withdrawal and Full Withdrawal - Closed.

The variable also contains inconsistent capitalization for the same categories. For example, No Withdrawal also appears as no withdrawal, while Partial Withdrawal appears as partial withdrawal and Full Withdrawal - Closed appears as full withdrawal - closed.

The variable contains 56 missing observations.

These inconsistencies may cause identical withdrawal-status categories to be treated as separate groups during analysis and visualization. Therefore, the Withdrawal Status variable will require categorical standardization during the data-cleaning stage.

At this stage, the original values are retained and no modifications are made.

Observations from Duplicate Check

The duplicate check identified **0 completely duplicated records** in the dataset.

This indicates that no two rows are identical across all 38 columns. Therefore, no records need to be removed on the basis of complete row duplication.

Although there are no exact duplicate rows, repeated values within individual columns are expected because multiple subscribers can legitimately have the same characteristics, such as gender, occupation, pension scheme, or account status. These repeated categorical values should not be treated as duplicate records.

The dataset will therefore be retained without any row removal at this stage.

Observations from Missing Value Assessment

The missing-value assessment shows that missing observations are present across several variables, although the extent of missingness differs between columns.

State and Pension Scheme Type contain only one missing observation each, whereas Date of Birth, Age, and Annual Income contain the highest number of missing observations, with 112 missing values each. Employer Name contains 100 missing observations, while several date, contribution, and financial variables contain between 67 and 84 missing observations.

Several categorical account-related variables, including Gender, Aadhaar Linked, PAN Linked, Occupation Type, Sector, Account Status, KYC Status,

IFSC Code, and Withdrawal Status, contain approximately 55–56 missing observations.

The results indicate that missing data is present across demographic, employment, financial, investment, and account-related variables. Since the amount and nature of missingness differ across variables, a single missing-value treatment would not be appropriate for the entire dataset.

Therefore, missing values will be examined according to the type and meaning of each variable before deciding whether they should be retained, replaced, or otherwise handled during preprocessing.

Observations from Missing Value Percentage

The missing-value percentage analysis shows that the overall level of missingness is relatively low across the dataset.

The highest missing-value percentage is approximately 2.02%, observed for Date of Birth, Age, and Annual Income. Other variables have missing-value percentages generally below 2%, while the asset-allocation variables have less than 0.35% missingness.

No variable has an extremely high proportion of missing observations. Therefore, removing entire columns because of missingness is not considered necessary at this stage.

Similarly, removing every row containing at least one missing value could result in unnecessary loss of observations. Instead, missing values will be handled according to the nature and meaning of each variable.

For categorical variables, appropriate categorical treatment will be considered, while numerical and date variables will be assessed separately before deciding on an appropriate imputation or retention strategy.

No missing values are modified at this stage.

Observations from the Missing Value Summary

The consolidated missing-value summary shows that missing observations are present across demographic, employment, financial, investment, nominee, and account-related variables.

The highest level of missingness is observed in Age, Annual Income, and Date of Birth, with 112 missing observations each, representing approximately 2.02% of the dataset. Nominee Name and Employer Name have 101 and 100 missing observations respectively.

Several financial and account-related variables contain between 56 and 84 missing observations, while the asset-allocation variables have relatively low levels of missingness, ranging from approximately 0.18% to 0.34%.

State and Pension Scheme Type have only one missing observation each.

Since the percentage of missing values is relatively low across all variables, no column will be removed solely because of missingness. Similarly, complete-case

deletion of rows is not considered appropriate at this stage because it could unnecessarily reduce the available data.

The missing values will be handled later according to the data type, meaning, and analytical importance of each variable. Numerical, categorical, and date variables will be assessed separately before an appropriate treatment is applied.

Observations from Column Name Standardization

The column names were reviewed for leading and trailing whitespace. A formatting inconsistency was identified in the PAN_Linked column, where a leading space was present in the original column name.

The whitespace was removed using the str.strip() method. The resulting column list confirms that PAN_Linked is now consistently named and can be referenced reliably during further analysis.

No other column names were modified at this stage, and no underlying data values were changed.

Observations After Cleaning Gender

The Gender variable has been successfully standardized into three consistent categories: Male, Female, and Other.

The different representations caused by capitalization, leading whitespace, and abbreviations have been consolidated into their corresponding categories. As a result, the variable now contains only the standardized category labels along with the original missing observations.

The 55 missing values have been retained because there is insufficient information to determine the actual gender of these records. They will be considered separately during the missing-value treatment stage.

The cleaned Gender variable is now suitable for further analysis and visualization.

Observations After Creating Occupation Groups

The detailed occupation categories have been successfully consolidated into five broader occupational groups: Private Sector, Government, Self-Employed, Professional, and Other/Informal.

The Private Sector group has the highest number of observations, followed by Other/Informal and Government. Self-Employed and Professional represent smaller groups within the dataset.

The 56 missing values in Occupation Group correspond directly to records where Occupation Type was missing. These values have been retained as missing rather than assigning an occupation group without sufficient information.

The original Occupation Type variable has been preserved, while Occupation Group will provide a simplified classification for subsequent comparative analysis.

Observations After Cleaning Sector

The Sector variable has been successfully standardized into six consistent sector categories: All Citizen Model, Unorganized Sector, Organized Private Sector, Central/State Government, Corporate, and Central Government.

The inconsistent representations identified during exploratory analysis, including differences in capitalization and leading or trailing whitespace, have been consolidated into their corresponding categories.

The 56 missing observations have been retained because no reliable sector classification can be determined from the available data.

The cleaned Sector variable is now suitable for subsequent grouping, comparison, and visualization.

Observations After Cleaning Pension Scheme Type

The Pension Scheme Type variable has been successfully standardized into six consistent pension scheme categories: NPS - All Citizen, Atal Pension Yojana, Employees' Pension Scheme, NPS - Government, NPS - Corporate, and Unified Pension Scheme.

Several inconsistent representations, including abbreviations, differences in capitalization, punctuation, and variations in scheme names, were consolidated into their corresponding standardized categories.

During the cleaning process, 16 records containing the generic value NPS were examined further. Since all 16 records belonged to the All Citizen Model sector, they were classified as NPS - All Citizen.

Only one observation remains missing. It has been retained as missing because there is insufficient information to reliably determine the pension scheme.

The cleaned Pension Scheme Type variable is now suitable for subsequent comparative analysis and visualization.

Observations After Cleaning Investment Choice

The Investment Choice variable has been successfully standardized into four consistent categories: Auto Choice - Moderate, Auto Choice - Conservative, Active Choice, and Auto Choice - Aggressive.

The inconsistent representations identified during exploratory analysis, including differences in capitalization and leading or trailing whitespace, have been consolidated into their corresponding categories.

The 67 missing observations have been retained because no reliable investment choice can be determined for these records. These missing values will be addressed later as part of the overall missing-value treatment.

The cleaned Investment Choice variable is now suitable for subsequent analysis of subscriber investment preferences and their relationship with pension and financial characteristics.

Observations After Cleaning Pension Fund Manager

The Pension Fund Manager variable has been successfully standardized into ten consistent fund manager categories.

The inconsistencies identified during exploratory analysis, including differences in capitalization and leading or trailing whitespace, have been consolidated into single representations for each fund manager.

The 67 missing observations have been retained because no reliable pension fund manager can be determined for these records. The cleaned Pension Fund Manager variable is now suitable for subsequent analysis and comparison of fund-related characteristics.

Observations After Cleaning KYC Status

The KYC Status variable has been successfully standardized into three consistent categories: Verified, Pending, and Rejected.

The inconsistent representations identified during exploratory analysis, including differences in capitalization and abbreviated values such as V, P, and R, have been consolidated into their corresponding categories.

The Verified category contains the largest number of observations, followed by Pending and Rejected.

The 56 missing observations have been retained because the KYC status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned KYC Status variable is now suitable for subsequent analysis and visualization.

Observations After Cleaning Tax Benefit Section Claimed

The Tax Benefit Section Claimed variable has been successfully standardized into four consistent categories: 80CCD(1B), 80CCD(2), 80CCD(1), and Not Claimed.

The inconsistent representations identified during exploratory analysis, including differences in spacing, capitalization, and formatting, have been consolidated into their corresponding categories.

The 70 missing observations have been retained because the tax-benefit status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned Tax Benefit Section Claimed variable is now suitable for subsequent analysis of tax-benefit patterns and their relationship with pension contributions and subscriber characteristics.

Observations After Cleaning Withdrawal Status

The Withdrawal Status variable has been successfully standardized into three consistent categories: No Withdrawal, Partial Withdrawal, and Full Withdrawal - Closed.

The inconsistent representations identified during exploratory analysis, mainly differences in capitalization and formatting, have been consolidated into their corresponding categories.

The No Withdrawal category contains the largest number of observations, followed by Partial Withdrawal and Full Withdrawal - Closed.

The 56 missing observations have been retained because the withdrawal status cannot be reliably determined from the available data. These missing values will be addressed later during the overall missing-value treatment stage.

The cleaned Withdrawal Status variable is now suitable for subsequent analysis of withdrawal patterns and their relationship with account and pension characteristics.

6.7.2 Converting Age to Numeric Data Type

The Age variable represents the age of each pension subscriber and was identified as an object data type during the initial data inspection.

Since Age is a numerical variable, it needs to be converted to a numeric data type before performing calculations and logical validation.

The conversion will also allow the Age variable to be compared with Date of Birth to identify invalid or inconsistent records. Any values that cannot be converted to numeric form will be treated as missing and addressed later during the missing-value treatment stage.

6.7.4 Identifying Invalid Negative Age Values

The Age variable contains several negative values, which are logically impossible because age cannot be negative.

Further investigation shows that these negative values are associated with plausible dates of birth. For example, a subscriber with a Date of Birth of 1995-01-07 has an Age recorded as -31, while the corresponding logical age is approximately 31 years.

This indicates that the negative sign is likely a data-entry or data-generation error rather than an indication of an incorrect Date of Birth.

Therefore, negative Age values will be investigated against Date of Birth before correction. Where the Date of Birth supports the corresponding positive age, the negative sign can be removed by converting the value to its absolute value.

Records with missing or invalid Date of Birth values will be handled separately rather than automatically corrected.

6.7.5 Correcting Invalid Age Values

The validation of the Age variable identified two types of logical errors.

First, 26 records contain negative ages. Examination of their corresponding Date of Birth values shows that most of these negative values represent sign errors. For example, an Age of -31 is associated with a Date of Birth of 1995-01-07, which corresponds to an age of approximately 31 years. Therefore, these validated negative ages can be corrected by removing the negative sign.

However, the record with Age -45 and Date of Birth 2031-01-15 cannot be validated because the Date of Birth itself is logically invalid. Similarly, the negative Age record with a missing Date of Birth cannot be independently validated. These records will therefore be treated separately.

Second, the dataset contains ages of 150, 210, and 999 years. These values are logically impossible and are therefore classified as invalid values rather than statistical outliers.

The invalid values will be converted to missing values so that they do not distort subsequent descriptive and exploratory analysis.

Observation After Converting Date of Birth

The Date of Birth variable has been successfully converted from an object data type to datetime format.

After conversion, 112 observations remain missing. These missing values have been preserved as NaT

The standardized datetime format will now allow the Date of Birth variable to be checked for logical inconsistencies, such as future birth dates, and compared with the Age variable.

6.8.3 Handling Negative Age Values

The Age variable contains several negative values, which are not logically valid. Initial examination of the corresponding Date of Birth values indicates that most of these negative ages are likely caused by sign errors. For example, an Age of -31 is associated with a Date of Birth in 1995, which is consistent with an age of approximately 31 years.

Therefore, negative ages will not be removed immediately. They will first be compared with the corresponding Date of Birth values. Negative ages that can be validated using a reasonable Date of Birth will be corrected by removing the negative sign. Negative ages that cannot be validated because the Date of Birth is missing or invalid will be treated as missing values.

6.8.4 Verification of Age Cleaning

After handling the invalid Age values, the dataset was checked to ensure that no negative ages remained.

The verification returned zero records with negative Age values. This confirms that the identified negative ages were either corrected using the corresponding Date of Birth or converted to missing values when they could not be reliably validated.

Similarly, ages above 100 years were previously identified as implausible and converted to missing values. The cleaned Age variable can now be used for further validation and exploratory analysis.

6.9.5 Converting Monthly Contribution to Numeric

The Monthly_Contribution_INR variable is stored as an object data type even though it represents a quantitative monetary amount.

The variable will be converted to a numeric data type so that missing values, invalid values, and potential outliers can be identified correctly.

6.9.9 Converting Years of Contribution to Numeric

The Years_of_Contribution variable is currently stored as an object data type even though it represents a quantitative duration.

The variable will be converted to a numeric data type so that its values can be checked for missing, negative, and logically inconsistent observations.

6.9.12 Converting Total Contribution to Numeric

The Total_Contribution_Till_Date_INR variable is currently stored as an object data type even though it represents a quantitative monetary amount.

The variable will be converted to a numeric data type so that invalid values, missing values, and potential outliers can be identified accurately.

6.9.14 Verification of Current NAV

After replacing the invalid negative NAV values with missing values, the variable was checked again.

The verification returned no negative observations, confirming that all remaining Current NAV values are non-negative and suitable for further analysis.

6.9.18 Validation of Expected Monthly Pension

The Expected Monthly Pension variable is already stored in numeric format. The variable contains negative values and an extremely large maximum value.

Negative pension values are logically invalid and will be treated as missing values. The extremely large values will be investigated separately before deciding whether they represent valid observations or data-quality errors.

The large difference between the mean and median indicates that extreme observations are strongly influencing the distribution. Therefore, the mean will not be used automatically for replacing extreme values.

6.9.21 Validation of Years to Retirement

The Years to Retirement variable is already stored in numeric format. The variable ranges from -31 to 37 years, with a median of 14 years.

Negative values are logically invalid because an individual cannot have a negative number of years remaining until retirement. These observations will therefore be treated as data-quality errors and converted to missing values.

The maximum value of 37 years is not inherently implausible and will be retained for further analysis.

6.9.22 Verification of Years to Retirement

After replacing the invalid negative values with missing values, the Years to Retirement variable was checked again.

The verification returned no negative observations, confirming that all remaining values are non-negative and suitable for further analysis.

6.9.23 Validation of Asset Allocation Percentages

The asset allocation variables represent the percentage of pension fund assets allocated to different asset classes. Therefore, valid values must fall between 0% and 100%.

The following variables will be checked for logically invalid negative values or values exceeding 100%:

- Asset_Allocation_Equity_E_Percent
- Asset_Allocation_CorporateDebt_C_Percent
- Asset_Allocation_GovtSecurities_G_Percent
- Asset_Allocation_Alternative_A_Percent

Invalid percentage values will be treated as missing values rather than replacing them with statistical estimates.

```
allocation_cols = [
    "Asset_Allocation_Equity_E_Percent",
    "Asset_Allocation_CorporateDebt_C_Percent",
    "Asset_Allocation_GovtSecurities_G_Percent",
    "Asset_Allocation_Alternative_A_Percent"
]


df[allocation_cols].describe()
```

	Asset_Allocation_Equity_E_Percent	Asset_Allocation_CorporateDebt_C_Percent	Asset_Allocation_GovtSecurities_G_Percent	Asset_Allocation_Alternative_A_Percent
count	5537.000000	5534.000000	5543.000000	5539.000000
mean	36.667329	32.551319	23.920440	6.558585

	Asset_Allocation_Equity_E_Percent	Asset_Allocation_CorporateDebt_C_Percent	Asset_Allocation_GovtSecurities_G_Percent	Asset_Allocation_Alternative_A_Percent
std	16.115804	8.530433	14.557951	6.302701
min	-50.000000	-60.000000	-45.000000	-20.000000
25%	15.000000	30.000000	15.000000	5.000000
50%	35.000000	35.000000	25.000000	5.000000
75%	50.000000	35.000000	45.000000	5.000000
max	77.000000	77.000000	66.000000	59.000000

6.9.24 Handling Invalid Asset Allocation Values

The asset allocation variables contain negative percentage values, which are logically invalid because an asset allocation percentage cannot be below 0%.

The maximum values for all four allocation variables are below 100%, so no values exceed the upper logical limit.

Therefore, negative allocation percentages will be treated as missing values, while valid values between 0% and 100% will be retained.

```
[146]: for col in allocation_cols:
           df.loc[df[col] < 0, col] = np.nan

[147]: (df[allocation_cols] < 0).sum()
```

6.9.25 Verification of Asset Allocation Percentages

After replacing invalid negative allocation percentages with missing values, all four asset allocation variables were checked again.

The verification returned zero negative observations for Equity, Corporate Debt, Government Securities, and Alternative Investments. Therefore, no negative allocation percentages remain in the dataset.

```
[148]: df.isna().sum().sort_values(ascending=False)

[148]: Annual_Income_INR                          356
       Monthly_Contribution_INR                   272
       Total_Contribution_Till_Date_INR           250
       Age                                        247
       Years_of_Contribution                      222
       Date_of_Birth                              128
       Nominee_Name                               101
       Employer_Name                              100
       Total_Units_Held                            89
       Expected_Monthly_Pension_INR                89
       Years_to_Retirement                         89
       Current_NAV_INR                             89
       Date_of_Joining                             84
       Employer_Monthly_Contribution_INR           84
       Last_Transaction_Date                       84
       Tax_Benefit_Section_Claimed                 70
       Nominee_Relationship                        67
       Bank_Name                                   67
       Investment_Choice                           67
       Current_Fund_Value_INR                      67
       Pension_Fund_Manager                        67
       Account_Status                              56
       KYC_Status                                  56
       IFSC_Code                                   56
       Withdrawal_Status                           56
       Occupation_Group                            56
       Sector                                      56
       Occupation_Type                             56
       PAN_Linked                                  56
       Aadhaar_Linked                              56
       City                                        56
       Gender                                      55
       Marital_Status                              54
       Asset_Allocation_Alternative_A_Percent      33
       Asset_Allocation_CorporateDebt_C_Percent    28
       Asset_Allocation_Equity_E_Percent           20
       Asset_Allocation_GovtSecurities_G_Percent   16
       Pension_Scheme_Type                          1
       State                                        1
       dtype: int64
```

6.9.26 Final Missing-Value Review After Data Cleaning¶

A final missing-value assessment was performed after correcting the identified data-quality issues.

The number of missing values increased for several numeric variables because logically invalid observations, such as negative income, negative contributions, negative NAV, negative units, negative pension values, and invalid asset allocation percentages, were converted to missing values.

The largest remaining missingness is observed in Annual Income, Monthly Contribution, Total Contribution Till Date, Age, and Years of Contribution. These missing values will be considered in the subsequent analysis and will not be filled automatically without considering the nature of each variable.

Categorical variables generally contain a smaller number of missing observations, while State and Pension Scheme Type have only one missing value each.

The cleaned dataset will now proceed to outlier analysis, where statistically extreme but potentially valid observations will be distinguished from logically invalid values already addressed during data cleaning

[149]:
```
numeric_outlier_cols = [
    "Annual_Income_INR",
    "Monthly_Contribution_INR",
    "Employer_Monthly_Contribution_INR",
    "Total_Contribution_Till_Date_INR",
    "Current_NAV_INR",
    "Total_Units_Held",
    "Current_Fund_Value_INR",
    "Expected_Monthly_Pension_INR"
]

outlier_summary = pd.DataFrame(index=numeric_outlier_cols)

for col in numeric_outlier_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outlier_summary.loc[col, "Q1"] = Q1
    outlier_summary.loc[col, "Q3"] = Q3
    outlier_summary.loc[col, "Lower_Bound"] = lower
    outlier_summary.loc[col, "Upper_Bound"] = upper
    outlier_summary.loc[col, "Outlier_Count"] = (
        (df[col] < lower) | (df[col] > upper)
    ).sum()

outlier_summary
```

[149]:

	Q1	Q3	Lower_Bound	Upper_Bound	Outlier_Count
Annual_Income_INR	398000.00000	1.448800e+06	-1.178200e+06	3.025000e+06	0.0
Monthly_Contribution_INR	1250.00000	8.380000e+03	-9.445000e+03	1.907500e+04	0.0
Employer_Monthly_Contribution_INR	0.00000	8.710000e+03	-1.306500e+04	2.177500e+04	0.0
Total_Contribution_Till_Date_INR	94800.00000	1.145900e+06	-1.481850e+06	2.722550e+06	493.0
Current_NAV_INR	29.46270	5.334360e+01	-6.358650e+00	8.916495e+01	0.0
Total_Units_Held	2787.63755	3.685776e+04	-4.831755e+04	8.796295e+04	489.0
Current_Fund_Value_INR	115003.13250	1.397464e+06	-1.808688e+06	3.321154e+06	506.0
Expected_Monthly_Pension_INR	1428.06750	6.959650e+03	-6.869306e+03	1.525702e+04	584.0

[150]:
```
Q1 = df["Expected_Monthly_Pension_INR"].quantile(0.25)
Q3 = df["Expected_Monthly_Pension_INR"].quantile(0.75)
IQR = Q3 - Q1

upper = Q3 + 1.5 * IQR

df.loc[
    df["Expected_Monthly_Pension_INR"] > upper,
    "Expected_Monthly_Pension_INR"
].sort_values(ascending=False).head(20)
```

[150]:
```
1138    45709.44
2174    43434.95
4140    43353.94
2105    42745.15
987     42668.78
247     42054.43
5079    40162.71
5292    39684.13
459     39488.70
2180    39479.93
890     39405.06
107     38927.68
4797    38910.21
5259    38874.01
3857    38330.00
1463    38322.96
5360    38224.25
4661    38049.53
2453    37763.90
874     37517.33
Name: Expected_Monthly_Pension_INR, dtype: float64
```

6.9.27 Review of Expected Monthly Pension Outliers

The IQR method identified 584 observations as statistical outliers for Expected Monthly Pension.

Inspection of the highest values shows that the extreme observations now range from approximately ₹37,500 to ₹45,700 per month. These values are substantially higher than the median but are not inherently impossible for pension subscribers.

The previously identified ₹9,999,999 data-quality values were already corrected separately. Therefore, the remaining statistical outliers are considered potentially valid extreme observations and will be retained rather than removed.

[151]:
```
Q1 = df["Total_Contribution_Till_Date_INR"].quantile(0.25)
Q3 = df["Total_Contribution_Till_Date_INR"].quantile(0.75)
IQR = Q3 - Q1

upper = Q3 + 1.5 * IQR

df.loc[
    df["Total_Contribution_Till_Date_INR"] > upper,
    "Total_Contribution_Till_Date_INR"
].sort_values(ascending=False).head(20)
```

[151]:
```
818     7904820.0
39      7378360.0
1138    6829100.0
4797    6435600.0
4140    6433700.0
987     6298300.0
459     6163200.0
934     6150540.0
2105    6143900.0
4591    6129900.0
3857    6116200.0
5292    6089200.0
2174    6083400.0
271     6072000.0
3606    6071800.0
2455    6048900.0
517     5959700.0
182     5930900.0
247     5916600.0
5259    5914800.0
Name: Total_Contribution_Till_Date_INR, dtype: float64
```

6.9.28 Review of Total Contribution Outliers

The IQR method identified 493 statistical outliers for Total Contribution Till Date.

Inspection of the highest values shows contributions ranging from approximately ₹5.9 million to ₹7.9 million. These values are high relative to the central distribution but are not inherently impossible for cumulative pension contributions.

Therefore, the identified statistical outliers will be retained as potentially valid observations. No automatic deletion or mean/median replacement will be applied.

[152]:
```
Q1 = df["Current_Fund_Value_INR"].quantile(0.25)
Q3 = df["Current_Fund_Value_INR"].quantile(0.75)
IQR = Q3 - Q1

upper = Q3 + 1.5 * IQR

df.loc[
    df["Current_Fund_Value_INR"] > upper,
    "Current_Fund_Value_INR"
].sort_values(ascending=False).head(20)
```

[152]:
```
97      17690629.17
2968    12559081.50
4192    10049945.64
845      9729559.50
1138     9141888.94
2174     8686989.27
2105     8549029.41
987      8533755.38
247      8410885.78
5079     8032541.44
5292     7936826.76
459      7897739.82
2180     7895985.91
890      7881012.34
107      7785536.94
4797     7782042.27
5259     7774801.07
3857     7665999.86
1463     7664592.99
5360     7644850.13
Name: Current_Fund_Value_INR, dtype: float64
```

6.9.29 Review of Current Fund Value Outliers

The IQR method identified 506 statistical outliers for Current Fund Value.

Inspection of the highest observations shows fund values ranging from approximately ₹76 lakh to ₹1.77 crore among the most extreme observations. Although these values are substantially higher than the central distribution, they are not inherently impossible for accumulated pension fund values.

Therefore, the identified statistical outliers are considered potentially valid high-value observations and will be retained. No automatic deletion or mean/median replacement will be applied.

[153]:
```
Q1 = df["Total_Units_Held"].quantile(0.25)
Q3 = df["Total_Units_Held"].quantile(0.75)
IQR = Q3 - Q1

upper = Q3 + 1.5 * IQR

df.loc[
    df["Total_Units_Held"] > upper,
    "Total_Units_Held"
].sort_values(ascending=False).head(20)
```

[153]:
```
1138    497921.5222
5259    396622.9343
1463    382247.2740
5079    370549.1175
5524    365473.3106
874     357459.4404
3677    334585.6707
5       303721.1860
987     301012.1756
2174    295122.4816
107     289814.0977
805     278624.3850
800     277006.4335
1476    270393.5393
2453    266464.5969
1648    266026.9378
5418    261371.4741
1591    260775.9488
655     260279.9207
663     257853.5941
Name: Total_Units_Held, dtype: float64
```

6.9.30 Review of Total Units Held Outliers

The IQR method identified 489 statistical outliers for Total Units Held.

Inspection of the highest observations shows values ranging from approximately 257,854 to 497,922 units among the most extreme observations. Although these values are statistically high relative to the central distribution, they are not inherently impossible.

Therefore, the identified statistical outliers are retained as potentially valid observations. No automatic deletion or mean/median replacement is applied.

[154]:
```
df[[
    "Total_Units_Held",
    "Current_NAV_INR",
    "Current_Fund_Value_INR"
]].head(10)
```

[154]:

	Total_Units_Held	Current_NAV_INR	Current_Fund_Value_INR
0	17694.7854	48.4636	857553.00
1	43411.7083	NaN	1008922.83
2	13061.6514	52.2828	682899.71
3	1734.5708	39.1377	67887.11
4	2310.9092	25.2364	58319.03
5	303721.1860	20.8156	6322138.72
6	69620.5931	34.2882	2387164.82
7	18123.9008	25.4715	461642.94
8	20136.3980	25.2813	509074.32
9	2314.1750	60.1085	139101.59

[155]:
```
df["Calculated_Fund_Value_INR"] = (
    df["Total_Units_Held"] * df["Current_NAV_INR"]
)

df["Fund_Value_Difference"] = (
    df["Current_Fund_Value_INR"] - df["Calculated_Fund_Value_INR"]
)

df[[
    "Total_Units_Held",
    "Current_NAV_INR",
    "Current_Fund_Value_INR",
    "Calculated_Fund_Value_INR",
    "Fund_Value_Difference"
]].head(10)
```

[155]:

	Total_Units_Held	Current_NAV_INR	Current_Fund_Value_INR	Calculated_Fund_Value_INR	Fund_Value_Difference
0	17694.7854	48.4636	857553.00	8.575530e+05	-0.001711
1	43411.7083	NaN	1008922.83	NaN	NaN
2	13061.6514	52.2828	682899.71	6.828997e+05	0.002184
3	1734.5708	39.1377	67887.11	6.788711e+04	-0.001599
4	2310.9092	25.2364	58319.03	5.831903e+04	0.001065
5	303721.1860	20.8156	6322138.72	6.322139e+06	0.000698
6	69620.5931	34.2882	2387164.82	2.387165e+06	-0.000331
7	18123.9008	25.4715	461642.94	4.616429e+05	0.000773
8	20136.3980	25.2813	509074.32	5.090743e+05	0.001243
9	2314.1750	60.1085	139101.59	1.391016e+05	0.002012

6.9.31 Consistency Check of Current Fund Value

The relationship between Total Units Held, Current NAV, and Current Fund Value was examined using the calculation:

Current Fund Value = Total Units Held × Current NAV

The calculated values closely match the recorded Current Fund Value, with only very small differences attributable to decimal precision and rounding.

This confirms that the three variables are internally consistent for observations where the required values are available. No correction is required for Current Fund Value based on this consistency check.

[156]:
```
df.drop(
    columns=["Calculated_Fund_Value_INR", "Fund_Value_Difference"],
    inplace=True
)
```

[157]:
```
df[[
    "Years_of_Contribution",
    "Monthly_Contribution_INR",
    "Total_Contribution_Till_Date_INR"
]].head(10)
```

[157]:

	Years_of_Contribution	Monthly_Contribution_INR	Total_Contribution_Till_Date_INR
0	5.5	9570.0	608400.0
1	3.7	9530.0	941400.0
2	7.2	8230.0	660200.0
3	1.9	1250.0	55100.0
4	1.8	1250.0	47300.0
5	13.6	16480.0	5572300.0
6	10.4	6460.0	1888000.0
7	4.7	6740.0	353300.0
8	15.8	1250.0	469100.0
9	7.6	1230.0	110500.0

[158]:
```
df[[
    "Years_of_Contribution",
    "Monthly_Contribution_INR",
    "Employer_Monthly_Contribution_INR",
    "Total_Contribution_Till_Date_INR"
]].head(10)
```

[158]:

	Years_of_Contribution	Monthly_Contribution_INR	Employer_Monthly_Contribution_INR	Total_Contribution_Till_Date_INR
0	5.5	9570.0	0.0	608400.0
1	3.7	9530.0	13340.0	941400.0
2	7.2	8230.0	0.0	660200.0
3	1.9	1250.0	1250.0	55100.0
4	1.8	1250.0	1250.0	47300.0
5	13.6	16480.0	16480.0	5572300.0
6	10.4	6460.0	9040.0	1888000.0
7	4.7	6740.0	0.0	353300.0
8	15.8	1250.0	1250.0	469100.0
9	7.6	1230.0	0.0	110500.0

6.9.32 Consistency Check of Total Contributions

The relationship between Years of Contribution, employee monthly contribution, employer monthly contribution, and Total Contribution Till Date was reviewed.

The observed total contributions are broadly consistent with the contribution period and monthly contribution amounts. Differences are expected because monthly contributions may have changed over time and the recorded total may reflect the actual contribution history.

No obvious logical inconsistency was identified from this preliminary check. Therefore, the contribution variables are retained without further modification.

[159]:
```
df.duplicated().sum()
```

[159]:
```
np.int64(4)
```

6.9.33 Identification of Duplicate Records

A duplicate-record check identified 4 rows that are exact duplicates of other observations.

Exact duplicate records do not provide additional information and may lead to double-counting during statistical analysis. Therefore, the duplicate rows will be removed while retaining the first occurrence of each record.

```
[160]: df = df.drop_duplicates(keep="first").reset_index(drop=True)

[161]: df.duplicated().sum()

[161]: np.int64(0)
```

Removal of Duplicate Records

The duplicate rows were removed while retaining the first occurrence of each record. The index was then reset to maintain a continuous row index.

A final duplicate check confirmed that 0 duplicate rows remained.

6.9.34 Standardization of Date Fields

The date fields were standardized to a consistent YYYY-MM-DD format. The conversion removes the unnecessary time component (for example, 00:00:00) from date values while preserving unavailable dates as missing values.

```
[162]: df["Date_of_Birth"] = pd.to_datetime(
           df["Date_of_Birth"], errors="coerce"
       ).dt.strftime("%Y-%m-%d")

       df["Date_of_Joining"] = pd.to_datetime(
           df["Date_of_Joining"], errors="coerce"
       ).dt.strftime("%Y-%m-%d")

       df["Last_Transaction_Date"] = pd.to_datetime(
           df["Last_Transaction_Date"], errors="coerce"
       ).dt.strftime("%Y-%m-%d")

[163]: df[["Date_of_Birth", "Date_of_Joining", "Last_Transaction_Date"]].head(10)
```

[163]:

	Date_of_Birth	Date_of_Joining	Last_Transaction_Date
0	1969-01-26	2021-01-09	2026-04-14
1	1998-09-04	NaN	2026-04-22
2	1967-06-05	NaN	2026-01-22
3	1958-11-03	NaN	NaN
4	1982-07-18	NaN	NaN
5	1965-10-05	NaN	2026-06-07
6	1958-08-11	NaN	2026-01-05
7	1980-11-19	NaN	2025-04-26
8	1975-12-18	NaN	2026-01-19
9	1984-03-01	NaN	2025-10-09

6.9.35 Outlier Detection Using Boxplots

Boxplots were used to visually examine the distribution of the numerical variables and identify potential outliers. The boxplot displays the median, first quartile (Q1), third quartile (Q3), and the interquartile range (IQR).

Observations falling below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR are displayed as individual points and are considered potential outliers.

The boxplots were used as a preliminary diagnostic rather than as an automatic basis for removing observations. Extreme values may represent genuine differences between individuals in income, contributions, pension fund value, NAV, units held, or other financial variables. Therefore, potential outliers were reviewed in the context of the dataset and were not removed solely because they appeared outside the boxplot whiskers.

[167]:
```python
# 6.9.35 Outlier Detection Using Boxplots

import matplotlib.pyplot as plt
import seaborn as sns

boxplot_columns = [
    "Annual_Income_INR",
    "Monthly_Contribution_INR",
    "Employer_Monthly_Contribution_INR",
    "Total_Contribution_Till_Date_INR",
    "Current_Fund_Value_INR",
    "Expected_Monthly_Pension_INR",
    "Current_NAV_INR",
    "Total_Units_Held",
    "Years_of_Contribution",
    "Years_to_Retirement"
]

fig, axes = plt.subplots(2, 5, figsize=(20, 11))

colors = sns.color_palette("Set2", 10)

for ax, column, color in zip(axes.flatten(), boxplot_columns, colors):

    sns.boxplot(
        y=df[column],
        ax=ax,
        color=color,
        width=0.55
    )

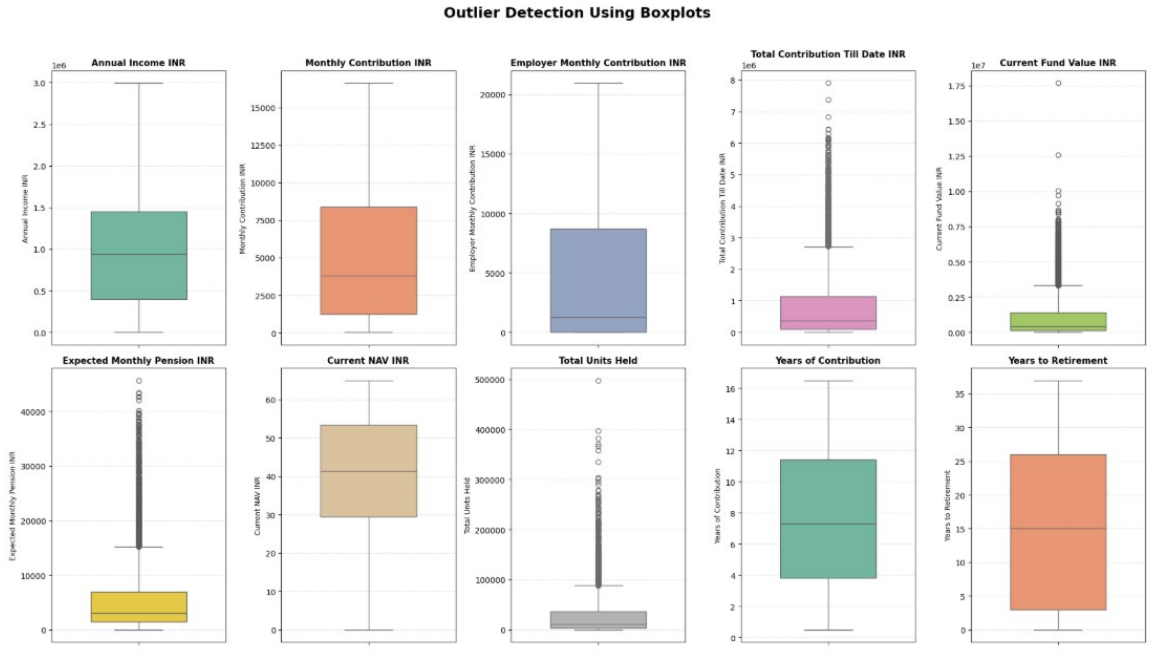
    ax.set_title(
        column.replace("_", " "),
        fontsize=11,
        fontweight="bold"
    )

    ax.set_xlabel("")
    ax.set_ylabel(column.replace("_", " "), fontsize=9)

    ax.grid(
        axis="y",
        linestyle="--",
        alpha=0.35
    )
```

```python
fig.suptitle(
    "Outlier Detection Using Boxplots",
    fontsize=18,
    fontweight="bold",
    y=1.02
)

plt.tight_layout()
plt.show()
```



6.9.36 Why Potential Outliers Were Not Removed

The boxplots identified several observations that lie beyond the upper whiskers and may therefore be considered statistical outliers. However, these observations were **not removed automatically**.

In a pension dataset, unusually high values for income, contributions, total contributions, current fund value, pension amount, or units held may represent genuine differences between individuals rather than data-entry errors. Removing such observations solely because they fall outside the IQR-based boxplot limits could result in the loss of meaningful financial information.

Therefore, the boxplots were used as a **diagnostic tool to identify unusual observations**, rather than as a rule for deleting records. Observations were retained unless there was a clear data-quality reason to consider them invalid.

This approach preserves the original variation in the dataset and avoids introducing bias by removing legitimate high-value pension records.

7. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset before performing detailed statistical analysis or drawing conclusions from the data.

The main purpose of EDA is to understand the structure, distribution, patterns, relationships, and potential irregularities present in the dataset. It helps identify important characteristics of the data and provides a foundation for selecting appropriate analytical methods and visualisations.

For the India Pension Scheme dataset, EDA is used to understand the demographic characteristics of pension account holders, their pension scheme participation, contribution patterns, financial outcomes, investment preferences, and relationships between important variables.

The EDA is organised into the following stages:

1. **Descriptive Analysis** – summarises the numerical characteristics of the dataset.
2. **Categorical Analysis** – examines the distribution of important categorical variables.
3. **Univariate Analysis** – studies individual variables and their distributions.
4. **Bivariate Analysis** – examines relationships between two variables.
5. **Multivariate Analysis** – examines patterns involving three or more variables.
6. **Correlation Analysis** – measures the strength and direction of relationships between numerical variables.

The analysis is primarily descriptive and exploratory. Therefore, observed patterns and relationships are interpreted as characteristics of the dataset and are not automatically treated as causal relationships.

7.1 Descriptive Analysis

Descriptive analysis provides a numerical summary of the dataset. It helps describe the central tendency, variability, and range of numerical variables without attempting to make predictions or establish causal relationships.

For numerical variables, measures such as the mean, median, standard deviation, minimum, maximum, and quartiles are examined. These measures help identify the typical values in the dataset, the amount of variation between observations, and the presence of unusually large or small values.

In the pension dataset, descriptive analysis is particularly useful for variables such as age, annual income, monthly contribution, total contribution, current fund value, expected monthly pension, and years of contribution.

This stage provides an initial numerical understanding of the dataset before visual analysis is performed.

[170]:
```
# 7.1 Descriptive Analysis

# Select important numerical variables for descriptive analysis
descriptive_columns = [
    "Age",
    "Years_of_Contribution",
    "Years_to_Retirement",
    "Annual_Income_INR",
    "Monthly_Contribution_INR",
    "Employer_Monthly_Contribution_INR",
    "Total_Contribution_Till_Date_INR",
    "Current_Fund_Value_INR",
    "Expected_Monthly_Pension_INR",
    "Current_NAV_INR",
    "Total_Units_Held"
]

# Generate descriptive statistics
descriptive_stats = df[descriptive_columns].describe().T

# Add median separately because describe() does not include it
descriptive_stats["median"] = df[descriptive_columns].median()

# Arrange columns in a logical order
descriptive_stats = descriptive_stats[
    ["count", "mean", "median", "std", "min", "25%", "50%", "75%", "max"]
]

# Display the results
descriptive_stats
```

[170]:

	count	mean	median	std	min	25%	50%	75%	max
Age	5302.0	4.542286e+01	46.00000	1.322889e+01	23.0000	34.000000	46.00000	5.700000e+01	6.800000e+01
Years_of_Contribution	5327.0	7.706307e+00	7.30000	4.539820e+00	0.5000	3.800000	7.30000	1.140000e+01	1.650000e+01
Years_to_Retirement	5460.0	1.537656e+01	15.00000	1.217665e+01	0.0000	3.000000	15.00000	2.600000e+01	3.700000e+01
Annual_Income_INR	5193.0	9.681649e+05	936050.00000	6.194831e+05	1.0000	397700.000000	936050.00000	1.447600e+06	2.995800e+06
Monthly_Contribution_INR	5277.0	5.017021e+03	3820.00000	4.489424e+03	42.0000	1250.000000	3820.00000	8.380000e+03	1.663000e+04
Employer_Monthly_Contribution_INR	5465.0	4.576759e+03	1250.00000	6.094283e+03	0.0000	0.000000	1250.00000	8.700000e+03	2.096000e+04
Total_Contribution_Till_Date_INR	5299.0	8.689211e+05	360400.00000	1.185298e+06	300.0000	94650.000000	360400.00000	1.144700e+06	7.904820e+06
Current_Fund_Value_INR	5482.0	1.068991e+06	428862.25500	1.491874e+06	307.3100	114844.415000	428862.25500	1.396640e+06	1.769063e+07
Expected_Monthly_Pension_INR	5460.0	5.877174e+03	3000.00000	7.062988e+03	0.5000	1428.067500	3000.00000	6.941452e+03	4.570944e+04

Key Observations

- The median age of pension account holders is **46 years**, while the median years of contribution is **7.3 years**.
- The median annual income is approximately **₹9.36 lakh**, and the median monthly contribution is **₹3,820**.
- The median total contribution till date is approximately **₹3.60 lakh**, while the median current fund value is approximately **₹4.29 lakh**.
- The median expected monthly pension is **₹3,000**.

- The mean current fund value (**₹10.69 lakh**) is considerably higher than the median (**₹4.29 lakh**), indicating a **right-skewed distribution** with some accounts having substantially higher fund values.
- Annual income, total contribution, current fund value, and total units held show a **wide range of values**, indicating considerable variation among pension account holders.
- The maximum current fund value is approximately **₹1.77 crore**, substantially higher than the median, highlighting the presence of high-value pension accounts.
- The number of observations (count) varies across numerical variables because of **missing values** in the dataset.
- Overall, the descriptive statistics indicate substantial variation in the **demographic, contribution, and financial characteristics** of the pension account holders.

7.2 Categorical Analysis

Categorical analysis examines variables that contain distinct groups or categories rather than continuous numerical values.

The purpose of categorical analysis is to understand how observations are distributed across different groups. Frequency counts and percentages can be used to identify the most common and least common categories.

For the India Pension Scheme dataset, important categorical variables include gender, marital status, state, sector, pension scheme type, investment choice, pension fund manager, account status, occupation type, and occupation group.

Understanding these distributions is important because it provides an overview of the composition of the pension dataset and helps identify which groups are most represented.

The categorical analysis is descriptive and does not imply that a category is inherently better or worse than another category.

[171]:
```python
# 7.2 Categorical Analysis

categorical_columns = [
    "Gender",
    "Marital_Status",
    "Sector",
    "Pension_Scheme_Type",
    "Investment_Choice",
    "Pension_Fund_Manager",
    "Account_Status",
    "Occupation_Type",
    "Occupation_Group"
]

for column in categorical_columns:

    print(f"\n{'=' * 70}")
    print(f"{column.replace('_', ' ')}")
    print(f"{'=' * 70}")

    category_summary = (
        df[column]
        .value_counts(dropna=False)
        .rename_axis("Category")
        .reset_index(name="Count")
    )

    category_summary["Percentage"] = (
        category_summary["Count"] / len(df) * 100
    ).round(2)

    display(category_summary)
```

==========

Gender

==========

	Category	Count	Percentage
0	Male	3188	57.45
1	Female	2241	40.39
2	Other	65	1.17
3	<NA>	55	0.99

==========

Marital Status

==========

	Category	Count	Percentage
0	Married	3354	60.44
1	Single	1583	28.53
2	Widowed	374	6.74
3	Divorced	154	2.78
4	<NA>	54	0.97
5	Unknown	30	0.54

==========

Sector

==========

	Category	Count	Percentage
0	All Citizen Model	1231	22.18
1	Unorganized Sector	1109	19.99
2	Organized Private Sector	1041	18.76
3	Central/State Government	943	16.99
4	Corporate	805	14.51
5	Central Government	364	6.56

	Category	Count	Percentage
6	<NA>	56	1.01

==========

Pension Scheme Type

==========

	Category	Count	Percentage
0	NPS - All Citizen	1246	22.45
1	Atal Pension Yojana	1118	20.15
2	Employees' Pension Scheme	1049	18.9
3	NPS - Government	953	17.17
4	NPS - Corporate	818	14.74
5	Unified Pension Scheme	364	6.56
6	<NA>	1	0.02

==========

Investment Choice

==========

	Category	Count	Percentage
0	Auto Choice - Moderate	1399	25.21
1	Auto Choice - Conservative	1383	24.92
2	Active Choice	1362	24.54
3	Auto Choice - Aggressive	1338	24.11
4	<NA>	67	1.21

==========

Pension Fund Manager

==========

	Category	Count	Percentage
0	Tata Pension Management	583	10.51
1	Kotak Mahindra Pension Fund	573	10.33
2	Uti Retirement Solutions	572	10.31
3	Aditya Birla Sun Life Pension	569	10.25
4	Sbi Pension Funds	553	9.97
5	Lic Pension Fund	553	9.97
6	Axis Pension Fund	542	9.77

	Category	Count	Percentage
	Management		
7	Max Life Pension Fund	538	9.7
8	Hdfc Pension Management	506	9.12
9	Icici Prudential Pension Fund	493	8.88
10	<NA>	67	1.21

==========

Account Status

==========

	Category	Count	Percentage
0	Active	4344	78.28
1	Inactive	695	12.52
2	Dormant	298	5.37
3	Frozen	156	2.81
4	<NA>	56	1.01

==========

Occupation Type

==========

	Category	Count	Percentage
0	Private Sector Employee	1755	31.63
1	Government Employee	1303	23.48
2	Factory Worker	485	8.74
3	Self-Employed	444	8.0
4	Professional	401	7.23
5	Domestic Worker	247	4.45
6	Driver	231	4.16
7	Vendor	225	4.05
8	Daily Wage Worker	215	3.87
9	Farmer	187	3.37
10	<NA>	56	1.01

==========

Occupation Group

==========

	Category	Count	Percentage
0	Private	1755	31.63

	Category	Count	Percentage
	Sector		
1	Other/ Informal	1590	28.65
2	Government	1303	23.48
3	Self-Employed	444	8.00
4	Professional	401	7.23
5	NaN	56	1.01

Key Observations

- **Gender:** Male account holders form the largest group at **57.45%**, followed by females at **40.39%**. The Other category represents **1.17%** of the dataset.
- **Marital Status: Married** account holders are the largest group at **60.44%**, followed by **Single** at **28.53%**.
- **Sector:** The **All Citizen Model** has the largest representation at **22.18%**, followed by the **Unorganized Sector (19.99%)** and **Organized Private Sector (18.76%)**.
- **Pension Scheme Type: NPS - All Citizen** is the most represented scheme at **22.45%**, followed by **Atal Pension Yojana (20.15%)** and **Employees' Pension Scheme (18.90%)**.
- **Investment Choice:** The four investment choices have relatively similar representation. **Auto Choice - Moderate** is the most common at **25.21%**, while **Auto Choice - Aggressive** accounts for **24.11%**.
- **Pension Fund Manager:** The distribution across fund managers is relatively balanced. **Tata Pension Management** has the highest representation at **10.51%**, while **ICICI Prudential Pension Fund** has the lowest at **8.88%** among the listed managers.
- **Account Status:** The majority of accounts are **Active (78.28%)**. Inactive accounts represent **12.52%**, while Dormant and Frozen accounts represent **5.37%** and **2.81%**, respectively.
- **Occupation Type: Private Sector Employees** form the largest occupational group at **31.63%**, followed by **Government Employees (23.48%)**.

- **Occupation Group: Private Sector** is the largest occupation group at **31.63%**, followed by **Other/Informal (28.65%)** and **Government (23.48%)**.
- Overall, the categorical distributions show that the dataset contains substantial representation from **private-sector, government, and informal-sector participants**, while the major pension schemes and investment choices are comparatively distributed across several categories.
- Missing values are relatively small across most categorical variables, although they are present and should be considered when interpreting the category percentages.

7.3 Univariate Analysis

Univariate analysis examines one variable at a time. The purpose is to understand the distribution, concentration, spread, and general pattern of an individual variable.

For numerical variables, visualisations such as histograms, density plots, and boxplots can be used to examine the shape and spread of the distribution. For categorical variables, bar charts can be used to compare the frequency of different categories.

In this pension dataset, univariate analysis will focus on important demographic, contribution, financial, and pension-related variables.

The analysis helps establish an understanding of individual variables before examining relationships between multiple variables.

7.3.1 Distribution of Age

Why This Visual Is Suitable

A **histogram with a KDE curve** is suitable for analysing age because age is a numerical variable and the main objective is to understand its distribution across the dataset.

The histogram shows how frequently account holders occur within different age ranges, while the KDE curve provides a smoother view of the overall distribution pattern.

This visual is more appropriate than a bar chart because age is a numerical variable with many possible values, and the analysis focuses on its distribution rather than comparing unrelated categories.

[173]:
```python
# 7.3.1 Distribution of Age

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))

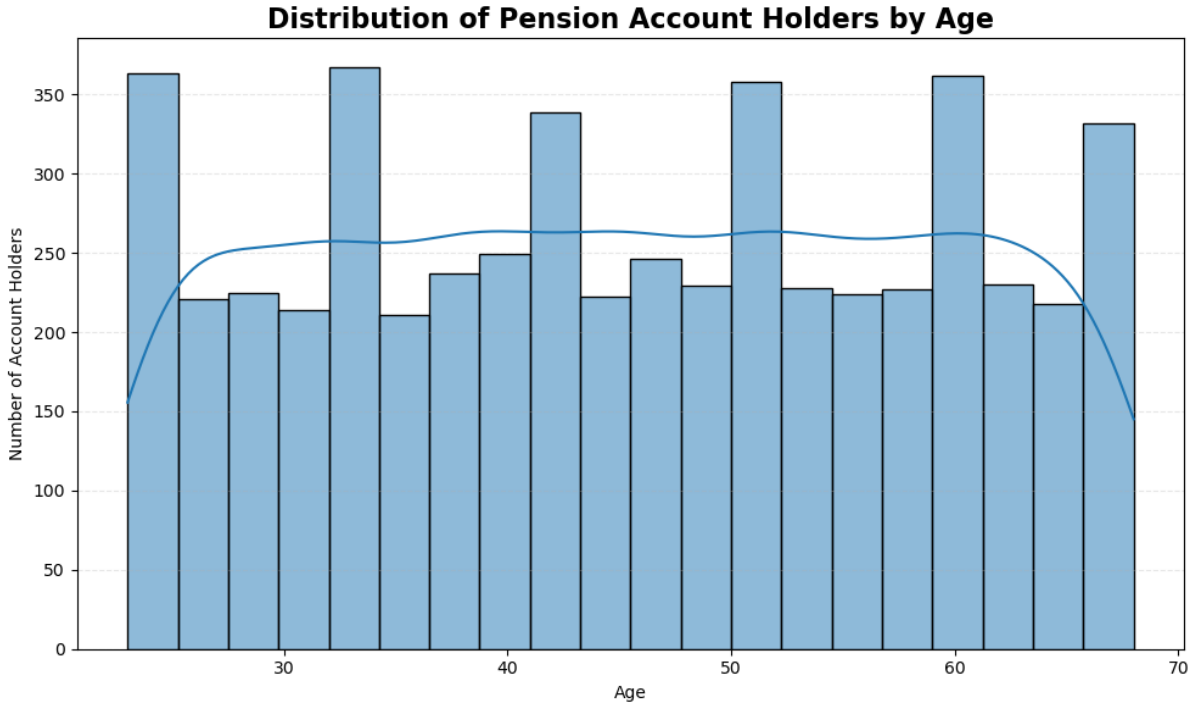
sns.histplot(
    data=df,
    x="Age",
    bins=20,
    kde=True
)

plt.title(
    "Distribution of Pension Account Holders by Age",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Age")
plt.ylabel("Number of Account Holders")

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- The age distribution covers pension account holders from approximately **23 to 68 years**.
- The observations are **fairly spread across the age range**, rather than being concentrated within a narrow age interval.
- Account holders are represented across both younger and older age groups, with observations occurring throughout the distribution.
- There is **no strong overall concentration around a single age range**, indicating a broad age composition in the dataset.

- The relatively smooth KDE curve suggests that the age distribution does not contain one dominant peak.
- The broad variation in age provides a useful basis for examining relationships between **age and pension-related financial variables** in later analyses.

7.3.2 Distribution of Annual Income

Why This Visual Is Suitable

An **ECDF (Empirical Cumulative Distribution Function)** is suitable for analysing annual income because it shows the proportion of pension account holders whose income is less than or equal to each income level.

Unlike a histogram, which depends on selected bin widths, an ECDF displays the cumulative distribution directly and makes it easier to understand the relative position of different income levels.

This visual is particularly useful for identifying how income is distributed across the dataset and whether a large proportion of account holders falls within the lower, middle, or higher income ranges.

[175]:
```
# 7.3.2 Distribution of Annual Income

plt.figure(figsize=(10, 6))

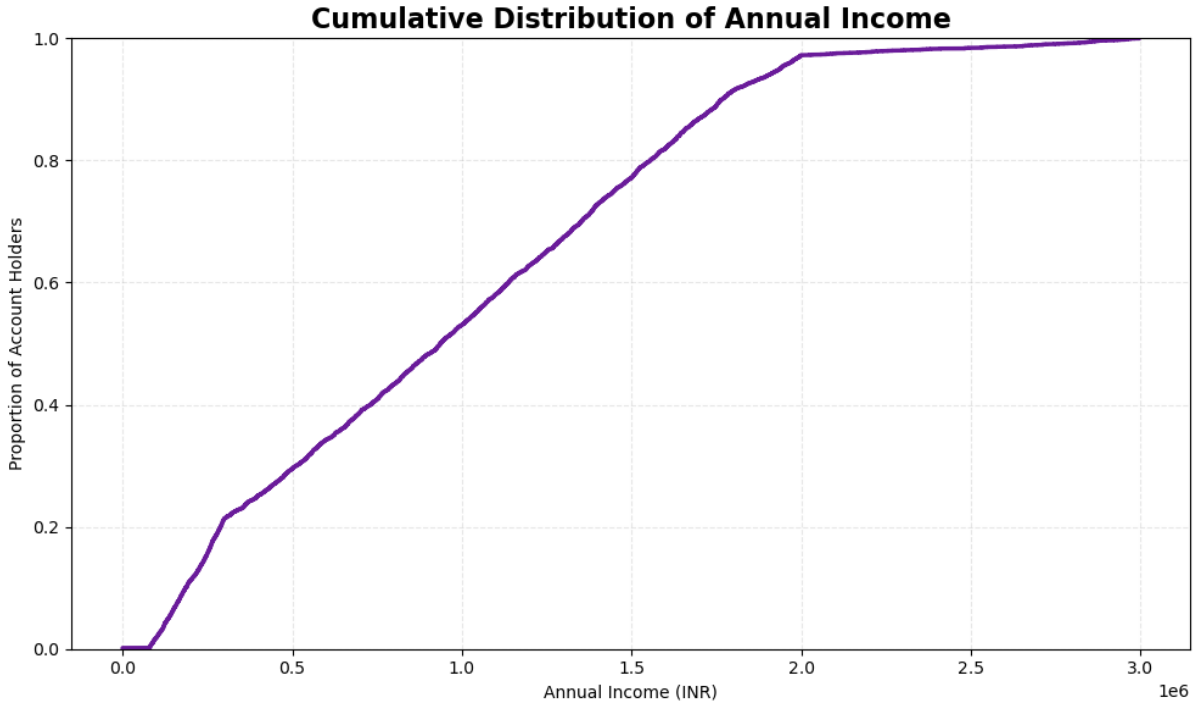
sns.ecdfplot(
    data=df,
    x="Annual_Income_INR",
    color="#6A1B9A",
    linewidth=2.5
)

plt.title(
    "Cumulative Distribution of Annual Income",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Annual Income (INR)")
plt.ylabel("Proportion of Account Holders")

plt.grid(
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- Annual income ranges from very low values to approximately **₹30 lakh** in the dataset.
- The ECDF rises steadily through the middle-income range, indicating that observations are distributed across a broad range of income levels.
- Approximately **50% of the non-missing income observations are at or below ₹10 lakh**, which is consistent with the median annual income of approximately ₹9.36 lakh.
- The curve reaches approximately **80% around ₹15 lakh**, indicating that most account holders have annual incomes below this level.
- The curve becomes much flatter after approximately **₹20 lakh**, showing that relatively fewer account holders have incomes above this level.
- The long upper tail indicates that a smaller proportion of account holders have substantially higher annual incomes.
- The ECDF therefore provides evidence of a **right-skewed income distribution**, consistent with the difference between the mean and median income observed in the descriptive statistics.

7.3.3 Distribution of Monthly Contribution

Why This Visual Is Suitable

A **violin plot** is suitable for analysing monthly contribution because it combines information about the distribution and density of a numerical variable.

The width of the violin indicates where observations are more concentrated, while the internal boxplot shows the median and interquartile range.

This makes the violin plot useful for identifying the typical contribution level, concentration of observations, and the degree of variation in monthly contributions.

[176]:
```python
# 7.3.3 Distribution of Monthly Contribution

plt.figure(figsize=(10, 6))

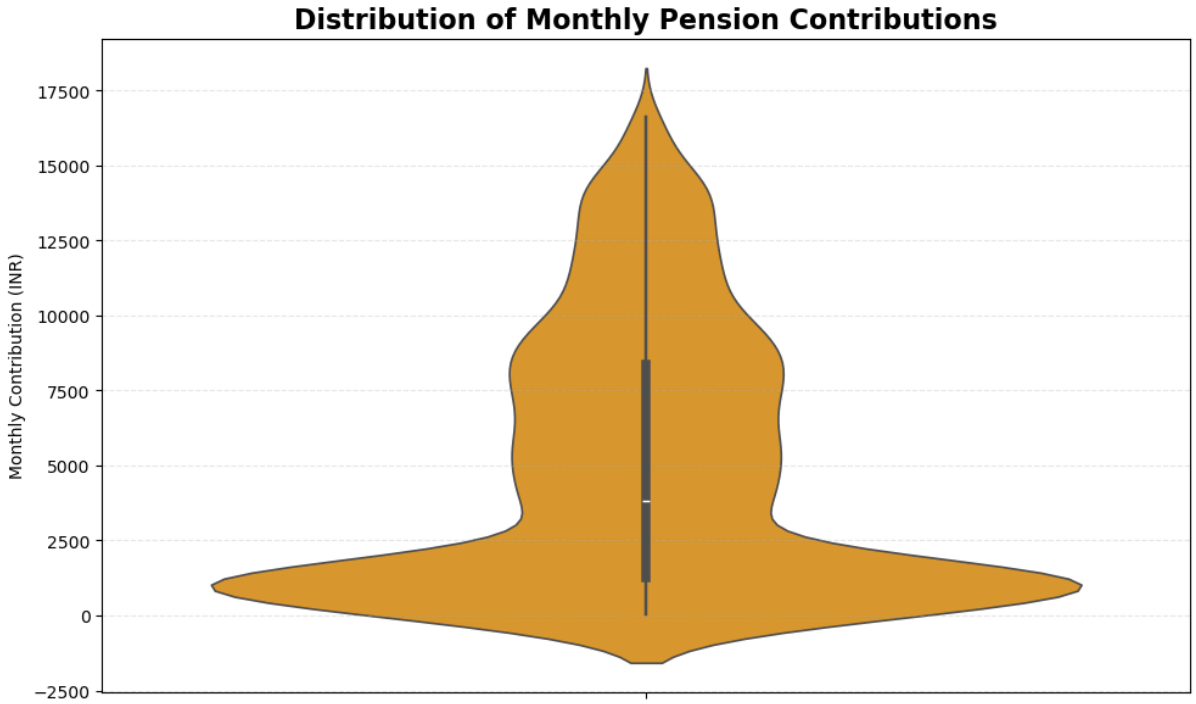
sns.violinplot(
    data=df,
    y="Monthly_Contribution_INR",
    color="#F39C12",
    inner="box",
    linewidth=1.2
)

plt.title(
    "Distribution of Monthly Pension Contributions",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("")
plt.ylabel("Monthly Contribution (INR)")

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- Monthly contributions range from approximately **₹42 to ₹16,630** based on the observed data.
- The median monthly contribution is **₹3,820**, indicating that half of the non-missing observations have contributions below this level and half above it.
- The middle 50% of observations fall approximately between **₹1,250 and ₹8,380**, showing substantial variation in contribution levels.

- The violin is widest at the lower contribution levels, indicating that **smaller monthly contributions are more common** in the dataset.
- The distribution also shows substantial density across the higher contribution ranges, particularly around **₹5,000–₹9,000**.
- The distribution extends considerably toward higher contribution values, indicating a **right-skewed pattern** with a smaller number of accounts making substantially larger monthly contributions.
- The violin plot therefore highlights both the **concentration of lower contributions and the wide variation in contribution levels**.

7.3.4 Distribution of Current Fund Value

Why This Visual Is Suitable

A **multi-colour bar chart** is suitable for analysing current fund value by grouping the values into meaningful ranges. This makes it easier to compare how many pension account holders fall within lower, middle, and higher fund-value ranges.

Unlike a boxplot or ECDF, this visual presents the distribution in a simple and direct form, making it easy to identify the most common fund-value range and the relative representation of higher-value accounts.

[179]:
```
# 7.3.4 Distribution of Current Fund Value

fund_bins = [0, 100000, 250000, 500000, 1000000, 2500000, float("inf")]

fund_labels = [
    "Below ₹1 Lakh",
    "₹1–2.5 Lakh",
    "₹2.5–5 Lakh",
    "₹5–10 Lakh",
    "₹10–25 Lakh",
    "Above ₹25 Lakh"
]

fund_groups = pd.cut(
    df["Current_Fund_Value_INR"],
    bins=fund_bins,
    labels=fund_labels,
    include_lowest=True
)

fund_counts = fund_groups.value_counts().reindex(fund_labels)

plt.figure(figsize=(10, 6))

bars = plt.bar(
    fund_counts.index,
    fund_counts.values,
    color=[
        "#42A5F5",
        "#66BB6A",
        "#FFA726",
        "#AB47BC",
        "#EF5350",
        "#26A69A"
    ]
)

plt.title(
    "Distribution of Current Pension Fund Value",
    fontsize=16,
    fontweight="bold"
)

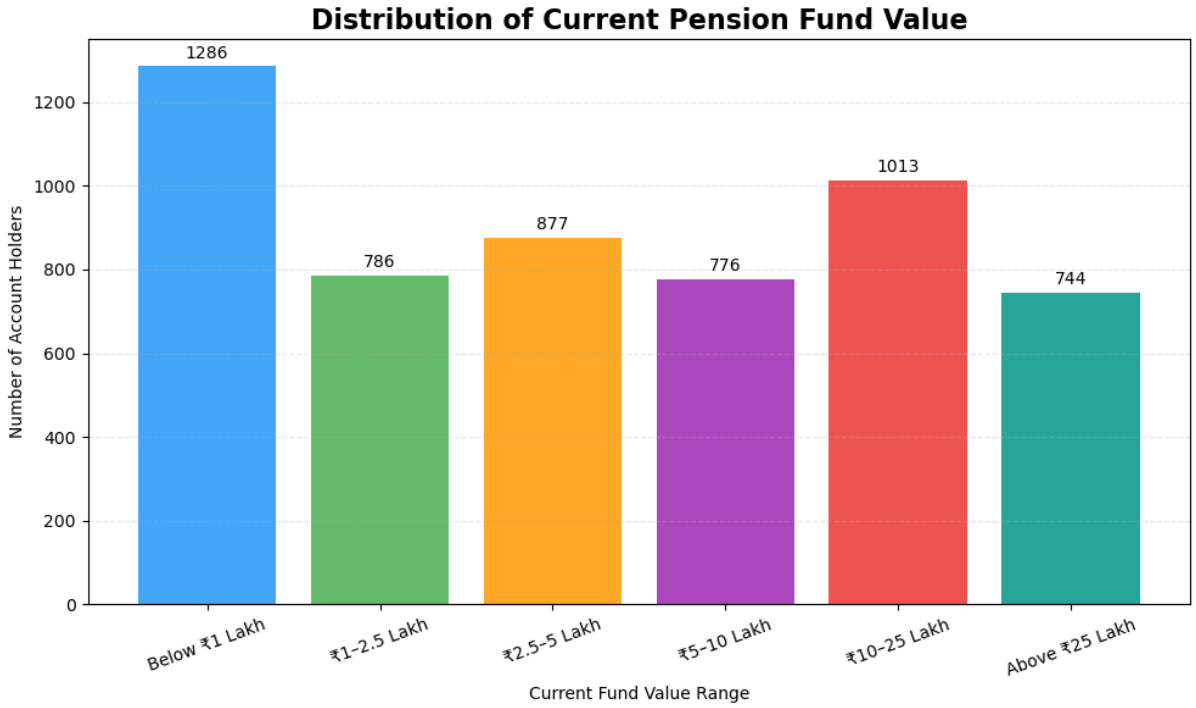
plt.xlabel("Current Fund Value Range")
plt.ylabel("Number of Account Holders")
```

```
plt.xticks(rotation=20)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

for bar in bars:
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 20,
        f"{int(bar.get_height())}",
        ha="center",
        fontsize=10
    )

plt.tight_layout()
plt.show()
```



Key Observations

- The **Below ₹1 Lakh** category has the highest number of account holders (**1,286**).
- The **₹10–25 Lakh** range is the second-largest group, with **1,013** account holders.
- The **₹2.5–5 Lakh** range contains **877** account holders.
- The **₹1–2.5 Lakh** and **₹5–10 Lakh** ranges have relatively similar representation.
- **744** account holders have fund values **above ₹25 Lakh**, showing that higher-value accounts are also meaningfully represented.
- Overall, account holders are distributed across all fund-value ranges rather than being concentrated in only one category.

7.3.5 Investment Choice Preference

Why This Visual Is Suitable

A **donut chart** is suitable for analysing investment choice because it shows how the total number of pension account holders is divided among the available investment options.

The visual provides an easy comparison of the relative share of each investment choice and helps identify whether one option is clearly preferred or whether preferences are relatively balanced.

This also provides a useful foundation for later analysis of how investment choices are associated with asset allocation and financial outcomes.

[181]:
```
# 7.3.5 Investment Choice Preference

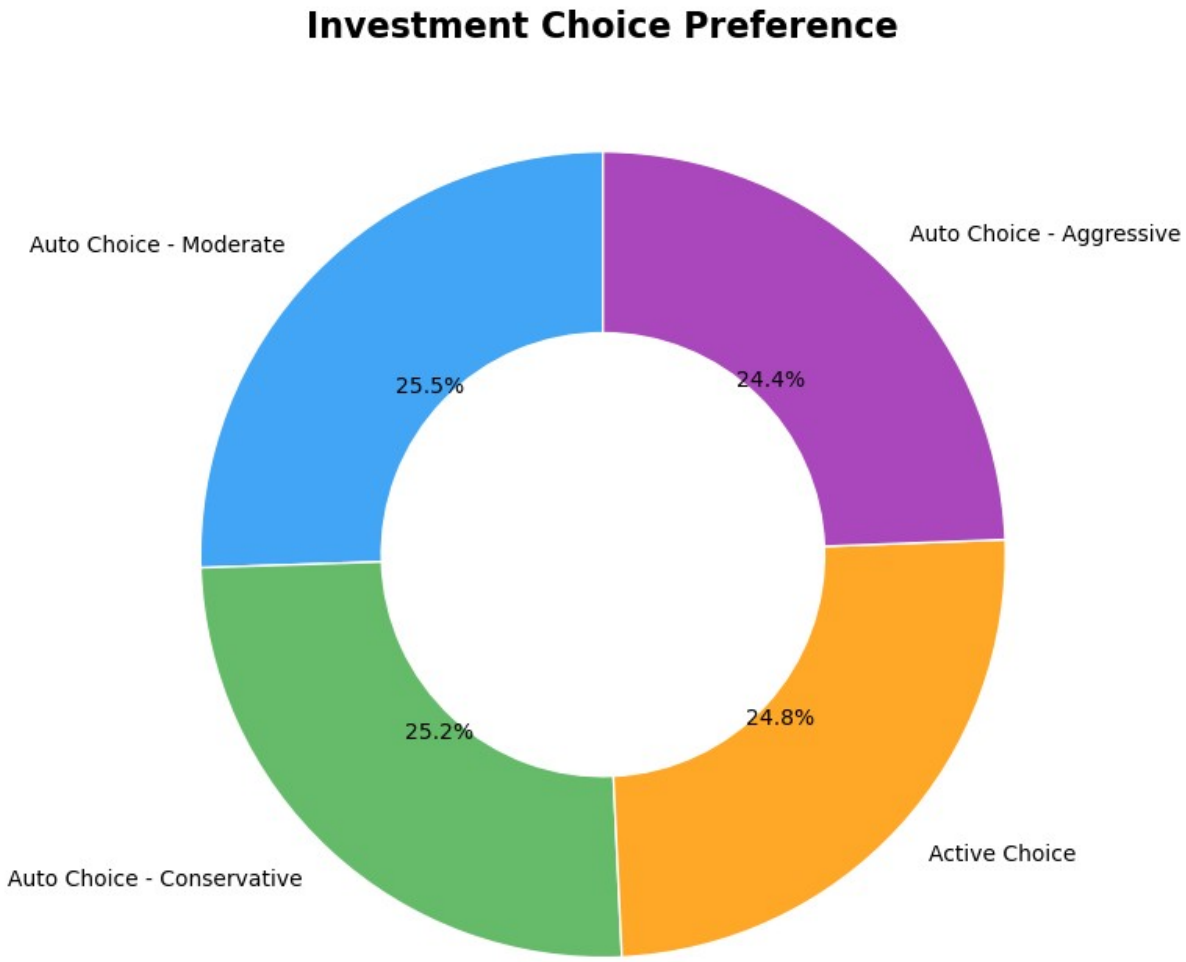
investment_counts = df["Investment_Choice"].value_counts().dropna()

plt.figure(figsize=(9, 7))

plt.pie(
    investment_counts.values,
    labels=investment_counts.index,
    autopct="%1.1f%%",
    startangle=90,
    colors=["#42A5F5", "#66BB6A", "#FFA726", "#AB47BC"],
    wedgeprops={"width": 0.45, "edgecolor": "white"},
    textprops={"fontsize": 10}
)

plt.title(
    "Investment Choice Preference",
    fontsize=16,
    fontweight="bold"
)

plt.tight_layout()
plt.show()
```



Key Observations

- **Auto Choice – Moderate** is the most preferred option at **25.5%**.
- **Auto Choice – Conservative** follows closely at **25.2%**.
- **Active Choice** represents **24.8%** of account holders.
- **Auto Choice – Aggressive** has the lowest share at **24.4%**.
- The difference between the most and least preferred choices is only about **1.1 percentage points**.
- Overall, the dataset shows **no single dominant investment choice**; preferences are relatively balanced across all four options.

7.3.6 Account Status Distribution

Why This Visual Is Suitable

A **lollipop chart** is suitable for analysing account status because it provides a clear comparison of the number of account holders across the different status categories.

It is visually lighter than a traditional bar chart while still making the differences between Active, Inactive, Dormant, and Frozen accounts easy to identify.

This visual helps understand the overall status of pension accounts in the dataset.

[182]:
```python
# 7.3.6 Account Status Distribution

status_counts = (
    df["Account_Status"]
    .value_counts()
    .dropna()
    .sort_values(ascending=True)
)

plt.figure(figsize=(10, 6))

plt.hlines(
    y=status_counts.index,
    xmin=0,
    xmax=status_counts.values,
    linewidth=3,
    color="#90CAF9"
)

plt.scatter(
    status_counts.values,
    status_counts.index,
    s=180,
    color=["#AB47BC", "#EF5350", "#FFA726", "#42A5F5"],
    edgecolor="white",
    linewidth=1.5
)

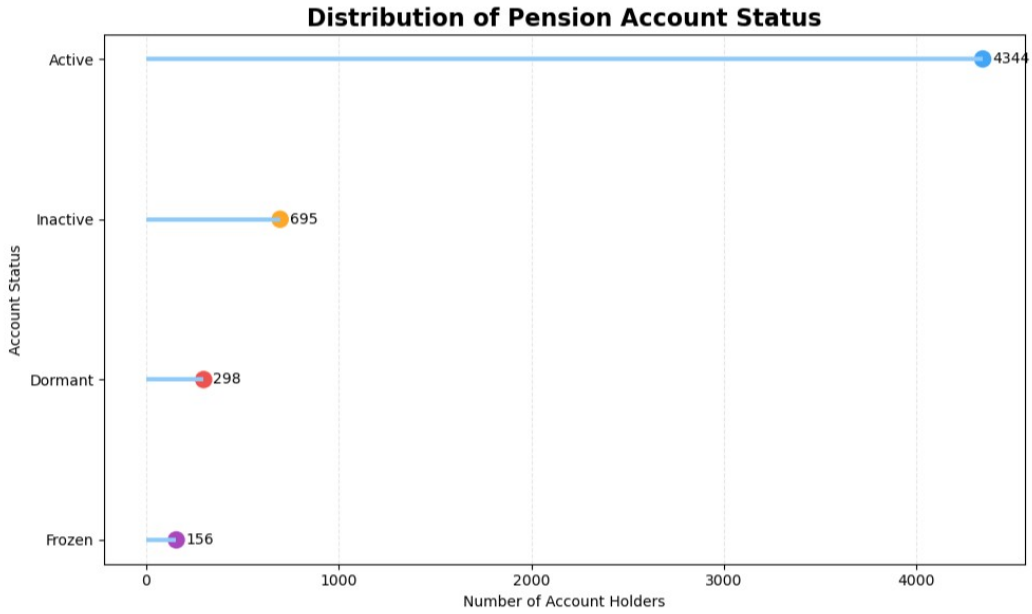
for value, category in zip(status_counts.values, status_counts.index):
    plt.text(
        value + 50,
        category,
        f"{value}",
        va="center",
        fontsize=10
    )

plt.title(
    "Distribution of Pension Account Status",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Number of Account Holders")
plt.ylabel("Account Status")

plt.grid(
    axis="x",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- **Active accounts** dominate the dataset, with **4,344 account holders**.

- **Inactive accounts** are the second-largest group, with **695 account holders**.
- **Dormant accounts** account for **298 account holders**.
- **Frozen accounts** are the smallest group, with **156 account holders**.
- Overall, the dataset is strongly concentrated in the **Active** account-status category.

7.3.7 Pension Scheme Distribution

Why This Visual Is Suitable

A **polar bar chart** is suitable for analysing pension scheme type because it compares the number of account holders across different schemes while presenting the categories in a visually distinct format.

The length of each segment represents the number of account holders, making it easy to identify the most and least represented pension schemes.

This visual also adds variety to the analysis without requiring additional packages or complex calculations.

[186]:
```
# 7.3.7 Pension Scheme Distribution

scheme_counts = (
    df["Pension_Scheme_Type"]
    .value_counts()
    .dropna()
    .sort_values(ascending=False)
)

values = scheme_counts.values
labels = scheme_counts.index

angles = np.linspace(
    0,
    2 * np.pi,
    len(values),
    endpoint=False
)

colors = [
    "#42A5F5",
    "#66BB6A",
    "#FFA726",
    "#AB47BC",
    "#EF5350",
    "#26A69A"
]

plt.figure(figsize=(10, 8))

ax = plt.subplot(111, polar=True)

bars = ax.bar(
    angles,
    values,
    width=0.85,
    color=colors,
    edgecolor="white",
    linewidth=2
)

ax.set_xticks(angles)
ax.set_xticklabels(labels, fontsize=9)

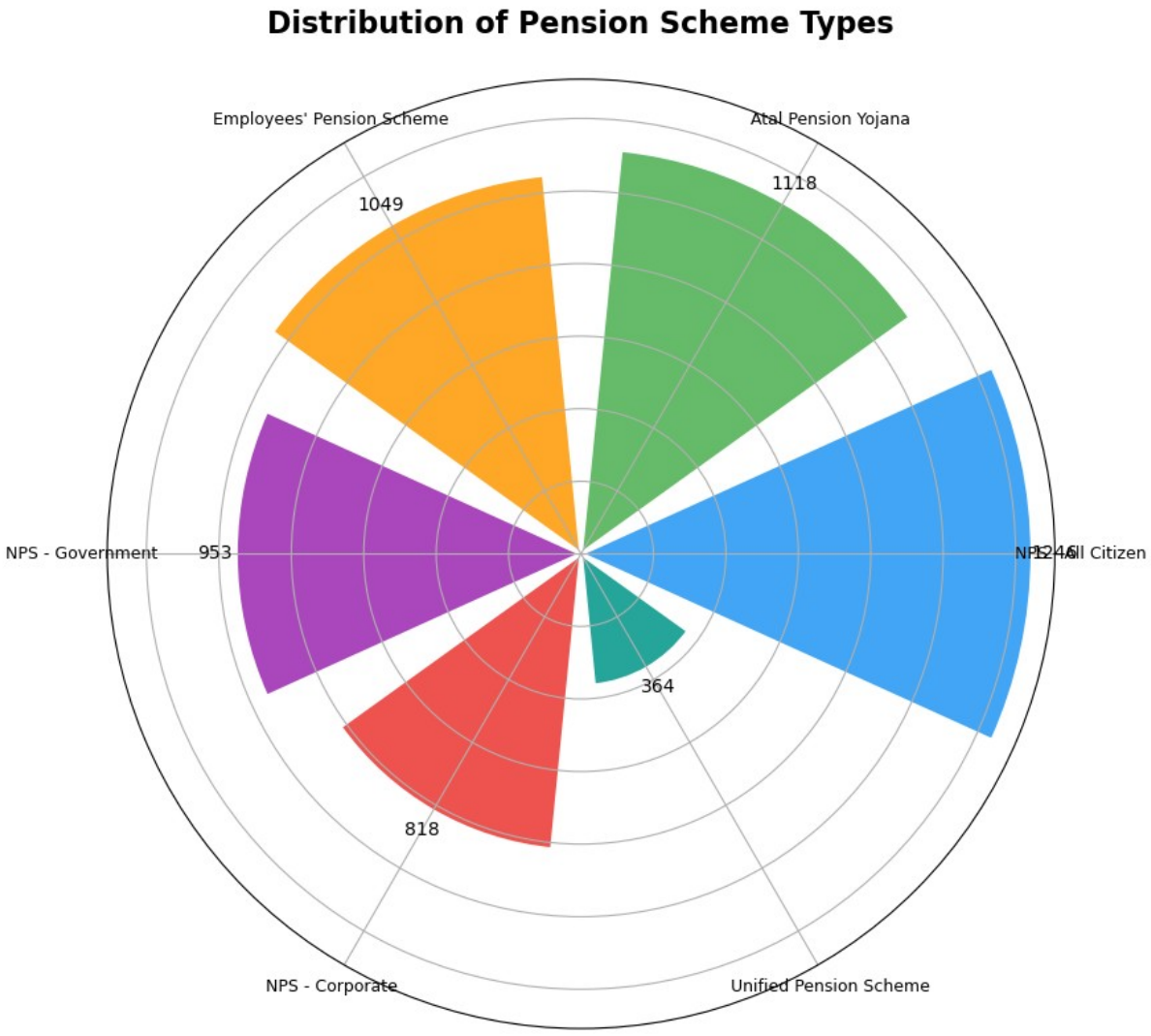
ax.set_title(
    "Distribution of Pension Scheme Types",
```

```
    "Distribution of Pension Scheme Types",
    fontsize=16,
    fontweight="bold",
    pad=25
)

ax.set_yticklabels([])

for angle, value in zip(angles, values):
    ax.text(
        angle,
        value + 60,
        f"{value}",
        ha="center",
        va="center",
        fontsize=10
    )

plt.tight_layout()
plt.show()
```



Key Observations

- **NPS - All Citizen** is the most represented pension scheme, with **1,246 account holders**.
- **Atal Pension Yojana** is the second-largest scheme, with **1,118 account holders**.
- **Employees' Pension Scheme** has **1,049 account holders**, followed by **NPS - Government** with **953**.

- **NPS - Corporate** represents **818 account holders**.
- **Unified Pension Scheme** has the lowest representation, with **364 account holders**.
- Overall, the distribution shows that **NPS - All Citizen and Atal Pension Yojana have the highest representation**, while Unified Pension Scheme has substantially fewer account holders in this dataset.

7.3.8 Sector Distribution

A sector distribution analysis shows how pension account holders are distributed across different employment and participation sectors.

A horizontal bar chart is suitable for this analysis because it allows the frequencies of multiple sector categories to be compared clearly and makes differences between the most and least represented sectors easy to identify.

This visual helps identify which sectors contribute the largest share of observations in the dataset and provides an overview of the composition of pension participation across sectors.

[223]:
```
sector_counts = (
    df["Sector"]
    .value_counts()
    .dropna()
    .sort_values(ascending=True)
)

plt.figure(figsize=(10, 6))

bars = plt.barh(
    sector_counts.index,
    sector_counts.values,
    color="#F4C430"
)

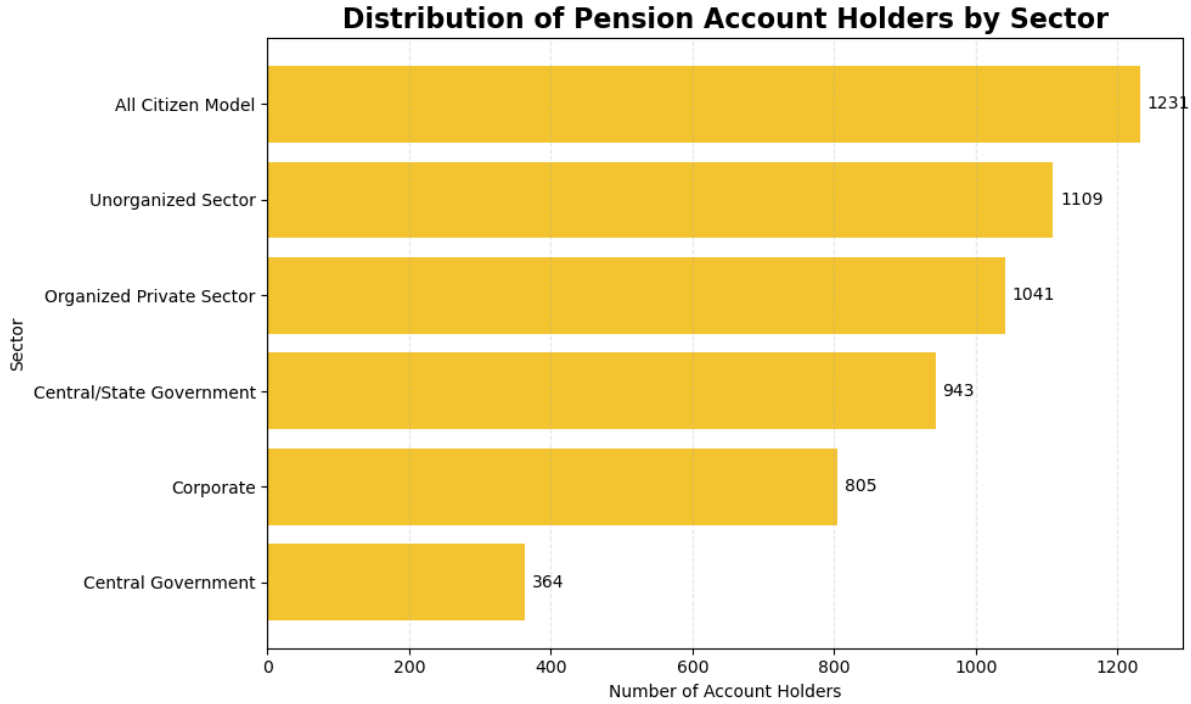
plt.title(
    "Distribution of Pension Account Holders by Sector",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Number of Account Holders")
plt.ylabel("Sector")

plt.grid(axis="x", linestyle="--", alpha=0.3)

for bar in bars:
    plt.text(
        bar.get_width() + 10,
        bar.get_y() + bar.get_height() / 2,
        f"{int(bar.get_width())}",
        va="center",
        fontsize=10
    )

plt.tight_layout()
plt.show()
```



Key Observations:

- The **All Citizen Model** has the highest number of account holders, with **1,231** observations.
- The **Unorganized Sector** follows with **1,109** account holders.
- The **Organized Private Sector** has **1,041** account holders.
- The **Central Government** category has the lowest representation, with **364** account holders.
- Overall, pension participation is distributed across several sectors, with the All Citizen Model being the most represented in this dataset.

7.3.9 State-wise Distribution

State-wise analysis examines how pension account holders are distributed across different states in India.

A dot plot is suitable for this analysis because it provides a simple comparison of the number of account holders across the most represented states without repeating the bar chart format used in earlier analyses.

The analysis focuses on the **top 10 states** to keep the visual clear and readable while highlighting the states with the highest representation in the dataset.

[235]:
```
state_counts = (
    df["State"]
    .value_counts()
    .dropna()
    .head(10)
    .sort_values()
)

plt.figure(figsize=(10, 6))

plt.plot(
    state_counts.values,
    state_counts.index,
    "o",
    markersize=9,
    color="#F4C430"
)

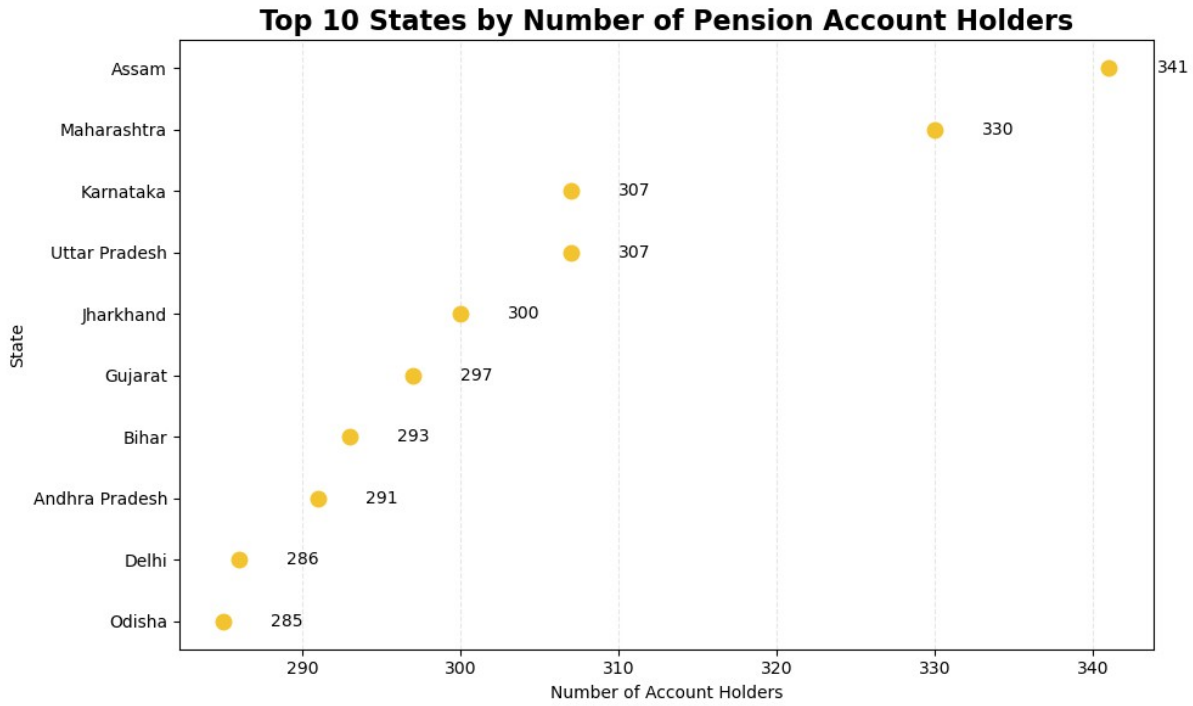
plt.title(
    "Top 10 States by Number of Pension Account Holders",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Number of Account Holders")
plt.ylabel("State")

plt.grid(axis="x", linestyle="--", alpha=0.3)

for value, state in zip(state_counts.values, state_counts.index):
    plt.text(
        value + 3,
        state,
        f"{value}",
        va="center",
        fontsize=10
    )

plt.tight_layout()
plt.show()
```



Key Observations:

- **Assam** has the highest representation among the top 10 states, with **341** pension account holders.
- **Maharashtra** follows with **330** account holders.
- **Karnataka** and **Uttar Pradesh** have equal representation, with **307** account holders each.

- **Jharkhand** has **300** account holders, followed by **Gujarat** with **297**.
- **Bihar, Andhra Pradesh, Delhi, and Odisha** have comparatively lower representation among the top 10 states.
- Overall, the difference between the highest and lowest represented states in the top 10 is relatively small, ranging from **285 to 341** account holders.

7.4 Bivariate Analysis

Bivariate analysis examines two variables simultaneously to understand whether a relationship or pattern exists between them.

The appropriate visualisation depends on the types of variables being compared. For example, scatter plots can be used for two numerical variables, while grouped bar charts or boxplots can be used when numerical and categorical variables are compared.

For the pension dataset, bivariate analysis can be used to investigate questions such as whether higher monthly contributions are associated with higher expected monthly pensions, whether income is related to contribution levels, and whether age is associated with financial outcomes.

The relationships identified through bivariate analysis describe patterns within the dataset and should not be interpreted as proof of causation.

7.4.1 Annual Income vs Monthly Contribution

Why This Visual Is Suitable

A **hexbin plot** is suitable for examining annual income and monthly contribution because both variables are numerical and many observations overlap at similar values.

Instead of displaying every observation separately, the hexagonal areas show where account holders are more concentrated. Darker areas represent regions with more observations, making the overall relationship easier to identify.

This provides a clearer view of the data than a highly crowded scatter plot.

[188]:
```python
# 7.4.1 Annual Income vs Monthly Contribution

plt.figure(figsize=(10, 6))

plt.hexbin(
    df["Annual_Income_INR"],
    df["Monthly_Contribution_INR"],
    gridsize=30,
    cmap="Purples",
    mincnt=1
)

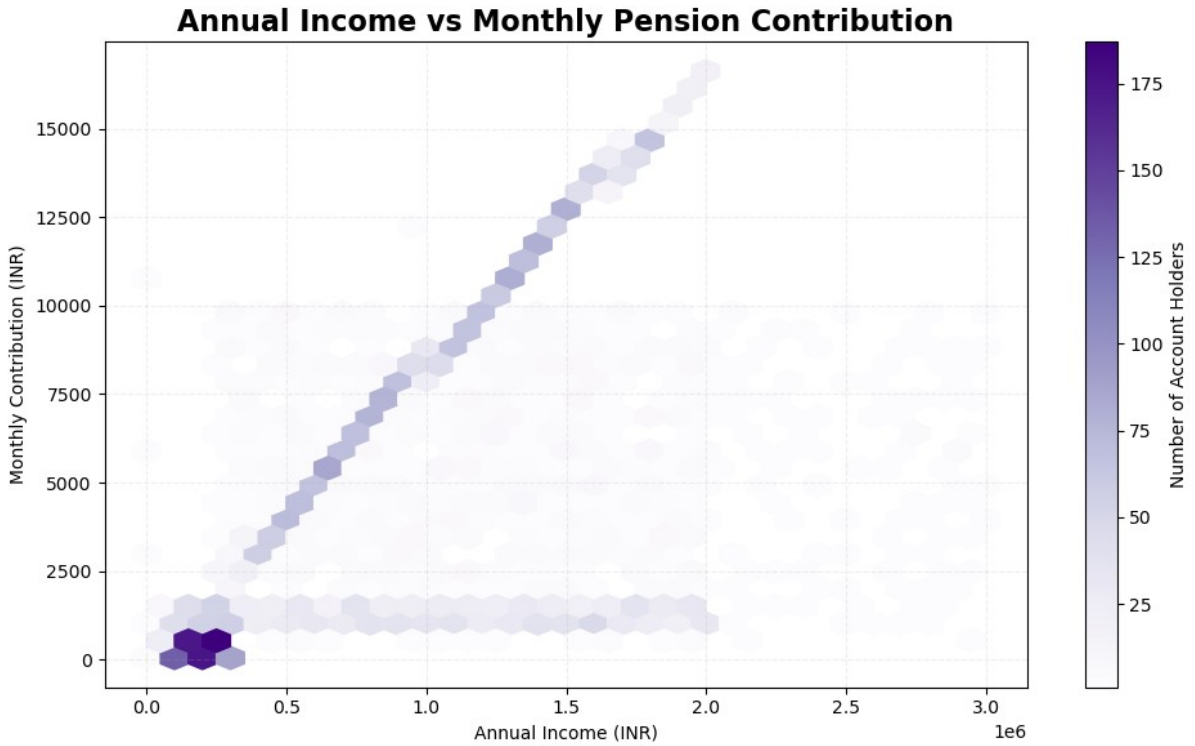
plt.colorbar(label="Number of Account Holders")

plt.title(
    "Annual Income vs Monthly Pension Contribution",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Annual Income (INR)")
plt.ylabel("Monthly Contribution (INR)")

plt.grid(
    linestyle="--",
    alpha=0.2
)

plt.tight_layout()
plt.show()
```



Key Observations

- The highest concentration of account holders occurs at **lower monthly contribution levels**, particularly below approximately **₹2,500**.
- As annual income increases, monthly contributions generally show an **upward association**, although the observations are widely dispersed.
- A smaller group of account holders has **substantially higher monthly contributions**, reaching above **₹10,000**.
- Higher-income account holders are distributed across a wider range of monthly contribution levels rather than following one uniform pattern.

- Overall, the visual suggests a **positive association between annual income and monthly contribution** in this dataset, while also showing considerable variation among individual account holders.

7.4.2 Employee vs Employer Monthly Contribution

Why This Visual Is Suitable

A **dumbbell chart** is suitable for comparing the typical monthly contribution made by the account holder and the employer.

The two endpoints show the median employee and employer contributions, while the connecting line highlights the difference between them.

This visual provides a direct comparison of the two contribution sources without using another bar or scatter chart.

[191]:
```
# 7.4.2 Employee vs Employer Monthly Contribution

employee_median = df["Monthly_Contribution_INR"].median()
employer_median = df["Employer_Monthly_Contribution_INR"].median()

plt.figure(figsize=(10, 4))

# Connecting line
plt.plot(
    [employer_median, employee_median],
    [0, 0],
    color="#B0BEC5",
    linewidth=6,
    zorder=1
)

# Employee contribution
plt.scatter(
    employee_median,
    0,
    s=220,
    color="#42A5F5",
    edgecolor="white",
    linewidth=2,
    zorder=2
)

# Employer contribution
plt.scatter(
    employer_median,
    0,
    s=220,
    color="#EF5350",
    edgecolor="white",
    linewidth=2,
    zorder=2
)

# Employee label
plt.annotate(
    f"Employee\n₹{employee_median:,.0f}",
    (employee_median, 0),
    xytext=(0, 20),
    textcoords="offset points",
    ha="center",
```

```
plt.annotate(
    f"Employee\n₹{employee_median:,.0f}",
    (employee_median, 0),
    xytext=(0, 20),
    textcoords="offset points",
    ha="center",
    fontsize=11,
    fontweight="bold"
)

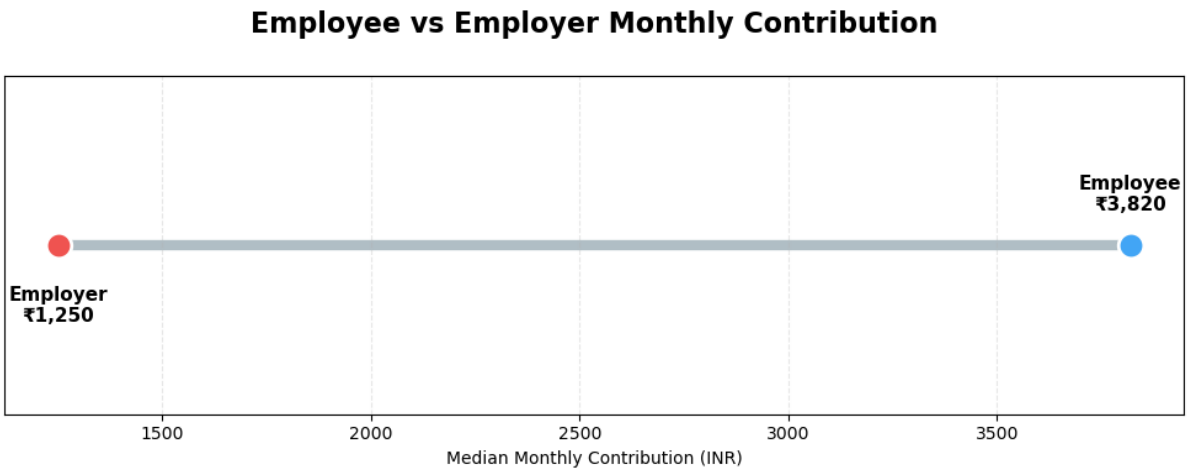
# Employer label
plt.annotate(
    f"Employer\n₹{employer_median:,.0f}",
    (employer_median, 0),
    xytext=(0, -45),
    textcoords="offset points",
    ha="center",
    fontsize=11,
    fontweight="bold"
)

plt.yticks([])
plt.xlabel("Median Monthly Contribution (INR)")

plt.title(
    "Employee vs Employer Monthly Contribution",
    fontsize=16,
    fontweight="bold",
    pad=25
)

plt.grid(
    axis="x",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- The median **employee monthly contribution is ₹3,820**.
- The median **employer monthly contribution is ₹1,250**.
- The employee contribution is substantially higher than the employer contribution based on the median values.
- The difference between the two contribution levels is clearly visible in the dumbbell chart.
- Overall, the dataset shows that **employee contributions are higher than employer contributions in terms of their median monthly values**.

7.4.3 Monthly Contribution vs Current Fund Value

Why This Visual Is Suitable

A **scatter plot** is suitable for examining the relationship between monthly contribution and current fund value because both variables are numerical.

Each point represents a pension account holder, allowing us to observe whether higher monthly contributions are generally associated with higher current fund values.

Using transparency helps reduce visual overcrowding when multiple observations are located close to one another.

```
[192]: # 7.4.3 Monthly Contribution vs Current Fund Value

plt.figure(figsize=(10, 6))

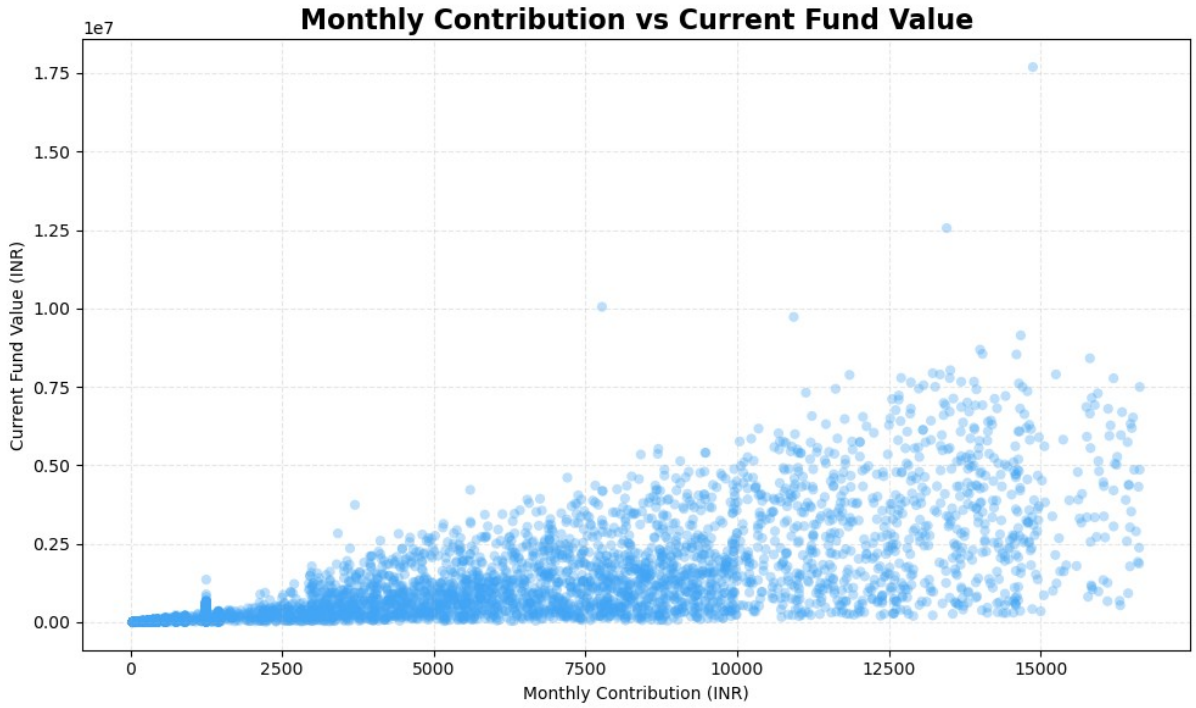
plt.scatter(
    df["Monthly_Contribution_INR"],
    df["Current_Fund_Value_INR"],
    alpha=0.35,
    s=35,
    color="#42A5F5",
    edgecolors="none"
)

plt.title(
    "Monthly Contribution vs Current Fund Value",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Monthly Contribution (INR)")
plt.ylabel("Current Fund Value (INR)")

plt.grid(
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- There is a clear **positive association** between monthly contribution and current fund value: account holders with higher monthly contributions generally tend to have higher current fund values.
- Most account holders are concentrated at **lower monthly contribution levels and lower current fund values**.
- As monthly contribution increases, the **spread of current fund values becomes wider**, indicating greater variation among account holders with higher contributions.
- Account holders with monthly contributions above approximately **₹10,000** show a much wider range of current fund values.
- A small number of account holders have **exceptionally high current fund values**, including values above ₹1 crore, compared with the majority of observations.
- Overall, the visual indicates a **positive but widely dispersed relationship** between monthly contribution and current fund value in this dataset.

7.4.4 Pension Scheme vs Current Fund Value

Why This Visual Is Suitable

A **vertical lollipop chart** is suitable for comparing the typical current fund value across pension schemes.

The **median current fund value** is used because the fund values have substantial variation and some accounts have very high values. The median provides a more representative comparison of the typical account holder within each scheme.

The vertical format makes it easy to compare the relative position of each pension scheme while keeping the visual simple and uncluttered.

[195]:
```
# 7.4.4 Pension Scheme vs Current Fund Value

scheme_fund_median = (
    df.groupby("Pension_Scheme_Type")["Current_Fund_Value_INR"]
    .median()
    .sort_values()
)

plt.figure(figsize=(11, 6))

plt.vlines(
    x=range(len(scheme_fund_median)),
    ymin=0,
    ymax=scheme_fund_median.values,
    color="#B0BEC5",
    linewidth=3
)

plt.scatter(
    range(len(scheme_fund_median)),
    scheme_fund_median.values,
    s=180,
    color="#AB47BC",
    edgecolor="white",
    linewidth=1.5,
    zorder=3
)

for i, value in enumerate(scheme_fund_median.values):
    plt.text(
        i,
        value + 50000,
        f"₹{value:,.0f}",
        ha="center",
        fontsize=10
    )

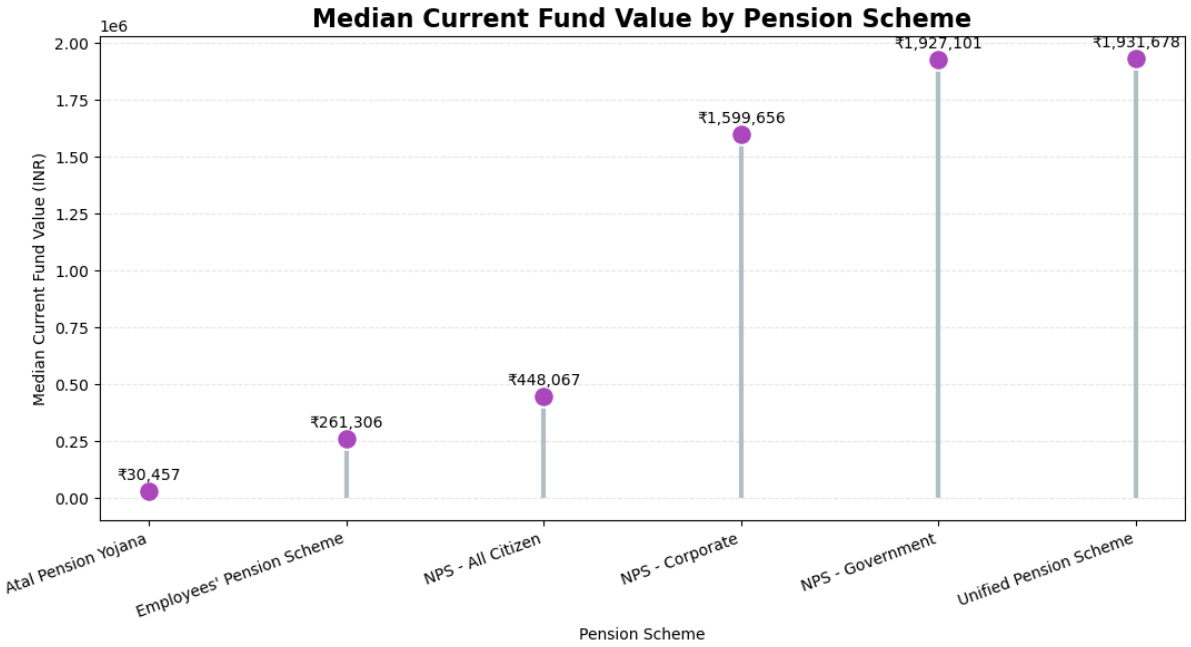
plt.title(
    "Median Current Fund Value by Pension Scheme",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Pension Scheme")
plt.ylabel("Median Current Fund Value (INR)")

plt.xticks(
    range(len(scheme_fund_median)),
    scheme_fund_median.index,
    rotation=20,
    ha="right"
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- **Unified Pension Scheme** has the highest median current fund value at approximately **₹19.32 lakh**.
- **NPS - Government** is very similar, with a median current fund value of approximately **₹19.27 lakh**.
- **NPS - Corporate** has a median current fund value of approximately **₹16.00 lakh**.
- **NPS - All Citizen** has a considerably lower median of approximately **₹4.48 lakh**.
- **Employees' Pension Scheme** has a median current fund value of approximately **₹2.61 lakh**.
- **Atal Pension Yojana** has the lowest median current fund value at approximately **₹30,457**.
- Overall, the chart shows substantial differences in the **typical current fund value across pension schemes** in this dataset.
- These differences describe the financial profiles of account holders in the dataset and **should not be interpreted as evidence that one pension scheme causes higher fund values**.

7.4.5 Account Status vs Withdrawal Status

Why This Visual Is Suitable

A **100% stacked bar chart** is suitable for comparing account status with withdrawal status because both variables are categorical.

Each bar represents one account-status category, while the segments show the percentage distribution of withdrawal status within that category.

Using percentages rather than raw counts makes the different account-status groups directly comparable, even though they contain different numbers of account holders.

[196]:
```
# 7.4.5 Account Status vs Withdrawal Status

status_withdrawal = pd.crosstab(
    df["Account_Status"],
    df["Withdrawal_Status"],
    normalize="index"
) * 100

status_order = ["Active", "Inactive", "Dormant", "Frozen"]
status_withdrawal = status_withdrawal.reindex(status_order)

status_withdrawal.plot(
    kind="bar",
    stacked=True,
    figsize=(10, 6),
    colormap="Set2"
)

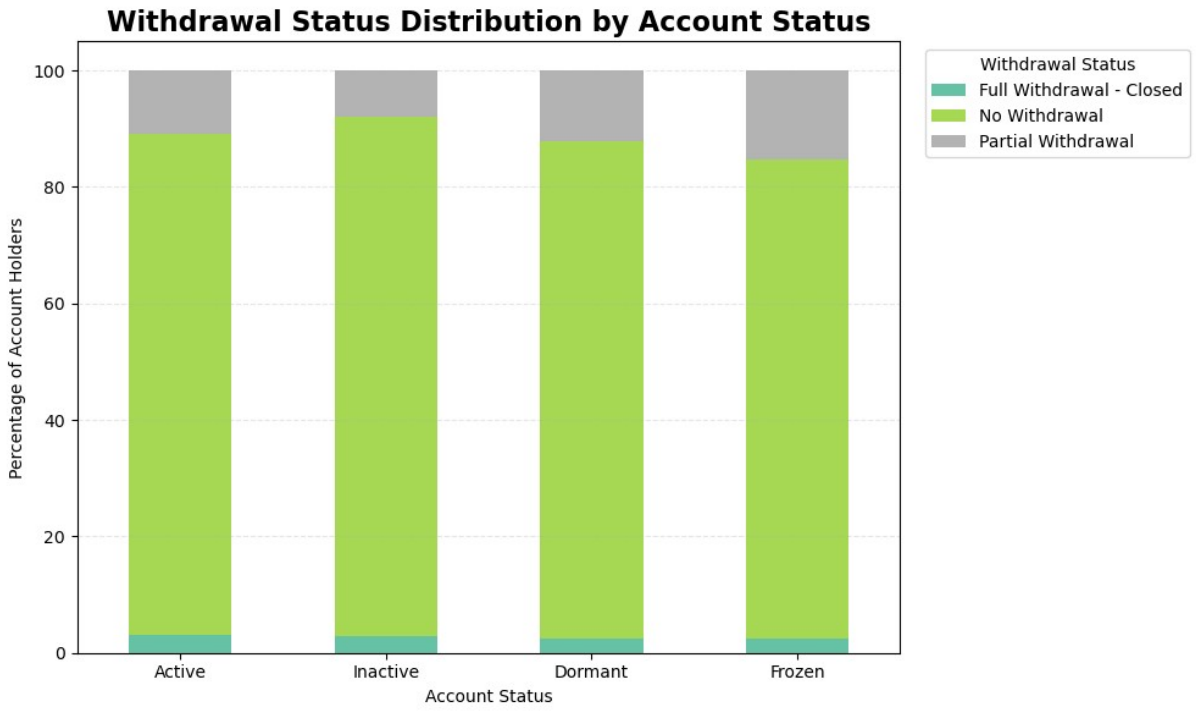
plt.title(
    "Withdrawal Status Distribution by Account Status",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Account Status")
plt.ylabel("Percentage of Account Holders")

plt.xticks(rotation=0)
plt.legend(
    title="Withdrawal Status",
    bbox_to_anchor=(1.02, 1),
    loc="upper left"
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- **No Withdrawal** is the dominant withdrawal status across all four account-status categories.

- **Active accounts** have approximately **89%** of observations in the No Withdrawal category.
- **Inactive accounts** show the highest share of No Withdrawal, at approximately **92%**.
- **Dormant accounts** have approximately **88%** No Withdrawal.
- **Frozen accounts** have the lowest share of No Withdrawal, at approximately **85%**, indicating a somewhat larger proportion of withdrawals or other withdrawal statuses.
- Overall, the withdrawal pattern is **largely similar across account-status groups**, with No Withdrawal representing the clear majority in every category.

7.4.6 Investment Choice vs Asset Allocation

Why This Visual Is Suitable

A **100% stacked horizontal bar chart** is suitable for comparing investment choices with asset allocation because it shows how each investment choice is divided among equity, corporate debt, government securities, and alternative assets.

Because each bar represents the complete asset allocation, the composition of the different investment choices can be compared directly.

The horizontal format makes the investment-choice labels easier to read and keeps the visual clear despite the longer category names.

This visual helps identify how the asset mix differs between **Active Choice** and the different **Auto Choice** options.

[198]:
```python
# 7.4.6 Investment Choice vs Asset Allocation

allocation_columns = [
    "Asset_Allocation_Equity_E_Percent",
    "Asset_Allocation_CorporateDebt_C_Percent",
    "Asset_Allocation_GovtSecurities_G_Percent",
    "Asset_Allocation_Alternative_A_Percent"
]

allocation_by_choice = (
    df.groupby("Investment_Choice")[allocation_columns]
    .mean()
    .dropna()
)

allocation_by_choice.columns = [
    "Equity",
    "Corporate Debt",
    "Government Securities",
    "Alternative Assets"
]

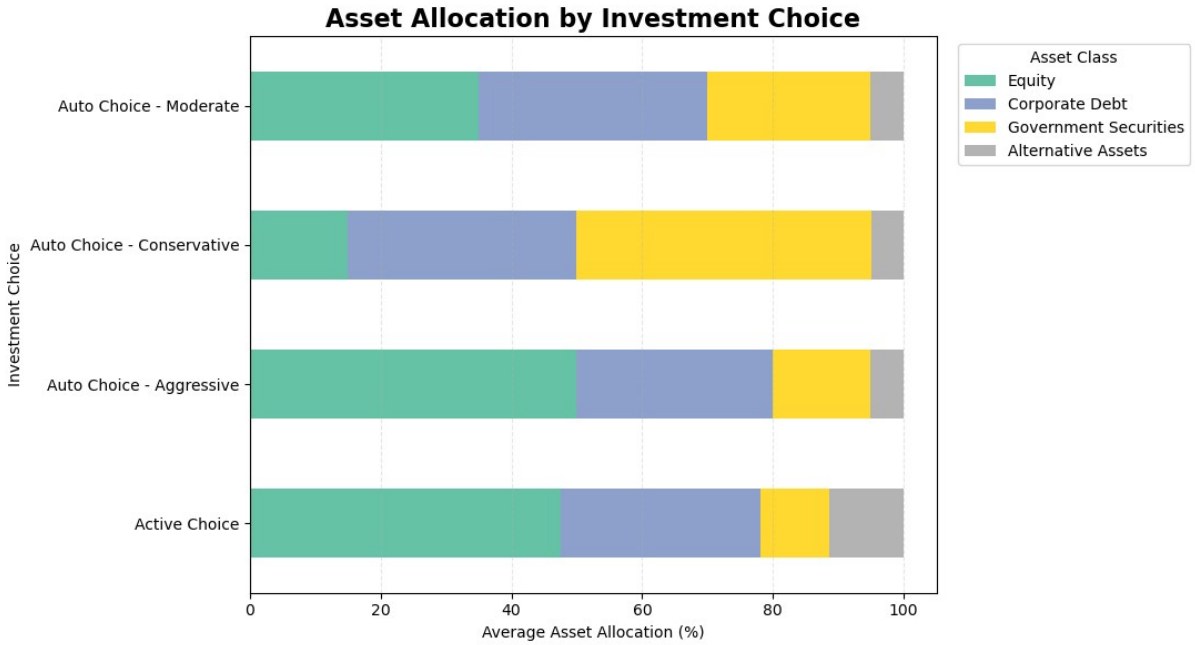
allocation_by_choice.plot(
    kind="barh",
    stacked=True,
    figsize=(11, 6),
    colormap="Set2"
)

plt.title(
    "Asset Allocation by Investment Choice",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Average Asset Allocation (%)")
plt.ylabel("Investment Choice")

plt.legend(
    title="Asset Class",
    bbox_to_anchor=(1.02, 1),
    loc="upper left"
)

plt.grid(
    axis="x",
```



Key Observations

- **Auto Choice – Conservative** has the highest allocation to **Government Securities**, at approximately **45%**, and the lowest allocation to Equity, at approximately **15%**.

- **Auto Choice – Aggressive** has the highest allocation to **Equity**, at approximately **50%**, while Government Securities account for approximately **15%**.
- **Auto Choice – Moderate** has a more balanced allocation, with approximately **35% Equity**, **35% Corporate Debt**, and **25% Government Securities**.
- **Active Choice** has the highest Equity allocation among the choices at approximately **47%**, together with around **31% Corporate Debt**.
- **Alternative Assets** represent a relatively small portion of all four investment choices, at approximately **5%** in the Auto Choice options and around **11%** in Active Choice.
- Overall, the chart shows a clear difference in asset allocation across investment choices, particularly in the balance between **Equity and Government Securities**.

7.4.7 Pension Scheme vs Expected Monthly Pension

Why This Visual Is Suitable

A **vertical lollipop chart** is suitable for comparing the typical expected monthly pension across pension schemes.

The **median expected monthly pension** is used because pension values vary across account holders and the median provides a clear measure of the typical value within each scheme.

This visual makes it easy to identify which pension schemes have higher or lower typical expected pension amounts in the dataset.

The comparison is descriptive and does not imply that one scheme causes a higher pension outcome.

[205]:
```
# 7.4.7 Pension Scheme vs Expected Monthly Pension

scheme_pension_median = (
    df.groupby("Pension_Scheme_Type")["Expected_Monthly_Pension_INR"]
    .median()
    .sort_values()
)

plt.figure(figsize=(11, 6))

plt.vlines(
    x=range(len(scheme_pension_median)),
    ymin=0,
    ymax=scheme_pension_median.values,
    color="#B0BEC5",
    linewidth=3
)

plt.scatter(
    range(len(scheme_pension_median)),
    scheme_pension_median.values,
    s=180,
    color="#26A69A",
    edgecolor="white",
    linewidth=1.5,
    zorder=3
)

for i, value in enumerate(scheme_pension_median.values):
    plt.text(
        i,
        value + 200,
        f"₹{value:,.0f}",
        ha="center",
        fontsize=10
    )

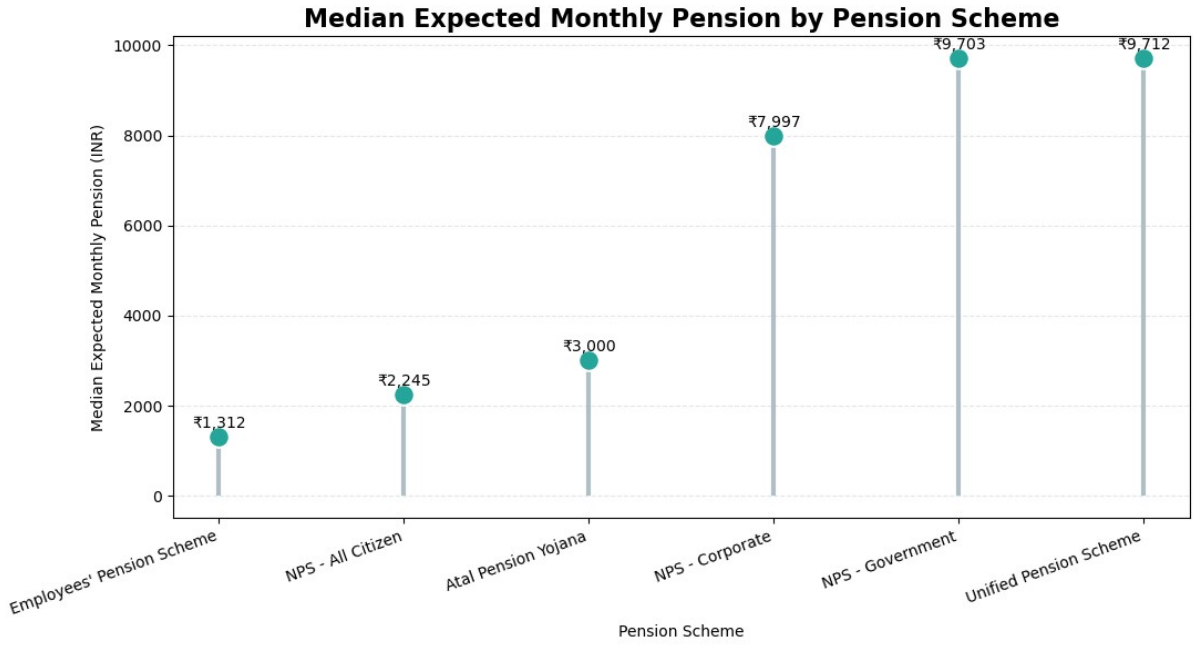
plt.title(
    "Median Expected Monthly Pension by Pension Scheme",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Pension Scheme")
plt.ylabel("Median Expected Monthly Pension (INR)")
```

```
plt.xticks(
    range(len(scheme_pension_median)),
    scheme_pension_median.index,
    rotation=20,
    ha="right"
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- **Unified Pension Scheme** has the highest median expected monthly pension at approximately **₹9,712**.
- **NPS - Government** is very close, with a median expected monthly pension of approximately **₹9,703**.
- **NPS - Corporate** has a median expected monthly pension of approximately **₹7,997**.
- **Atal Pension Yojana** has a median expected monthly pension of approximately **₹3,000**.
- **NPS - All Citizen** has a lower median expected monthly pension of approximately **₹2,245**.
- **Employees' Pension Scheme** has the lowest median expected monthly pension at approximately **₹1,312**.
- Overall, the chart shows **substantial differences in the typical expected monthly pension across pension schemes**.
- These differences describe the account profiles observed in this dataset and **should not be interpreted as evidence that one scheme causes a higher pension**.

7.5 Multivariate Analysis

Multivariate analysis examines patterns involving three or more variables or relationships between multiple categorical groups.

In this section, the analysis focuses on combinations of pension scheme, sector, investment choice, and other important characteristics to identify broader patterns within the dataset.

The purpose is to understand how different groups are distributed across multiple dimensions and to identify meaningful patterns that may not be visible when variables are examined individually.

The analysis remains descriptive and does not imply causal relationships.

7.5.1 Occupation Group vs Investment Choice

Why This Visual Is Suitable

A **row-normalized heatmap** is suitable for examining investment preferences across occupation groups because both variables are categorical.

The percentages are calculated within each occupation group, so the groups can be compared even though they contain different numbers of account holders.

The colour intensity makes it easy to identify which investment choices are relatively more or less preferred within each occupation group.

This visual helps identify whether investment preferences are broadly similar across occupations or whether some groups show noticeably different patterns.

[201]:
```
# 7.5.1 Occupation Group vs Investment Choice

occupation_investment = pd.crosstab(
    df["Occupation_Group"],
    df["Investment_Choice"],
    normalize="index"
) * 100

plt.figure(figsize=(11, 6))

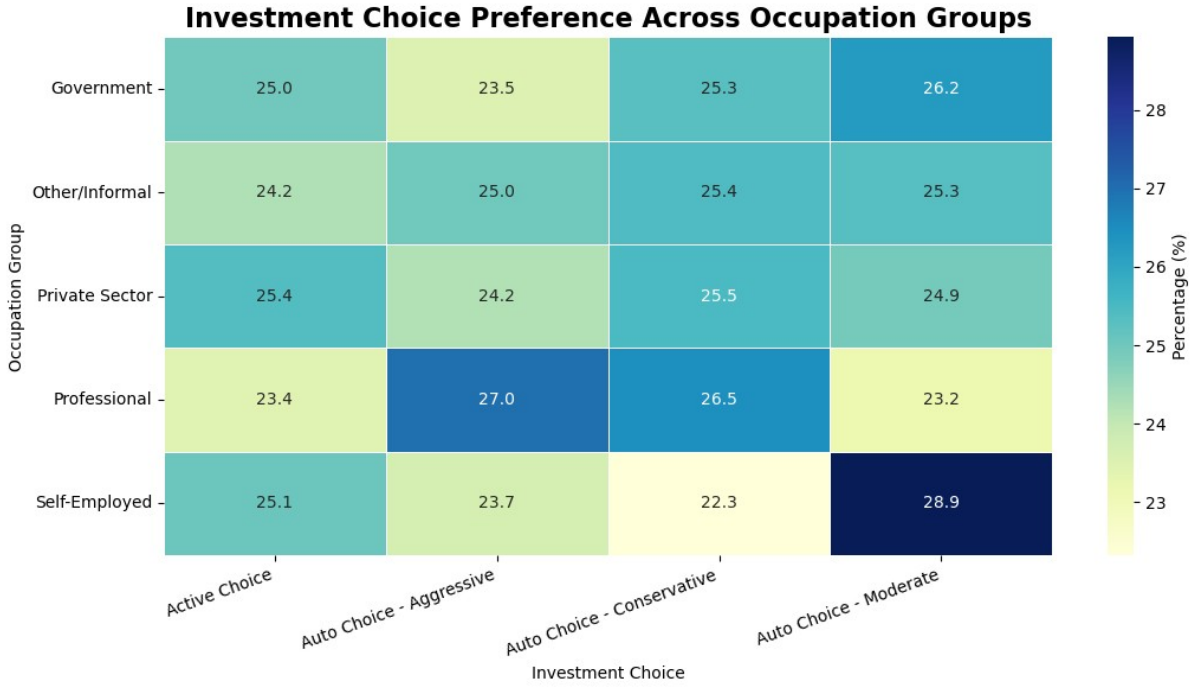
sns.heatmap(
    occupation_investment,
    annot=True,
    fmt=".1f",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={"label": "Percentage (%)"}
)

plt.title(
    "Investment Choice Preference Across Occupation Groups",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Investment Choice")
plt.ylabel("Occupation Group")

plt.xticks(rotation=20, ha="right")
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```



Key Observations

- **Self-Employed** account holders show the highest preference for **Auto Choice - Moderate (28.9%)**.
- **Professional** account holders show the highest preference for **Auto Choice - Aggressive (27.0%)**, followed closely by **Auto Choice - Conservative (26.5%)**.
- **Government** account holders show their highest preference for **Auto Choice - Moderate (26.2%)**.
- **Other/Informal** account holders have a relatively balanced distribution, with all four investment choices between approximately **24% and 25%**.
- **Private Sector** account holders also show a relatively balanced preference, with each investment choice between approximately **24% and 26%**.
- Overall, investment preferences vary somewhat across occupation groups, but **no single investment choice dominates across all occupation groups**.
- The differences are descriptive associations within this dataset and should not be interpreted as evidence that occupation determines investment choice.

7.5.2 Gender vs Investment Choice

Why This Visual Is Suitable

A **100% stacked horizontal bar chart** is suitable for comparing investment choice preferences across gender groups because both variables are categorical.

Each bar represents a gender group, while the segments show the percentage distribution of investment choices within that group.

Using percentages allows the investment preferences of the groups to be compared directly despite differences in group size.

This visual helps identify whether investment-choice preferences are broadly similar across gender groups or whether noticeable differences exist.

[207]:
```
# 7.5.2 Gender vs Investment Choice

gender_investment = pd.crosstab(
    df["Gender"],
    df["Investment_Choice"],
    normalize="index"
) * 100

gender_investment = gender_investment.dropna()

gender_investment.plot(
    kind="barh",
    stacked=True,
    figsize=(10, 5),
    colormap="Pastel1"
)

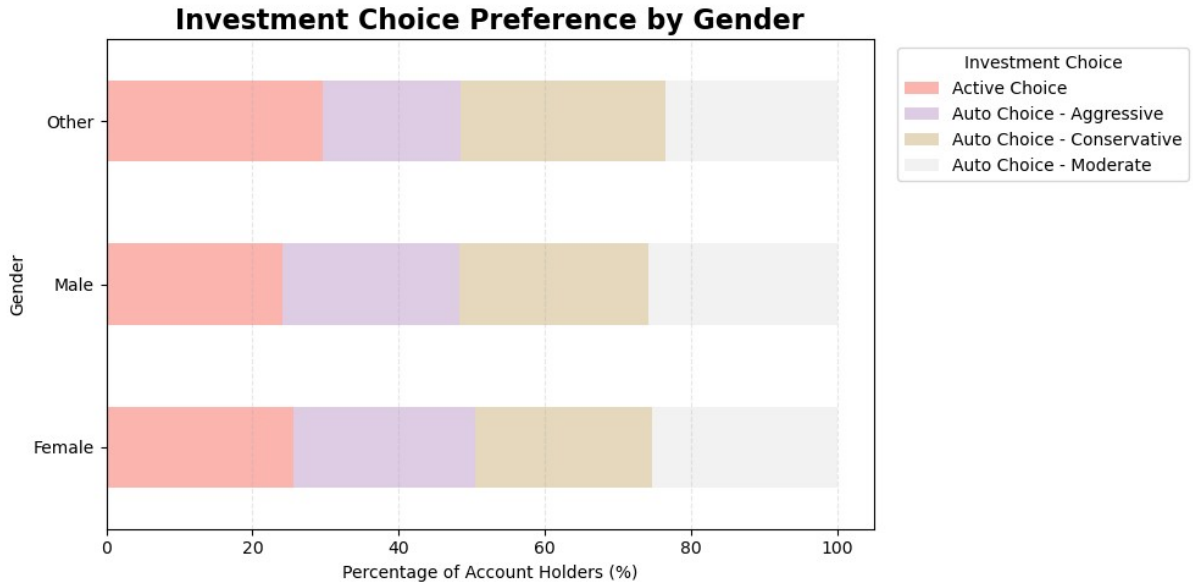
plt.title(
    "Investment Choice Preference by Gender",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Percentage of Account Holders (%)")
plt.ylabel("Gender")

plt.legend(
    title="Investment Choice",
    bbox_to_anchor=(1.02, 1),
    loc="upper left"
)

plt.grid(
    axis="x",
    linestyle="--",
    alpha=0.3
)

plt.tight_layout()
plt.show()
```



Key Observations

- **Female and male account holders show broadly similar investment-choice preferences** across all four options.

- Among **female account holders**, Auto Choice - Moderate has the largest share, followed by Auto Choice - Conservative, Auto Choice - Aggressive, and Active Choice.
- Among **male account holders**, Auto Choice - Moderate is also the largest category, while Active Choice has the smallest share.
- The differences between male and female preferences are relatively small, indicating that **investment choice is fairly balanced across these two gender groups**.
- The **Other** category shows a somewhat different distribution, but this group represents only **1.17% of the dataset**, so its pattern should be interpreted cautiously.
- Overall, gender does **not show a major difference in investment-choice preference** in this dataset.

7.5.2 Pension Scheme × Account Status × Current Fund Value

Why This Visual Is Suitable

This heatmap analysis examines three important variables together: pension scheme, account status, and current fund value. A heatmap is suitable because it allows the typical fund value for each combination of pension scheme and account status to be compared easily. This can reveal whether financial outcomes differ across account conditions within different pension schemes.

[212]:
```
scheme_account_fund = pd.pivot_table(
    df,
    values="Current_Fund_Value_INR",
    index="Pension_Scheme_Type",
    columns="Account_Status",
    aggfunc="median"
)

plt.figure(figsize=(11, 6))

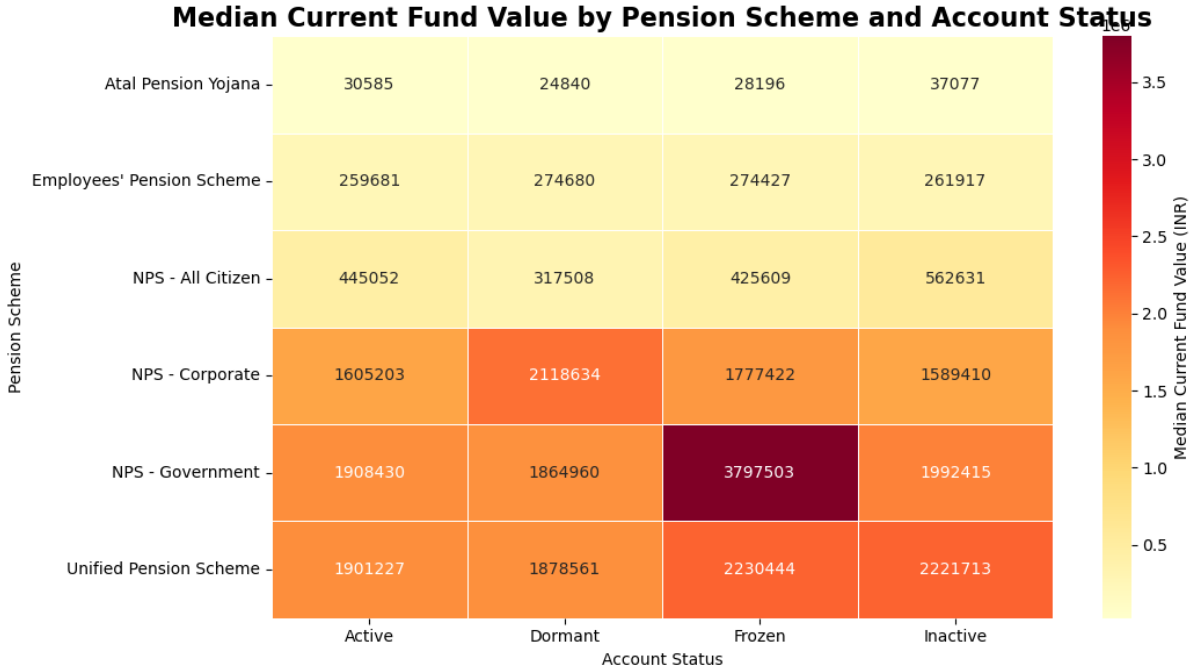
sns.heatmap(
    scheme_account_fund,
    annot=True,
    fmt=".0f",
    cmap="YlOrRd",
    linewidths=0.5,
    cbar_kws={"label": "Median Current Fund Value (INR)"}
)

plt.title(
    "Median Current Fund Value by Pension Scheme and Account Status",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Account Status")
plt.ylabel("Pension Scheme")

plt.xticks(rotation=0)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```



Key Observations

- **NPS - Government** and **Unified Pension Scheme** show the highest median current fund values across most account statuses.
- The **Frozen NPS - Government** accounts have the highest median fund value in the chart at approximately **₹37.98 lakh**.
- **Atal Pension Yojana** has substantially lower median fund values across all account statuses, with values around **₹25,000–₹37,000**.
- **Employees' Pension Scheme** also shows relatively low median fund values compared with the government-oriented schemes.
- Within individual schemes, fund values vary across account statuses, but the pattern is **not consistent across every scheme**.
- Overall, the chart shows that **pension scheme differences are much more pronounced than account-status differences** in the observed fund values.

Interpretation: These are descriptive patterns in the dataset and should not be interpreted as evidence that a particular pension scheme or account status causes higher or lower fund values.

7.5.3 Years of Contribution × Financial Outcomes

Why This Visual Is Suitable

This analysis examines years of contribution together with two important financial outcomes: current fund value and expected monthly pension.

A two-panel hexbin plot is suitable because the dataset contains many observations, with several observations concentrated in the same ranges.

Hexbin plots make these concentrations and overall relationships easier to identify than a conventional scatterplot.

The analysis uses the existing numerical variables without creating additional derived measures.

[213]:
```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Years of Contribution vs Current Fund Value
hb1 = axes[0].hexbin(
    df["Years_of_Contribution"],
    df["Current_Fund_Value_INR"],
    gridsize=25,
    cmap="Blues",
    mincnt=1
)

fig.colorbar(
    hb1,
    ax=axes[0],
    label="Number of Account Holders"
)

axes[0].set_title(
    "Years of Contribution vs Current Fund Value",
    fontsize=14,
    fontweight="bold"
)

axes[0].set_xlabel("Years of Contribution")
axes[0].set_ylabel("Current Fund Value (INR)")


# Years of Contribution vs Expected Monthly Pension
hb2 = axes[1].hexbin(
    df["Years_of_Contribution"],
    df["Expected_Monthly_Pension_INR"],
    gridsize=25,
    cmap="Oranges",
    mincnt=1
)

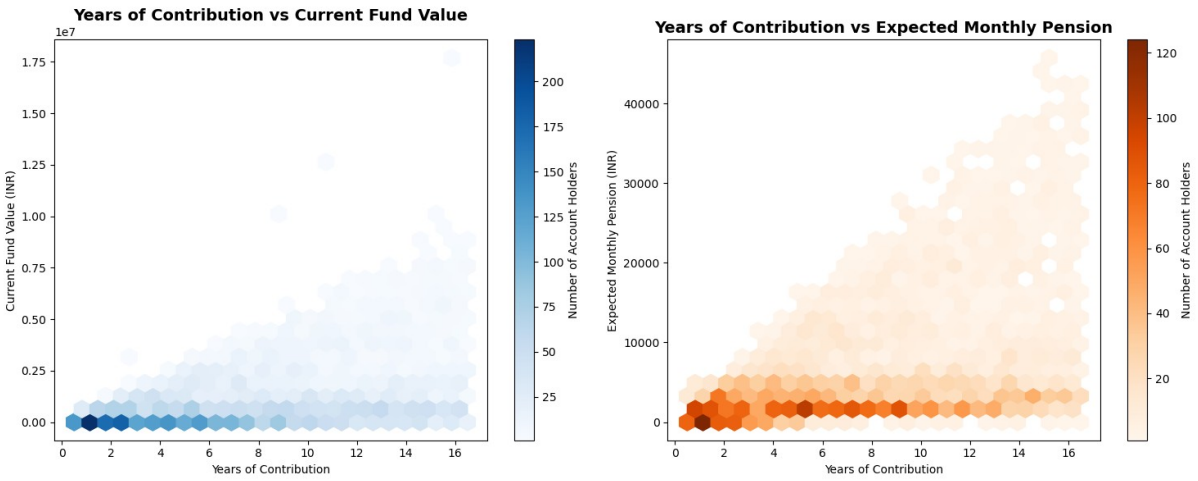
fig.colorbar(
    hb2,
    ax=axes[1],
    label="Number of Account Holders"
)

axes[1].set_title(
    "Years of Contribution vs Expected Monthly Pension",
    fontsize=14,
)
```

```
axes[1].set_title(
    "Years of Contribution vs Expected Monthly Pension",
    fontsize=14,
    fontweight="bold"
)

axes[1].set_xlabel("Years of Contribution")
axes[1].set_ylabel("Expected Monthly Pension (INR)")

plt.tight_layout()
plt.show()
```



Key Observations

- The first panel shows a **generally positive association** between years of contribution and current fund value.
- Current fund values are more concentrated at lower levels, while **greater variation is visible among account holders with longer contribution periods**.
- The second panel also shows a **positive association** between years of contribution and expected monthly pension.
- Expected monthly pension is more concentrated at lower values, while some higher pension values appear among account holders with longer contribution periods.
- Overall, **longer contribution periods are associated with higher current fund values and higher expected monthly pensions** in this dataset.
- The relationship is not uniform, as account holders with similar contribution periods can have substantially different financial outcomes.
- These findings describe **associations within the dataset and do not establish causation**.

7.5.4 Pension Scheme × Withdrawal Status × Current Fund Value

Why This Visual Is Suitable

This analysis examines pension scheme, withdrawal status, and current fund value together. A heatmap is suitable because it allows the typical current fund value to be compared across different withdrawal categories within each pension scheme.

This helps identify whether the relationship between withdrawal status and current fund value differs across pension schemes.

The analysis uses the existing variables without creating additional derived measures.

[214]:
```
scheme_withdrawal_fund = pd.pivot_table(
    df,
    values="Current_Fund_Value_INR",
    index="Pension_Scheme_Type",
    columns="Withdrawal_Status",
    aggfunc="median"
)

plt.figure(figsize=(12, 6))

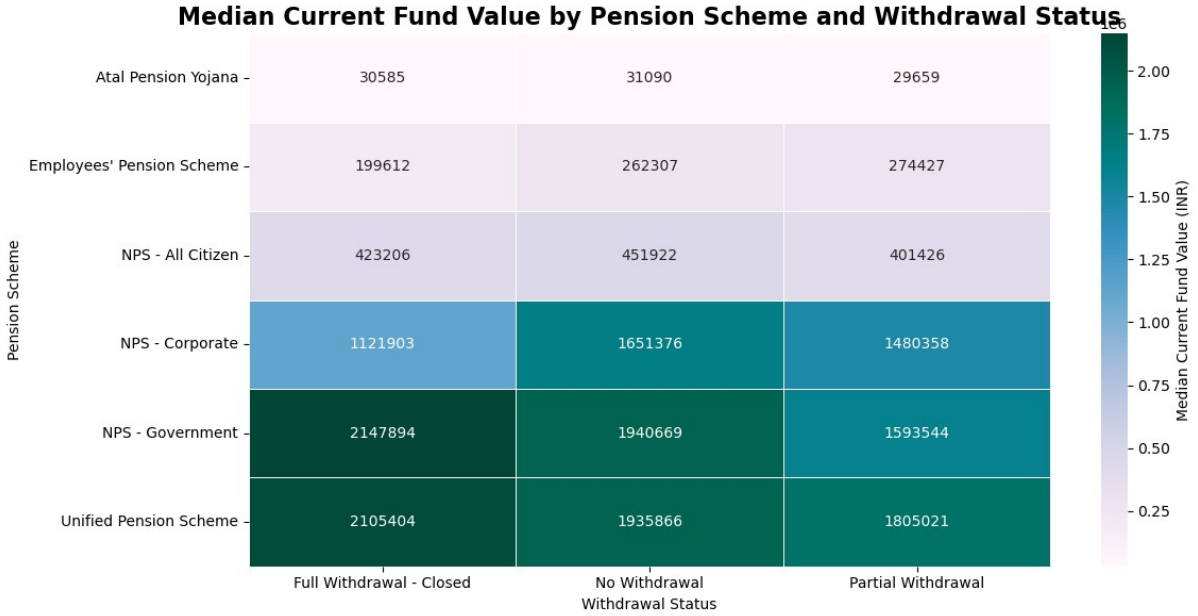
sns.heatmap(
    scheme_withdrawal_fund,
    annot=True,
    fmt=".0f",
    cmap="PuBuGn",
    linewidths=0.5,
    cbar_kws={"label": "Median Current Fund Value (INR)"}
)

plt.title(
    "Median Current Fund Value by Pension Scheme and Withdrawal Status",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Withdrawal Status")
plt.ylabel("Pension Scheme")

plt.xticks(rotation=0)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```



Key Observations

- **NPS - Government** and **Unified Pension Scheme** show the highest median current fund values across all withdrawal-status categories.
- For **Atal Pension Yojana**, median fund values remain relatively low across all withdrawal statuses, at approximately **₹30,000**.
- **NPS - Corporate** shows a higher median fund value for accounts with **no withdrawal** (approximately ₹16.51 lakh) compared with full or partial withdrawal.
- For **NPS - Government**, the highest median fund value is observed among **full withdrawal/closed accounts** at approximately **₹21.48 lakh**.

- For **Unified Pension Scheme**, full withdrawal/closed accounts also show a higher median fund value than accounts with no withdrawal or partial withdrawal.
- The relationship between withdrawal status and fund value **varies across pension schemes**, rather than following one consistent pattern.
- These are descriptive associations within the dataset and should not be interpreted as evidence that withdrawal status causes differences in current fund value.

7.5.5 Pension Scheme × Tax Benefit × Expected Monthly Pension

Why This Visual Is Suitable

This analysis examines pension scheme, tax benefit section claimed, and expected monthly pension together. A heatmap is suitable because it allows the typical expected pension to be compared across different tax-benefit categories within each pension scheme.

This helps identify whether expected pension patterns differ across tax-benefit categories within different pension schemes.

The analysis uses the existing variables without creating additional derived measures.

7.5.5 Pension Scheme × Tax Benefit × Expected Monthly Pension

Why This Visual Is Suitable

This analysis examines pension scheme, tax benefit section claimed, and expected monthly pension together. A dot plot is suitable because it allows the median expected monthly pension for each combination to be compared clearly while using colour to distinguish the different tax-benefit categories.

[216]:
```
scheme_tax_pension = (
    df.groupby(
        ["Pension_Scheme_Type", "Tax_Benefit_Section_Claimed"],
        dropna=False
    )["Expected_Monthly_Pension_INR"]
    .median()
    .reset_index()
)

plt.figure(figsize=(12, 7))

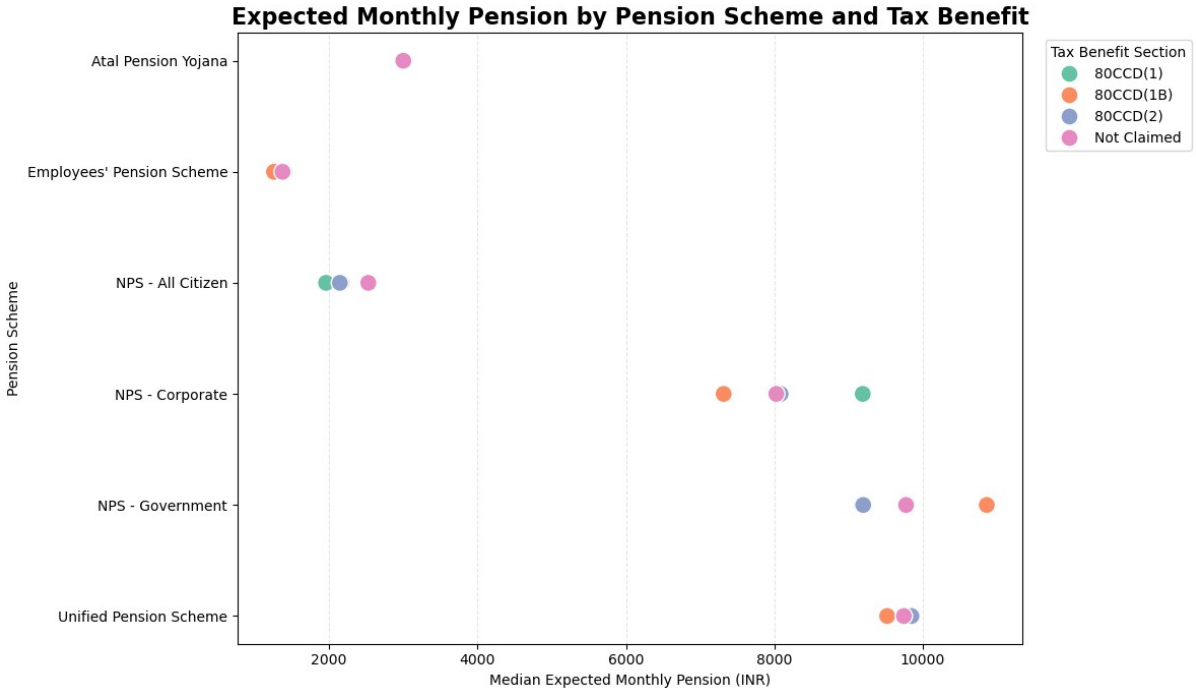
sns.scatterplot(
    data=scheme_tax_pension,
    x="Expected_Monthly_Pension_INR",
    y="Pension_Scheme_Type",
    hue="Tax_Benefit_Section_Claimed",
    s=150,
    palette="Set2"
)

plt.title(
    "Expected Monthly Pension by Pension Scheme and Tax Benefit",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel("Median Expected Monthly Pension (INR)")
plt.ylabel("Pension Scheme")

plt.grid(axis="x", linestyle="--", alpha=0.3)
plt.legend(
    title="Tax Benefit Section",
    bbox_to_anchor=(1.02, 1),
    loc="upper left"
)

plt.tight_layout()
plt.show()
```



Key Observations

- **NPS - Government** and **Unified Pension Scheme** show the highest median expected monthly pension across the tax-benefit categories.
- **NPS - Government** shows the widest variation by tax-benefit category, with median expected pension ranging from roughly **₹9,200 to ₹10,800**.
- **Unified Pension Scheme** shows relatively little variation across tax-benefit categories, with values clustered around **₹9,500–₹9,900**.
- **NPS - Corporate** also shows noticeable differences across tax-benefit categories, with median pensions ranging from roughly **₹7,300 to ₹9,200**.
- **Atal Pension Yojana** and **Employees' Pension Scheme** have substantially lower median expected pensions than the other schemes.
- Overall, **pension scheme differences are more pronounced than tax-benefit differences** in this dataset.
- These are descriptive associations and do not indicate that a particular tax benefit causes a higher or lower expected pension.

7.5.6 Gender wise Pension Scheme and Expected Monthly Pension

Why This Visual Is Suitable

This analysis examines gender differences in expected monthly pension across different pension schemes.

A dumbbell chart is suitable because it directly compares the median expected monthly pension for male and female account holders within each pension scheme. The connecting line makes the difference between the two groups easy to identify without using another bar chart or heatmap.

The analysis uses the existing variables without creating additional derived measures.

[217]:
```
gender_scheme_pension = (
    df[df["Gender"].isin(["Male", "Female"])]
    .groupby(["Pension_Scheme_Type", "Gender"])["Expected_Monthly_Pension_INR"]
    .median()
    .unstack()
)

gender_scheme_pension = gender_scheme_pension.dropna(subset=["Male", "Female"])

plt.figure(figsize=(11, 7))

for i, scheme in enumerate(gender_scheme_pension.index):
    male_value = gender_scheme_pension.loc[scheme, "Male"]
    female_value = gender_scheme_pension.loc[scheme, "Female"]

    plt.plot(
        [female_value, male_value],
        [i, i],
        color="gray",
        linewidth=2
    )

    plt.scatter(
        female_value,
        i,
        s=120,
        color="#E76F51",
        label="Female" if i == 0 else ""
    )

    plt.scatter(
        male_value,
        i,
        s=120,
        color="#457B9D",
        label="Male" if i == 0 else ""
    )

plt.yticks(
    range(len(gender_scheme_pension.index)),
    gender_scheme_pension.index
)

plt.xlabel("Median Expected Monthly Pension (INR)")
plt.ylabel("Pension Scheme")
```

```
)

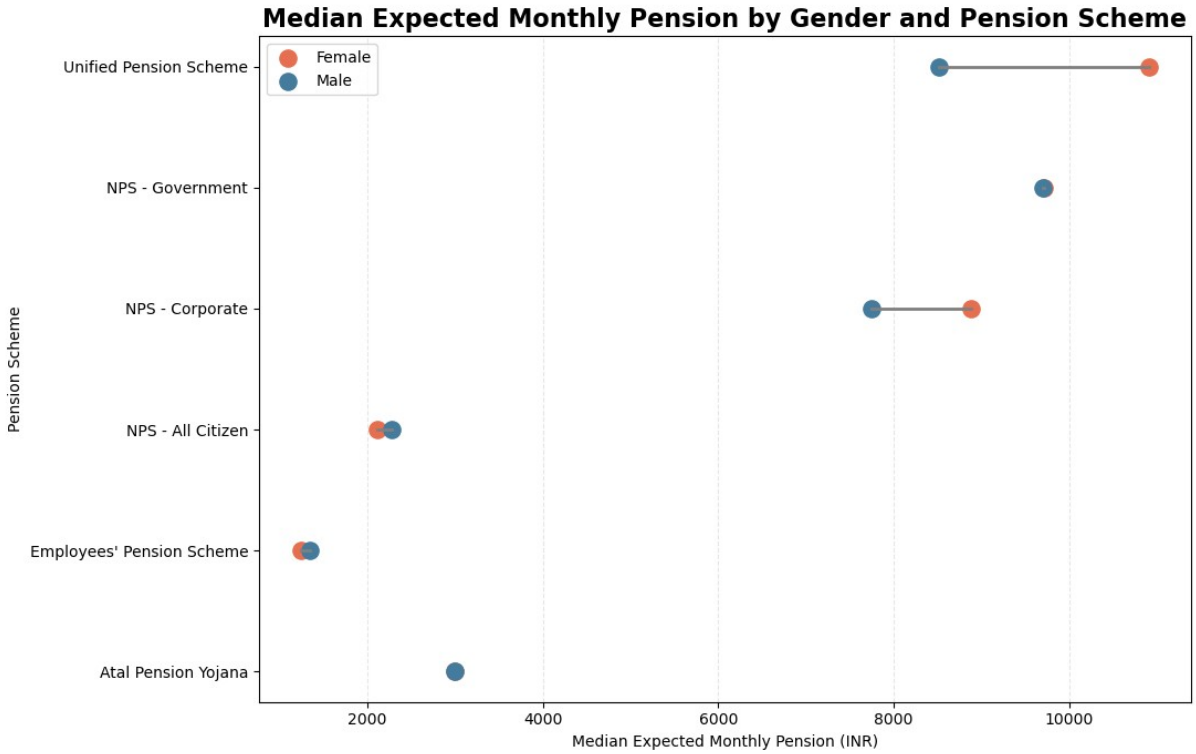
plt.xlabel("Median Expected Monthly Pension (INR)")
plt.ylabel("Pension Scheme")

plt.title(
    "Median Expected Monthly Pension by Gender and Pension Scheme",
    fontsize=16,
    fontweight="bold"
)

plt.grid(axis="x", linestyle="--", alpha=0.3)

plt.legend()

plt.tight_layout()
plt.show()
```



Key Observations

- **Unified Pension Scheme** shows the largest gender difference, with the median expected monthly pension higher for female account holders than for male account holders.
- **NPS - Corporate** also shows a noticeable difference, with the female median higher than the male median.
- For **NPS - All Citizen** and **Employees' Pension Scheme**, the male and female median pensions are relatively close.
- **NPS - Government** shows almost no visible difference between male and female median expected pension.
- The **Atal Pension Yojana** category shows only a male point in this comparison, so a gender-based difference cannot be assessed for that scheme from this visual.
- Overall, gender differences in expected monthly pension **vary across pension schemes rather than following one consistent pattern**.
- These are descriptive associations within the dataset and should not be interpreted as evidence that gender causes differences in expected pension.

7.6 Correlation Analysis

Correlation analysis examines the strength and direction of the relationship between numerical variables.

A correlation coefficient generally ranges from -1 to +1. A positive value indicates that two variables tend to increase together, while a negative value

indicates that one variable tends to increase as the other decreases. Values closer to zero indicate a weaker linear relationship.

For the pension dataset, correlation analysis is particularly useful for examining relationships among income, contributions, total contribution, current fund value, and expected monthly pension.

Correlation analysis provides a useful summary of numerical relationships, but correlation does not establish causation. A strong correlation between two variables does not by itself demonstrate that one variable causes changes in the other.

7.6.1 Correlation Network of Key Pension Variables

Why This Visual Is Suitable

A correlation network is suitable for showing several relationships simultaneously. Each variable is represented as a node, while the lines connecting the variables represent the strength of their Spearman correlations.

- **Thicker lines** indicate stronger relationships.
- **Thinner lines** indicate weaker relationships.
- The visual allows the strongest relationships in the pension dataset to be identified quickly without creating multiple separate charts.

[220]:
```python
import matplotlib.pyplot as plt
import numpy as np

# Correlation values already obtained from the analysis
correlations = {
    ("Monthly Contribution", "Current Fund Value"): 0.81,
    ("Total Contribution", "Current Fund Value"): 0.97,
    ("Total Contribution", "Expected Monthly Pension"): 0.75,
    ("Current Fund Value", "Expected Monthly Pension"): 0.76,
    ("Monthly Contribution", "Expected Monthly Pension"): 0.56,
    ("Annual Income", "Monthly Contribution"): 0.55,
    ("Years of Contribution", "Current Fund Value"): 0.43,
    ("Years of Contribution", "Expected Monthly Pension"): 0.47
}

# Variables
variables = [
    "Annual Income",
    "Monthly Contribution",
    "Years of Contribution",
    "Total Contribution",
    "Current Fund Value",
    "Expected Monthly Pension"
]

# Positions of variables
positions = {
    "Annual Income": (0, 2),
    "Monthly Contribution": (1, 2),
    "Years of Contribution": (0, 0),
    "Total Contribution": (2, 0),
    "Current Fund Value": (3, 1),
    "Expected Monthly Pension": (4, 1)
}

plt.figure(figsize=(14, 7))

# Draw connections
for (var1, var2), corr in correlations.items():

    x1, y1 = positions[var1]
    x2, y2 = positions[var2]

    linewidth = 1 + (corr * 5)
```

```
    plt.plot(
        [x1, x2],
        [y1, y2],
        linewidth=linewidth,
        alpha=0.65
    )

    # Correlation value at midpoint
    mid_x = (x1 + x2) / 2
    mid_y = (y1 + y2) / 2

    plt.text(
        mid_x,
        mid_y + 0.08,
        f"{corr:.2f}",
        ha="center",
        fontsize=10,
        fontweight="bold"
    )

# Draw variable nodes
for variable, (x, y) in positions.items():

    plt.scatter(
        x,
        y,
        s=1800,
        zorder=3,
        edgecolor="black",
        linewidth=1.2
    )

    plt.text(
        x,
        y,
        variable,
        ha="center",
        va="center",
        fontsize=9,
        fontweight="bold",
        wrap=True
    )

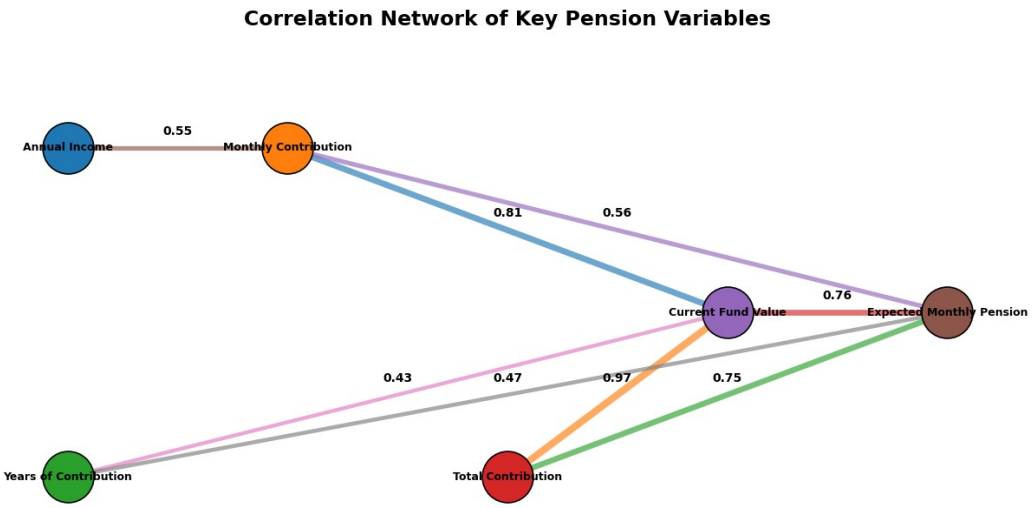
plt.title(
    "Correlation Network of Key Pension Variables",
    fontsize=17,
```

```
    "Correlation Network of Key Pension Variables",
    fontsize=17,
    fontweight="bold"
)

plt.xlim(-0.7, 4.7)
plt.ylim(-0.7, 2.7)

plt.axis("off")

plt.tight_layout()
plt.show()
```



Key Observations

- **Total Contribution and Current Fund Value** show the strongest positive association, with a Spearman correlation of **0.97**.
- **Monthly Contribution and Current Fund Value** also show a strong positive association of **0.81**.
- **Current Fund Value and Expected Monthly Pension** have a strong positive association of **0.76**.
- **Total Contribution and Expected Monthly Pension** show a strong positive association of **0.75**.
- **Annual Income and Monthly Contribution** have a moderate positive association of **0.55**.
- **Years of Contribution** has positive associations with both **Current Fund Value (0.43)** and **Expected Monthly Pension (0.47)**.
- Overall, the financial contribution variables show the strongest relationships with current fund value and expected monthly pension.
- These relationships represent **associations within the dataset and do not establish causation**.

7.7. Findings, Insights and Suggestions

The following table summarises the main findings obtained from the visualisations presented in the Exploratory Data Analysis (EDA).

Each visualisation is linked to its corresponding finding, analytical insight, and possible suggestion. The suggestions are based on descriptive patterns observed in the dataset and should not be interpreted as causal conclusions or individual financial advice.

7.7.1.1 Univariate Analysis Findings

No.	Visualisation Title	Visual Used	Finding	Insight	Suggestion
1	Age Distribution	Histogram + KDE	Most account holders are concentrated around the middle-age range, with a median age of approximately 46 years.	The dataset contains both younger contributors and individuals closer to retirement, with middle-aged subscribers forming a substantial part of the population.	Pension communication can be tailored to different age groups, with greater emphasis on long-term planning for younger contributors.
2	Annual Income Distribution	ECDF	Annual income varies considerably, with a median of approximately ₹9.36 lakh.	Pension subscribers have substantially different income levels and therefore different financial capacities.	Contribution planning and pension awareness can consider different income groups rather than assuming uniform financial capacity.
3	Monthly Contribution Distribution	Violin Plot	Monthly contributions vary widely, with a median of approximately ₹3,820.	There is substantial variation in contribution behaviour among subscribers.	Encourage consistent contributions and greater awareness of long-term contribution planning.
4	Current Fund Value Distribution	Range Bar Chart	Current fund values vary substantially, with many accounts concentrated at lower values and a smaller number at much higher values.	Retirement accumulation differs considerably across account holders.	Greater emphasis can be placed on long-term contribution continuity for subscribers with lower accumulated balances.
5	Investment Choice Preference	Donut Chart	The four investment choices are relatively evenly distributed, with Auto Choice - Moderate being slightly more common.	No single investment choice dominates the dataset.	Investment education should explain the characteristics and risk profiles of different choices rather than promoting one option universally.
6	Account Status Distribution	Lollipop Chart	Active accounts form the large majority, while inactive, dormant, and frozen accounts form smaller groups.	Most accounts are active, but a meaningful minority require attention.	Regular account reviews and follow-up mechanisms could help address inactive, dormant, and frozen accounts.
7	Pension Scheme Distribution	Polar Bar Chart	NPS - All Citizen has the largest representation, while Unified Pension Scheme has the smallest.	Participation is distributed across several pension schemes.	Pension communication can be tailored to the characteristics and requirements of different schemes.

7.7.2 Bivariate Analysis Findings

No.	Visualisation Title	Visual Used	Finding	Insight	Suggestion
8	Annual Income vs Monthly Contribution	Hexbin Plot	Higher annual income is generally associated with higher monthly contributions.	Contribution behaviour varies with income, although substantial variation remains within income levels.	Contribution planning can be adapted to different income levels and financial circumstances.
9	Employee vs Employer Monthly Contribution	Dumbbell Chart	Median employee contribution is approximately ₹3,820, compared with approximately ₹1,250 for employer contribution.	Employee and employer contributions show different contribution patterns in the dataset.	Where employer contributions apply, subscribers should understand their role in retirement accumulation.
10	Monthly Contribution vs Current Fund Value	Scatter Plot	Higher monthly contributions generally correspond to higher current fund values.	Contribution amount has an important positive association with accumulated fund value.	Encourage regular and sustainable contributions as part of long-term retirement planning.
11	Pension Scheme vs Current Fund Value	Vertical Lollipop	NPS - Government and UPS have the highest median current fund values, while APY is substantially lower.	Financial outcomes differ considerably across pension schemes in this dataset.	Schemes should be analysed separately rather than assuming similar financial characteristics.
12	Investment Choice vs Asset Allocation	100% Stacked Bar	Investment choices have clearly different asset-allocation patterns.	Investment choice corresponds to different portfolio allocation patterns.	Subscribers should understand how each investment choice affects asset allocation and risk exposure.
13	Pension Scheme vs Expected Monthly Pension	Vertical Lollipop	UPS and NPS - Government have the highest median expected pensions, while EPS and APY are substantially lower.	Expected pension outcomes differ substantially across schemes.	Pension planning should consider scheme-specific characteristics.

7.7.3 Multivariate Analysis Findings

No.	Visualisation Title	Visual Used	Finding	Insight	Suggestion
14	Occupation Group vs Investment Choice	Row-Normalized Heatmap	Investment preferences are broadly similar across occupation groups, with only modest differences.	Occupation alone does not create a strong separation in investment choice preferences.	Investment guidance should focus on individual financial circumstances and risk preferences rather than occupation alone.
15	Pension Scheme × Account Status × Current Fund Value	Heatmap	NPS - Government and UPS generally show higher median fund values across account statuses.	Scheme-level differences are more pronounced than many account-status differences.	Account status should be monitored separately from fund value because inactive or frozen accounts can still contain substantial balances.
16	Years of Contribution vs Financial Outcomes	Two-Panel Hexbin	Longer contribution periods are generally associated with higher current fund values and expected monthly pensions.	Contribution experience is positively associated with both accumulation and expected pension outcomes.	Encourage long-term contribution continuity and discourage unnecessary interruptions.
17	Pension Scheme × Withdrawal Status × Current Fund Value	Heatmap	Fund values vary across withdrawal categories, with NPS Government and UPS generally showing higher values.	Withdrawal patterns differ across schemes and are associated with different fund-value levels.	Subscribers should understand the potential long-term implications of withdrawals before making withdrawal decisions.
18	Pension Scheme × Tax Benefit × Expected Monthly Pension	Dot Plot	NPS - Government and UPS generally show the highest expected pensions across tax-benefit categories.	Scheme-level differences appear more pronounced than differences between tax-benefit categories.	Tax benefits should be considered as part of retirement planning, but should not be treated as the sole explanation for pension outcomes.
19	Gender × Pension Scheme × Expected Monthly Pension	Dumbbell Chart	Gender differences in expected pension vary across schemes rather than following one consistent pattern.	The relationship between gender and expected pension differs by pension scheme.	Pension planning should consider individual circumstances rather than assuming identical outcomes across demographic groups.

7.7.4 Correlation Analysis Findings

No.	Visualisation Title	Visual Used	Finding	Insight	Suggestion
20	Correlation Network of Key Pension Variables	Correlation Network	Total Contribution and Current Fund Value show the strongest positive association (0.97), followed by Monthly Contribution and Current Fund Value (0.81).	Contribution-related variables show the strongest relationships with financial outcomes in the dataset.	Maintaining regular contributions and building contribution continuity should be important considerations in pension planning.

8. Overall Analysis

The overall analysis brings together the major findings identified during the exploratory data analysis of the India Pension Scheme dataset.

While the previous sections examined individual variables, relationships, and patterns in detail, this section provides a consolidated interpretation of the most important findings from the univariate, bivariate, multivariate, and correlation analyses.

The overall analysis focuses on the demographic profile of pension account holders, employment and sector representation, pension scheme participation, investment preferences, account characteristics, contribution patterns, financial outcomes, and major relationships identified in the dataset.

The findings are descriptive and reflect patterns observed within the dataset. They should not be interpreted as evidence of causal relationships or as recommendations for selecting a particular pension scheme or investment option.

8.3 Suggestions

Based on the findings from the analysis, the following suggestions can be considered:

- **Start pension contributions early:** Earlier and longer participation may provide greater opportunity to build accumulated contributions and fund value over time.
- **Maintain regular contributions:** The strong association between contribution amounts and current fund value highlights the importance of consistent contributions.

- **Choose investment options carefully:** Participants should understand the asset allocation and risk characteristics of different investment choices before selecting an option.
- **Understand scheme-specific features:** Since financial outcomes differ across pension schemes in the dataset, participants should understand the contribution structure and benefits associated with their respective scheme.
- **Monitor pension accounts regularly:** Active monitoring can help participants identify inactive, dormant, or frozen account statuses and address them when necessary.
- **Plan according to individual financial capacity:** Differences in income and contribution levels suggest that pension planning should consider each participant's financial circumstances.

8.4 Implications for Different Stakeholders

1. Pension Account Holders

- Start pension contributions early to allow a longer period for fund accumulation.
- Maintain regular contributions to support long-term accumulation.
- Select investment choices based on individual financial circumstances and risk preferences.
- Understand the features and contribution structure of the selected pension scheme.
- Regularly monitor pension accounts and contribution records.

2. Pension Scheme Providers

- Improve awareness of scheme features and benefits among participants.
- Provide clear information about contribution requirements and investment options.
- Identify participant groups with lower contribution levels or account activity.
- Strengthen communication regarding long-term pension planning.
- Use participant data to improve targeted pension services and support.

3. Employers

- Encourage employees to participate in pension schemes from an early stage.
- Promote regular employee and employer contributions.
- Improve employee awareness of employer-supported pension contributions.
- Provide retirement-planning information to employees.

- Encourage long-term participation rather than irregular contributions.

4. Government and Policymakers

- Increase pension awareness across different employment and income groups.
- Encourage early and regular pension participation.
- Strengthen pension access among underrepresented employment sectors.
- Promote financial and pension literacy programmes.
- Use data-driven analysis to identify groups requiring greater pension support.

5. Pension Fund Managers and Financial Educators

- Explain asset allocation and investment risk clearly to participants.
- Provide guidance on differences between conservative, moderate, and aggressive investment choices.
- Improve financial education around long-term contribution planning.
- Help participants understand how contribution behaviour relates to accumulated fund value.
- Provide simple and accessible information to support informed pension decisions.

Summary

Pension Preparedness and Financial Security: An Analysis of Indian Pension Subscriber Data examines the demographic, employment, pension scheme, investment, contribution, and financial characteristics of Indian pension subscribers. The analysis uses descriptive statistics and a range of univariate, bivariate, multivariate, and correlation analyses to identify important patterns in pension participation and financial outcomes. The findings show that the dataset is largely represented by middle-aged subscribers, with the **All Citizen Model** being the most represented sector and **NPS - All Citizen** the most common pension scheme, while investment preferences are relatively balanced. Financial outcomes vary considerably across participants and pension schemes, with **NPS - Government and UPS** showing higher median fund values and expected monthly pensions in this dataset. Contribution-related variables show the strongest financial relationships, particularly the association between **total contribution and current fund value**. Overall, the project highlights the importance of early and consistent contributions, informed investment choices, scheme awareness, and regular account monitoring in supporting long-term pension preparedness and financial security.